

A PROJECT REPORT
ON
HOSPITAL MANAGEMENT SYSTEM

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE
BACHELOR OF COMPUTER APPLICATION(BCA)

SUBMITTED BY
SHREYA SAWANT

NIDHI VYAS

GULNAR PATEL

SALONI JHA

SANGAM JHA

UNDER THE GUIDANCE OF

MRS. MANASI GHULE

MS. VAIDEHI RAVAL



JANSEVA SAMITI SANCHALIT

SHRI. M. D. SHAH MAHILA COLLEGE OF ARTS AND COMMERCE (Autonomous)
(AFFILIATED COLLEGE TO S.N.D.T WOMEN'S UNIVERSITY)

B.J. PATEL ROAD MALAD (WEST) MUMBAI-400064
2025-2026

A PROJECT REPORT
ON
HOSPITAL MANAGGEMENT SYSTEM

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE
BACHELOR OF COMPUTER APPLICATION(BCA)

SUBMITTED BY
SHREYA SAWANT

NIDHI VYAS

GULNAR PATEL

SALONI JHA

SANGAM JHA

UNDER THE GUIDANCE OF

MRS. MANASI GHULE

MS. VAIDEHI RAVAL



JANSEVA SAMITI SANCHALIT

SHRI. M. D. SHAH MAHILA COLLEGE OF ARTS AND COMMERCE (Autonomous)
(AFFILIATED COLLEGE TO S.N.D.T WOMEN'S UNIVERSITY)

B.J. PATEL ROAD MALAD (WEST) MUMBAI-400064

2025-2026

DECLARATION

We Ms.Shreya Sawant, Ms.Nidhi Vyas, Ms.Gulnar Patel Ms. Saloni Jha, Ms.Sangam Jha, students of Bachelor of Computer Applications (BCA) Semester VI (2025-2026), hereby declare that the Project Report submitted to Shri M. D. Shah Mahila College has been entirely developed by us to fulfill the requirements of the final year project.

We affirm that this project has been completed within the specified time frame. The information provided by us is true and original to the best of our knowledge.

Shreya Sawant - _____

Nidhi Vyas - _____

Gulnar Patel - _____

Saloni Jha - _____

Sangam Jha - _____

ACKNOWLEDGEMENT

We undertook this project as a part of the B.C.A. curriculum, assigned to us by **[Local Clinic Name]**, with the objective of designing and developing an efficient Hospital Management System titled “**NEURAL MED.**” The journey of conceptualizing, designing, and implementing this project has been both intellectually stimulating and professionally enriching.

We express our sincere gratitude to **Shri M.D. Shah Mahila College** for providing a structured academic environment, essential resources, and continuous institutional support, all of which played a vital role in the successful completion of this project.

We are deeply indebted to our respected faculty members, **Mrs. Manasi Ghule, Ms. Vaidehi Raval, and Ms. Sakshi Patil**, whose technical guidance, critical insights, and unwavering encouragement proved indispensable throughout the development process. Their mentorship significantly contributed to strengthening the quality, functionality, and reliability of our system.

We also extend our respectful thanks to **Principal Dr. Deepa Sharma** and the College Management for fostering a culture of academic excellence and for granting us access to the facilities required to transform our ideas into a functional software solution.

Finally, we acknowledge our friends and peers for their constructive feedback, motivation, and collaborative spirit, which helped us refine our work and maintain enthusiasm until completion.

We are truly grateful to everyone who contributed, directly or indirectly, to the success of this project. Their support has left an enduring impact on our academic and professional development.

College Certificate:

Client Certificate:

INDEX

SR NO.	CONTENT	PAGE NO.
1.	Synopsis	
2.	Software development life cycle (SDLC)	
3.	System Planning: 3.1 Gantt chart	
4.	System Analysis: 4.1 Requirement gathering 4.2 Module (waterfall) 4.3 Feasibility study 4.4 Tool & technology 4.5 Software specification	
5.	System Design: 5.1 Decomposition Diagram 5.2 Flow Chart 5.3 Entity-Relationship (ER) 5.4 Data Schema 5.5 Screenshots of UI	
6.	System Coding: 6.1 Code	

	6.2 Validation Screenshots	
7.	System Testing: 7.1 Quality Assurance 7.2 Screenshot of Testing	
8.	Implementation: 8.1 User Guide 8.2 Installation	
9.	Conclusion	
10.	Future Enhancement	
11.	References and Bibliography	

PROJECT SYNOPSIS

Title: Hospital Management System

Class: TYBCA

Division: A and B

1.1 Project Name

Neural Med:

An AI-Driven Hospital Management System for Predictive and Intelligent Healthcare

1.2 Member Name

1. Shreya Sawant – TYBCA-A
2. Nidhi Vyas –TYBCA - A
3. Saloni Jha – TYBCA - A
4. Sangam Jha - TYBCA -A
5. Gulnar Patel -TYBCA-B

2. Project Introduction

The healthcare sector in 2026 is evolving rapidly with the integration of Artificial Intelligence and predictive analytics. Traditional hospital management systems focus mainly on storing patient records and administrative data, which limits their operational efficiency.

NeuralMed is a comprehensive Hospital Management System (HMS) designed to digitize and optimize daily medical workflows. Unlike traditional systems that serve as mere databases, NeuralMed utilizes Natural Language Processing (NLP) to provide an intelligent "Triage" layer, ensuring patients are categorized by medical urgency.

2.1 Client Details

Client Name: Multispecialty Hospital / Healthcare Organization

Client Type: Healthcare Service Provider

End Users: Patients, Doctors, Receptionists, Hospital Administrators

Industry: HealthTech

2.2 Objective and Purpose of Project

The primary objective of this project is to **design and develop an advanced AI-enabled Hospital Management System** that significantly improves overall hospital efficiency, enhances the quality of patient care, and supports **accurate clinical and operational decision-making**. The proposed system aims to transform conventional hospital workflows by integrating **Artificial Intelligence and data-driven automation** into core healthcare processes.

The purpose of the system is to **automate routine and repetitive hospital operations** such as patient registration, appointment scheduling, medical record management, and inventory tracking, thereby reducing manual workload, human errors, and administrative delays. By minimizing dependency on paper-based and manual processes, the system enables healthcare professionals to focus more on patient treatment rather than administrative tasks.

The system also emphasizes **secure and intelligent Electronic Health Record (EHR) management**, enabling structured storage and quick retrieval of medical histories to support faster and more informed clinical decisions.

2.3 Comparison of Existing System and Proposed System

Feature	Traditional Hospital Systems	NeuralMed (Proposed System)
Patient Triage	Manual assessment by a nurse; subjective and prone to delays.	AI-Powered Triage: NLP-based symptom analysis for instant, objective categorization.
Urgency Handling	First-come, first-served or manual urgency marking.	Intelligent Prioritization: Assigns scores (Urgent/Routine) to ensure critical cases are seen first.
Record Keeping	Paper-based files or basic digital folders; difficult to search.	Centralized EHR: Secure, searchable digital vault for records and prescriptions.
Prescription Process	Handwritten or manual entry of every detail.	One-Click Prescribe: Auto-fills Patient ID/Name, reducing documentation time for doctors.
Data Accessibility	Limited to hospital premises or specific desktops.	Web-Based Access: Real-time data availability for Doctors and Patients from anywhere.
Human Error	High risk of misplaced records or billing inaccuracies.	Automated Validation: Integrated billing and digital records minimize manual errors.
Security	Basic password protection or physical locks.	RBAC: Role-Based Access Control ensures users see only what they are authorized to see.

2.4 Scope and Features (Module Details)

1. Smart Patient & Triage Module:

- Patient registration and login
- AI-based symptom categorization

2. Intelligent Appointment & Scheduling Module:

- Doctor calendar view

3. Admin Dashboard & Analytics Module:

- Hospital performance visualization
- Role-Based Access Control (RBAC)

3. Technology Specification

3.1 Front-End Technology

- **React.js** (for a dynamic, single-page interface),
- HTML5, CSS3,
- JavaScript (ES6+).

3.2 Back-End Technology

- **Python** with the **Flask** framework (providing a lightweight yet powerful RESTful API).

3.3 Artificial Intelligence

- **Natural Language Processing (NLP)** libraries for symptom analysis and triage classification.

3.4 Database

- MySQL Workbench 8.0 CE

3.5 Development Tools

- VS Code, Git/GitHub for version control, and Postman for API testing.

4. System Requirements

Operating System: Windows 10 or above / Minimum 8 GB

Storage: Minimum 10 GB free space

Browser: Google Chrome / Firefox

5. Beneficiaries

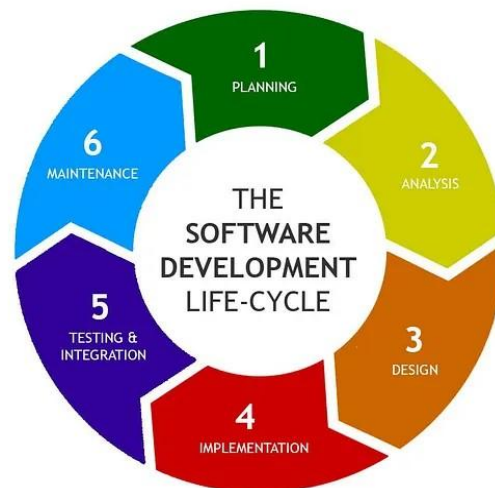
Patients receive faster and smarter healthcare services through automated processes such as AI-based triage, intelligent appointment scheduling, and quick access to medical records, which significantly reduce delays and improve overall patient experience.

Doctors benefit from the system by saving valuable time using AI-assisted documentation and automated medical report summarization, allowing them to focus more on diagnosis and patient care rather than paperwork.

Hospital management achieves improved operational efficiency through real-time analytics, centralized data access, and predictive insights that support better decision-making. The system also helps reduce patient waiting time and hospital overcrowding by optimizing patient flow and appointment schedules.

SOFTWARE DEVELOPMENT LIFE CYCLE

The Software Development Life Cycle (SDLC) is a highly structured, multi-phase process essential for planning, designing, building, testing, and maintaining robust software systems.



It provides a systematic, step-by-step approach that ensures a final product—such as a **Hospital Management System**—meets specific user requirements while adhering to high-quality standards and cost-effectiveness. By organizing complex hospital operations like patient registration, medical records, and billing into a reliable digital framework, the SDLC minimizes risks and ensures the software is delivered within the designated time and budget. The cycle typically begins with Planning to define the system's scope, followed by Defining requirements in a Software Requirement Specification (SRS). It then moves into Designing the architecture, Building the source code, and rigorous Testing to eliminate bugs. The process concludes with Deployment and ongoing maintenance to adapt to evolving hospital needs.

Phase 1: Planning

The Planning phase serves as the foundation of the entire project, where the core objectives and total scope of the Hospital Management System are meticulously identified. During this stage, project managers and analysts engage in extensive information gathering by interviewing hospital administrators, doctors, nurses, and receptionists to understand the daily pain points of the facility. This phase involves defining the functional requirements—such as how the system will manage patient admissions, discharges, and appointment scheduling—alongside non-functional requirements like system speed, usability, and data security. By establishing these goals early, the team can determine the project's feasibility and ensure it remains within the given time and budget.

Phase 2: Defining

Once the planning is complete, the project moves into the Defining phase, where all identified needs are translated into a formal document known as the Software Requirement Specification (SRS). This document acts as a definitive guide that outlines every feature the system must possess, the various user roles (such as "Admin" or "Doctor"), and the performance expectations. The SRS also details the system's constraints, ensuring there is a clear understanding of what the software can and cannot do. This document is then presented to the hospital stakeholders for review and approval; once signed off, it serves as the official agreement for the remainder of the development process.

Phase 3: Designing

In the Designing phase, the theoretical requirements of the SRS are transformed into a technical blueprint. Architects and designers focus on the system architecture, which includes the logical structure of the database and the visual layouts of the user interface. For a Hospital Management System, this involves mapping out how different modules—such as the Admin, Doctor, Patient, and Receptionist portals—will interact with one another. Designers ensure that the flow of data between these modules is seamless, ensuring, for example, that a doctor can instantly see a patient's registration details entered by the receptionist.

Phase 4: Building

The Building phase is where the actual construction of the software occurs as developers write the source code based on the design specifications. Developers utilize a modern technology stack—often including Python for logic, HTML/CSS/JavaScript for the front-end interface, and MySQL for data storage—to create the system. During this phase, the system is broken down into individual modules; each component (like the billing module or the lab report module) is developed and tested separately before it is integrated into the larger, complete system. This modular approach allows for better organization and helps developers identify coding errors early in the process.

Phase 5: Testing

The Testing phase is a rigorous quality-assurance stage designed to identify, report, and fix any bugs or errors before the system is used in a live hospital setting. Testing is conducted at various levels, including Unit Testing (for individual components), Integration Testing (to see how modules work together), and System Testing (to check the entire product). In an HMS, it is critical to verify that patient records are accurate, billing calculations are precise, and that the login system is secure. This ensures that when the system finally goes live, it performs its functions correctly and reliably under pressure.

Phase 6: Deployment

After the system has been successfully verified, it enters the Deployment phase, where it is installed and launched within the hospital's actual IT environment. This stage is not just about installation; it involves training the hospital staff to use the new interface and providing them with complete documentation for future reference. Following the initial launch, the system enters a continuous cycle of Maintenance. During maintenance, developers work to fix any unexpected bugs, improve the system's performance, and add new features or updates as the hospital's operational needs evolve over time.

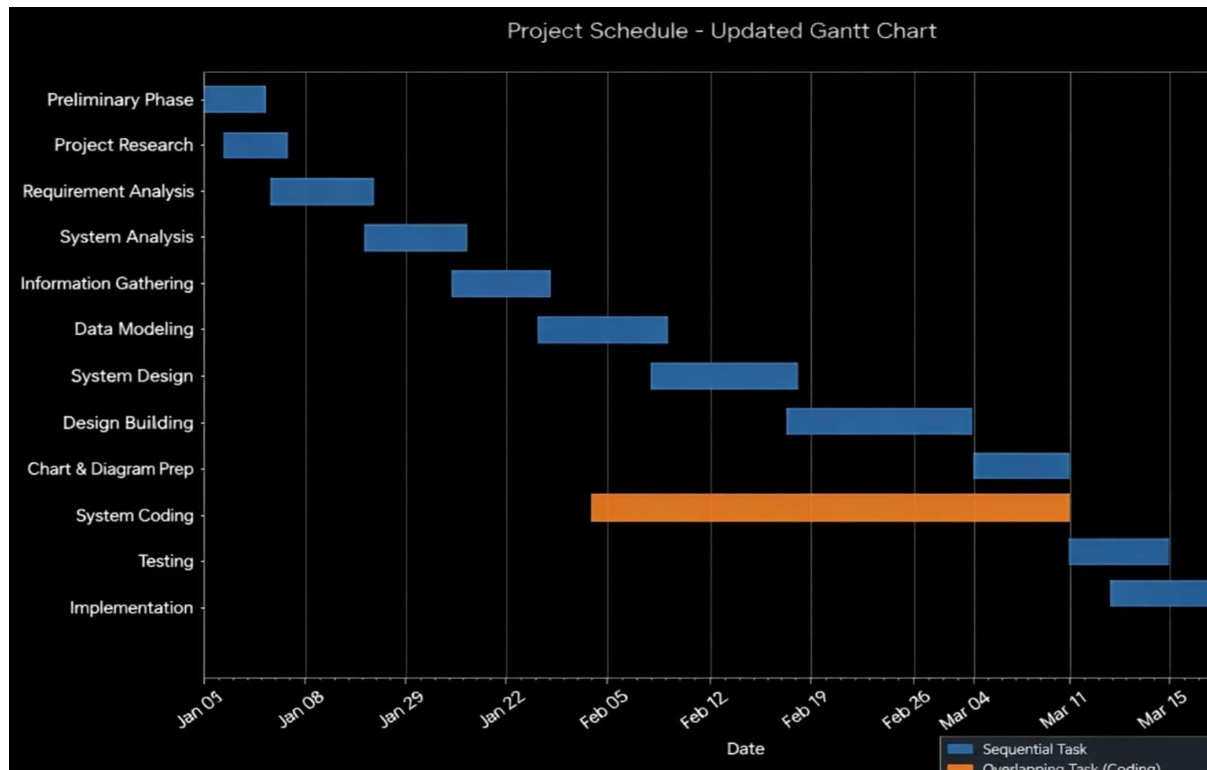
3. System Planning:

System Planning is the initial phase of software development in which the complete structure of the system is decided before actual coding begins. It involves identifying the problem, understanding user requirements, defining project objectives, selecting suitable technologies, and designing the overall architecture of the system. The main purpose of system planning is to ensure that the software is built in a structured, organized, and efficient manner. Proper system planning helps in reducing development errors, saving time, minimizing costs, and delivering a system that meets real-world user needs. system planning means thinking in advance about what the system should do, who will use it, and how it will be implemented.

In our project Neural Med – Hospital Management System, system planning was used to design a scalable and intelligent hospital solution instead of just creating a basic data storage system. First, we analyzed the limitations of existing hospital systems, such as long patient waiting times, manual appointment scheduling, paper-based records, and lack of predictive decision support. Based on this analysis, we defined clear objectives like improving hospital efficiency, reducing patient waiting time, automating routine tasks, and supporting doctors using Artificial Intelligence.

We then identified different user roles including patients, doctors, receptionists, and administrators to ensure role-based access control and data security. Suitable technologies were selected in advance, such as React JS for the frontend, Python (Flask) for the backend, and MYSQL for the database. The system was planned using a three-tier architecture consisting of a presentation layer, an application layer, and a data layer. Finally, the project was divided into five functional modules—Smart Patient & Triage, Appointment Scheduling, Electronic Health Records, and Admin Dashboard—to make development structured, manageable, and future-ready.

3.1 Gantt Chart:



Phase-Wise Work Description

1. Preliminary Phase

In this phase, our team finalized the project title “**Neural Med – Hospital Management System.**” We discussed the project idea with our project guide and clearly understood the expectations of the final-year project. A rough roadmap was prepared including timeline, tools to be used, and learning goals. We also decided the main objective of the project, which is to build a smart hospital system using Artificial Intelligence for better patient handling and decision support.

2. Project Research

During this phase, our team studied existing hospital management systems and modern AI-based healthcare platforms. We explored traditional hospital software as well as emerging health-tech applications. We referred to research papers, blogs, YouTube tutorials, and GitHub repositories to understand how hospitals manage patient data, appointments, and electronic health records. This research helped us identify the limitations of current systems and plan innovative AI features like symptom triage and report summarization.

3. Requirement Analysis

Here, our team identified all the functional and non-functional requirements of the system. Functional requirements included:

- Patient registration and login
- AI-based symptom triage
- Online appointment booking
- Doctor dashboard
- Electronic Health Records (EHR)
- Admin dashboard and analytics

Non-functional requirements included:

- Security through role-based access control
- Fast response time
- User-friendly interface
- Data privacy and reliability

We also defined different user roles such as **Admin, Doctor, Receptionist, and Patient** to ensure controlled access to sensitive information.

4. System Analysis

In this phase, our team closely studied how different users will interact with the Neural Med system. We analysed the complete journey of a patient starting from registration, entering symptoms, receiving an AI-generated urgency score, and booking an appointment. We also studied how doctors will log in, view patient details, access electronic health records, and use AI-generated summaries to support clinical decision-making.

For the admin role, we analysed how administrators will monitor system activity, manage users, view analytics, and control access permissions. We also carefully examined how data will move between the frontend interface, backend server, AI engine, and the database. This helped us design smooth data flow for storing and retrieving patient records, appointment details, and AI results. This phase ensured that the system logic is clear, efficient, and aligned with real hospital operations before starting the actual coding.

5. Information Gathering

In this phase, our team collected all the detailed information required for developing the Neural Med system. We studied real-world hospital workflows to understand how patient registration, consultations, and follow-up visits are handled in practice. We gathered sample patient records to understand what type of personal, medical, and appointment-related data needs to be stored in the system. We also collected standard appointment formats, doctor scheduling patterns, and medical report templates to design realistic and practical data structures. This helped us define proper input fields, validation rules, and report layouts.

Furthermore, our team researched different AI libraries and APIs that could be used for Natural Language Processing (NLP) and text summarization, such as Python-based NLP tools and LLM APIs. We compared their features, ease of integration, and performance to select the most suitable options for implementing AI-based symptom triage and report summarization. This phase provided a strong foundation for both the technical and functional development of the project.

6. Data Modeling

In this phase, our team designed the database structure for the project. We created and finalized tables such as:

- Patient
- Doctor
- Appointment
- Medical Records
- User Login

We defined primary keys, foreign keys, and relationships between tables. We also included AI-related fields like **Urgency Score** to store the result of the symptom triage. An ER Diagram was prepared to represent the database structure clearly.

7. System Design

This phase focused on designing the overall system architecture. Our team prepared:

- ER Diagram

- System Architecture Diagram
- Data Flow Diagram (DFD – Level 0)

We decided to follow a **3-tier architecture**:

- Frontend: React js
- Backend: Python (Flask API)
- Database: SQL (MySQL / SQLite)

We also designed UI wireframes for the homepage, login page, patient portal, and doctor dashboard.

8. Design Building

In this phase, our team started building the actual user interface.
We created:

- Homepage
- Login & Registration page
- Navigation bar
- Basic dashboard layout

Using, React js, we implemented responsive layouts and improved visual styling. Buttons, forms, and cards were designed to make the system easy to use and visually appealing.

9. Chart & Diagram Preparation

Here, our team prepared all documentation-related diagrams required for the project report.
These included:

- ER Diagram
- Gantt Chart
- System Architecture Diagram
- Data Flow Diagram

These diagrams help explain system structure, data flow, and development planning clearly to evaluators.

10. System Coding

This was the main development phase of the project.
Our team implemented:

- Frontend logic using JavaScript
- Backend APIs using Python (Flask API)
- Database connectivity using SQL
- AI modules like symptom triage and report summarization
- Appointment scheduling logic
- Role-based access control (RBAC)

This phase included continuous coding, integration, and debugging to make all modules work together smoothly.

11. Testing

In this phase, our team tested each module individually and then tested the full system.
We checked:

- User login and registration
- AI symptom triage accuracy
- Appointment booking flow
- Data storage and retrieval
- Access control for different roles

Errors and bugs were identified and fixed to improve system stability and reliability.

12. Implementation

Here, our team integrated all modules into a single working system.

Final UI improvements were made.

The system was deployed locally (and made cloud-ready).

Final performance testing and user flow testing were completed to ensure the system works as expected.

4. System Analysis:

4.1 Requirement Gathering

Requirement gathering is a key phase in the development of Neural Med because it helps define what the system should do before starting design and coding. The main aim of this phase was to understand hospital workflows and identify real-world problems that can be solved using an AI-enabled system. To gather requirements, **we** studied existing hospital management systems, online healthcare platforms, and case studies. **we** focused on common challenges such as long patient waiting times, manual record keeping, difficulty in managing appointments, and slow access to patient medical history. **we** also explored how Artificial Intelligence can improve these areas through smart triage and automated summaries.

During this phase, **we** identified the main users of the system as Patients, Doctors, Receptionists, and Admins. For each user role, **we** listed the actions they should be able to perform. Patients should be able to register, log in, describe symptoms, and book appointments. Doctors should be able to view patient histories, see appointments, and access AI-generated summaries. Admins should be able to manage users and monitor system performance.

We then defined the functional requirements such as patient registration, AI symptom triage, appointment scheduling, electronic health records, and dashboards. Non-functional requirements like security, fast response time, user-friendly interface, and data privacy were also identified. All gathered requirements were documented clearly and used as the foundation for system design and development

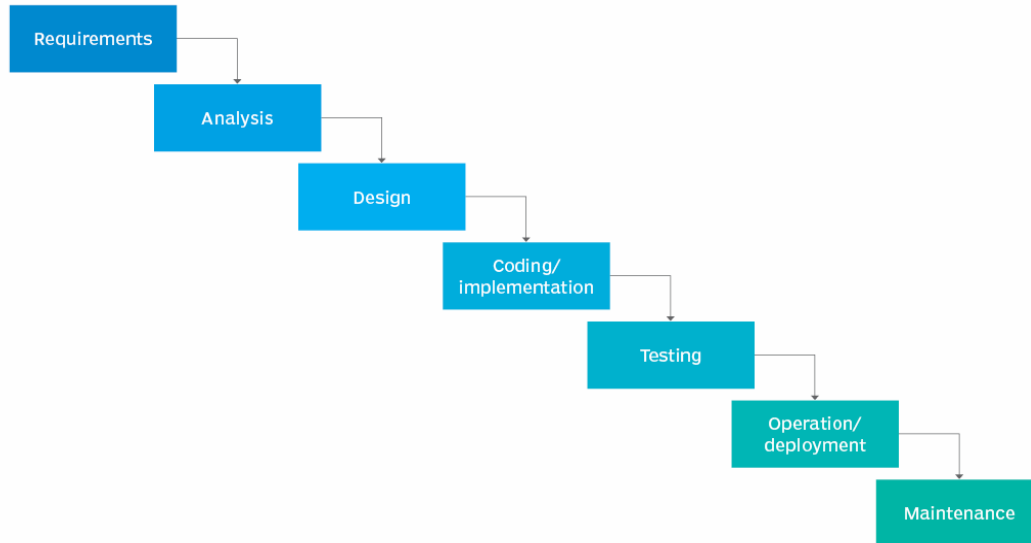
4.2 Module

In this project, the Waterfall Model was chosen as the software development methodology to ensure a structured, linear approach to developing the Movie and online stream. The Waterfall Model divides the project into distinct phases, where each phase must be completed before the next one begins. This approach was selected because the project's requirements were clearly defined from the outset, making it suitable for a sequential and predictable development process.

The waterfall model is a software development model used in the context of large, complex projects, typically in the field of information technology. It is characterized by a structured, sequential approach to project management and software development.

The waterfall model is useful in situations where the project requirements are well-defined and the project goals are clear. It is often used for large-scale projects with long timelines, where there is little room for error and the project stakeholders need to have a high level of confidence in the outcome.

Waterfall model



Phases of Waterfall Model

1. Requirements Analysis and Specification

This phase focuses on clearly understanding and documenting the customer's needs.

- **Requirement Gathering and Analysis:**
Customer requirements are collected and carefully examined to remove errors, confusion, and inconsistencies.
- **Requirement Specification:**
The approved requirements are documented in the Software Requirement Specification, which serves as a formal agreement between the customer and the development team.

2. Design

In this phase, the requirements from the Software Requirement Specification are converted into a system design that can be implemented in code.

- **High-Level Design (HLD):**
Defines the overall system architecture, major components, and their interactions.
- **Low-Level Design (LLD):**
Provides detailed design of each component, including logic and data flow, to guide developers during coding.
- All designs are documented in the Software Design Document

3. Development

This phase involves converting the design into actual working software.

- Developers write source code based on the design documents.
- Suitable programming languages and tools are used.
- Unit testing is performed to verify that each module works correctly on its own.

4. Testing and Deployment

This phase ensures that the integrated software functions correctly and is successfully delivered for real world use.

Testing: After unit testing, software modules are integrated incrementally and tested at each stage. Once all modules are successfully integrated, full system testing is performed to ensure the system functions as expected.

- **Alpha Testing:** Performed by the development team.
- **Beta Testing:** Performed by selected end users.
- **Acceptance Testing:** Performed by the customer to approve the software.

Deployment: After successful testing, the software is deployed to a live environment for end users. This phase includes environment setup, user training, and final checks to ensure smooth operation in real world conditions.

5. Maintenance

Maintenance ensures the software continues to function effectively after deployment.

- **Corrective Maintenance:** Fixes errors found after release.

- **Perfective Maintenance:** Enhances features based on user needs.
- **Adaptive Maintenance:** Updates software to work in new environments, such as new hardware or operating systems.

4.3 Feasibility Study:

Before developing the Hospital Management System (HMS), a feasibility study was conducted to evaluate whether the proposed system is practical, implementable, and beneficial for the hospital. The feasibility study helps in deciding whether the project should proceed further or not. For this project, the following three feasibility aspects were analysed:

1. Technical Feasibility

Technical feasibility determines whether the required technology and technical resources are available to build and operate the system successfully.

The proposed Hospital Management System will be developed using **Python** for backend processing, **JavaScript** for frontend interaction, and **MySQL** for database management. These technologies are widely used, open-source, and well supported, making them suitable for long-term usage.

The hospital has access to standard computer systems and network infrastructure capable of running the HMS software without performance issues. The system architecture is designed in a modular and scalable way so that it can handle increasing numbers of patients, records, and users in the future.

Since the development team possesses sufficient knowledge of Python, JavaScript, and database management, and no specialized or costly hardware is required, the project is considered **technically feasible**.

2. System Feasibility

System feasibility evaluates whether the proposed system can function effectively within the hospital's existing working environment.

The Hospital Management System is designed to integrate smoothly with the hospital's current processes such as patient registration, appointment booking, billing, and report management. The system also supports future integration with laboratory systems and diagnostic equipment if required.

Existing patient records, whether paper-based or stored digitally, can be migrated into the MySQL database to maintain data continuity. The system includes user authentication and role-

based access control to ensure data security and privacy in compliance with healthcare regulations.

The hospital's network infrastructure is sufficient to support continuous availability of the system for departments such as OPD, reception, and emergency services. Therefore, the system is **system feasible**.

3. Functional Feasibility

Functional feasibility determines whether the proposed system meets the actual needs of hospital staff and management.

The HMS includes all essential modules such as **Admin, Doctor, Patient, and Receptionist**. Each module is designed to perform specific tasks like appointment scheduling, patient record management, billing, and report generation.

The user interface is kept simple and user-friendly so that doctors, nurses, and reception staff can use the system efficiently with minimal training. The appointment scheduling feature is expected to reduce patient waiting time and improve workflow efficiency.

The system automates manual tasks such as billing, report preparation, and patient data storage, thereby reducing paperwork and human errors. Hence, the proposed HMS is considered **functionally feasible**.

4.4 Tools And Technology:

The Hospital Management System (HMS) was developed using simple, reliable, and widely supported tools and technologies to ensure ease of development, maintenance, security, and future scalability. The selected tools help in building a web-based system that is efficient, user-friendly, and suitable for real-world hospital operations. Each technology was chosen based on its performance, compatibility, and ease of learning.

❖ Programming Languages

• Python

Python is used for backend development due to its simplicity, readability, and strong community support. It handles core business logic such as user authentication, appointment scheduling,

billing calculations, and interaction with the database. Python also provides excellent support for web frameworks, making it ideal for building scalable and secure web applications.

- **React.js**

React.js is used for frontend development to build a modern, interactive, and responsive user interface. It is a JavaScript library that allows the creation of reusable UI components, improving code maintainability and development efficiency. React's component-based architecture enables efficient state management, dynamic data rendering, and seamless navigation without full page reloads. This enhances the overall user experience by making the application faster, more responsive, and easier to use.

❖ **Frontend Technologies**

- **Javascript**

JavaScript is a high-level, interpreted programming language used to create dynamic and interactive web applications.

JavaScript is used to handle user interactions, manage frontend functionality, and communicate with backend APIs. It helps in improving the responsiveness and usability of the web application by allowing real-time updates without reloading the entire webpage.

- **CSS**

CSS (Cascading Style Sheets) is used to style the user interface and enhance the visual appearance of the system. It helps in creating a clean layout, consistent colours, proper spacing, and responsive design, ensuring that the application looks professional and easy to use.

- **React.js**

React.js is used to enhance the user experience by building dynamic and interactive user interfaces. It enables efficient client-side validation, ensuring user input is verified before data is processed, which reduces errors and improves system reliability. React.js also allows real-time updates of content—such as displaying appointment details, form validation messages, and dashboard data—without reloading the page, resulting in a smooth and responsive application.

❖ Backend Technology

• Flask / (Python Framework)

A lightweight Python web framework is used to develop the backend of the HMS. It handles HTTP requests and responses between the frontend and the database. The framework manages functionalities such as user login, patient registration, appointment scheduling, medical record storage, and billing operations.

Flask / supports RESTful APIs, making it easy to connect the frontend with the backend. It also provides flexibility to add new features and integrate third-party services in the future. Its lightweight nature ensures fast performance and easier deployment.

❖ Database Technology

• MySQL

My SQL is used as the relational database management system (RDBMS) to store and manage hospital data securely. It stores structured data such as patient records, doctor details, appointment schedules, billing information, and user login credentials.

MySQL supports data integrity, indexing, and fast query processing, which helps in retrieving patient and appointment information quickly. It also allows backup and recovery of data, ensuring that critical hospital records are protected from data loss.

❖ Development Tools

• Visual Studio Code (VS Code)

Visual Studio Code is used as the primary code editor for writing, editing, and debugging the source code. It provides features such as syntax highlighting, auto-completion, extensions, and integrated terminal support, which improve developer productivity and code quality.

• Postman

Postman is used to test backend APIs and verify request–response behavior. It helps in checking whether API endpoints for login, appointment booking, patient registration, and billing are working correctly. Postman is useful for debugging errors and ensuring smooth communication between frontend and backend.

❖ Operating System

• Windows

Windows is used as the development and testing environment for the project. It provides a stable platform for running development tools such as VS Code, Python, MySQL server, and web browsers. Windows also supports easy installation and configuration of required software, making development and debugging more convenient.

4.5 Software Specification

The software specification defines the technical and functional details of the proposed Hospital Management System. It describes the software environment, system architecture, functional modules, and performance expectations of the system.

❓ Purpose of the System

The purpose of the Hospital Management System is to automate and manage core hospital operations such as patient registration, appointment scheduling, doctor consultations, billing, and medical record storage. The system aims to reduce paperwork, improve efficiency, and provide quick access to patient information for hospital staff.

❓ Scope of the System

The Hospital Management System will provide a centralized platform for managing hospital data and operations. The system will support multiple user roles and enable smooth interaction between patients, doctors, and administrative staff.

In Scope:

- Patient registration and profile management
- Appointment scheduling and tracking
- Doctor consultation records
- Billing and invoice generation
- Medical report storage
- User authentication and role-based access

Out of Scope (for current version):

- Integration with real-time medical devices
- Online payment gateway
- Mobile application version

- Telemedicine/video consultation

❓ User Classes and Characteristics

User Type	Description
Admin	Manages users, doctors, departments, and system settings
Doctor	Views appointments, updates patient records, prescribes treatment
Receptionist	Registers patients, schedules appointments, generates bills
Patient	Views profile, appointments, and medical reports

❓ Operating Environment.

The Hospital Management System will operate in the following environment:

- **Frontend:** JavaScript, CSS, React js
- **Backend:** Python version 3.11 (Flask)
- **Database:** MySQL
- **Operating System:** Windows
- **Browser:** Google Chrome, Microsoft Edge,

❓ System Architecture

The system follows a **three-tier architecture**:

1. **Presentation Layer (Frontend):**
Handles user interaction and displays forms, dashboards, and reports.
2. **Application Layer (Backend):**
Implements business logic, processes requests, and communicates with the database.
3. **Data Layer (Database):**
Stores patient records, appointments, billing information, and user credentials.

❓ 6. Functional Requirements

1. The system shall allow patients to register and update personal information.
2. The system shall allow receptionists to schedule and manage appointments.
3. The system shall allow doctors to view assigned appointments and update medical records.
4. The system shall store and retrieve medical reports securely.
5. The system shall provide login authentication and role-based access control.

❓ Non-Functional Requirements

- **Performance:**
The system should respond to user requests within 2–3 seconds.
- **Scalability:**
The system should support an increasing number of users and patient records.
- **Security:**
The system should use encrypted passwords and restrict access based on user roles.
- **Reliability:**
The system should be available 24/7 with minimal downtime.
- **Usability:**
The system should be user-friendly and require minimal training.

- **Hardware Requirements**

Component	Minimum Requirement
Processor	Intel i3 or above
RAM	4 GB (8 GB recommended)
Storage	100 GB free disk space
Network	Broadband Internet
Display	1024 × 768 resolution

❓ Software Requirements

Software Component	Requirement
Operating System	Windows 10 / Linux
Database Server	MySQL 8.0
Programming Language	Python 3.11, Flask React js
Browser	Chrome, Firefox, Edge
IDE	VS Code

Installation Steps

1. Flask Installation (Backend)

Flask is a lightweight Python web framework used for backend development.

****Step 1: Install Python****

- * Download and install **Python 3.9 or above** from the official Python website.
- * During installation, ensure that **“Add Python to PATH”** is checked.

****Step 2: Verify Python Installation****

```
python --version
```

or

```
python3 --version
```

****Step 3: Create a Virtual Environment (Optional but Recommended)****

```
python -m venv venv
```

****Step 4: Activate the Virtual Environment****

- * On Windows:

```
venv\Scripts\activate
```

- * On Linux:

```
source venv/bin/activate
```

****Step 5: Install Flask****

```
pip install flask
```

****Step 6: Verify Flask Installation****

```
flask --version
```

Flask is now successfully installed and ready for backend development.

2. React.js Installation (Frontend)

React.js is used for building the frontend user interface.

****Step 1: Install Node.js and npm****

* Download and install ****Node.js (LTS version)**** from the official Node.js website.

* npm (Node Package Manager) is installed automatically with Node.js.

****Step 2: Verify Node.js and npm Installation****

```
node --version
```

```
npm --version
```

****Step 3: Create a New React Application****

```
npx create-react-app frontend
```

****Step 4: Navigate to the React Project Directory****

```
cd frontend
```

****Step 5: Start the React Development Server****

```
npm start
```

The React application will run on:

“ <http://localhost:5000> ”

React.js is now successfully installed and configured for frontend development.

Constraints

- The system is designed for web-based usage only.
- Real-time device integration is not supported in the current version.
- Internet connectivity is required for multi-user access.
- Advanced analytics and AI modules may be added in future versions.

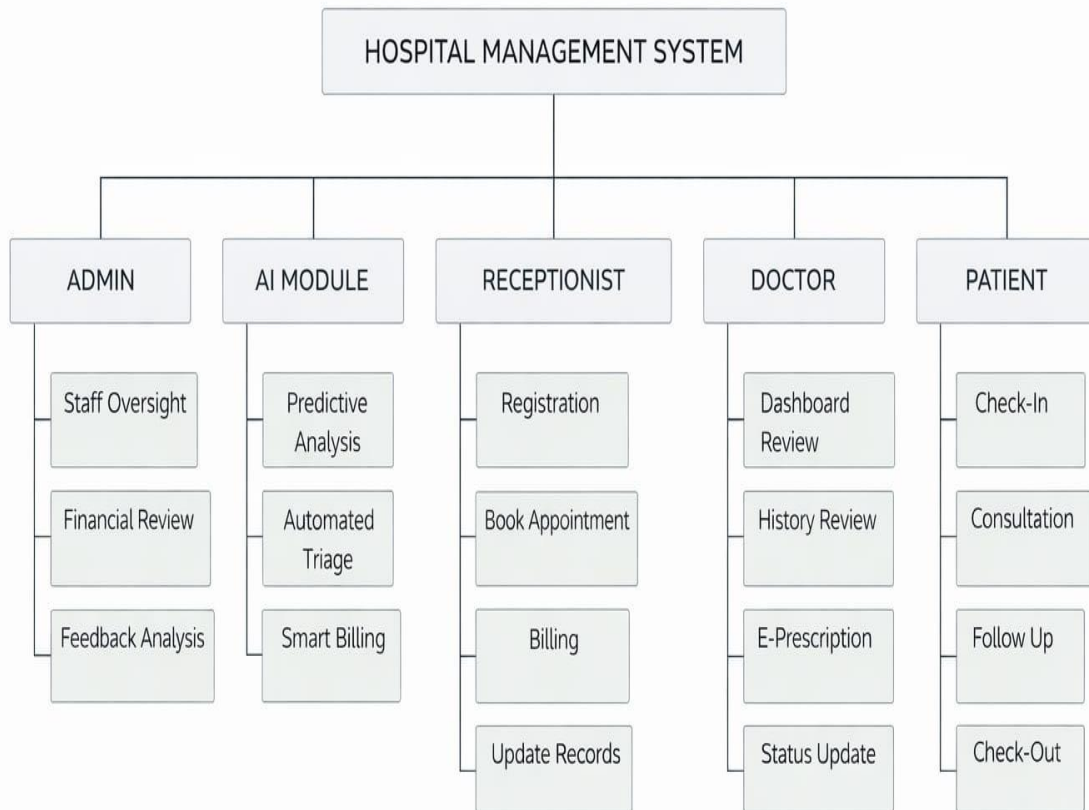
Assumptions and Dependencies

- Users have basic computer knowledge.
- Hospital staff will provide accurate data entry.
- System maintenance will be performed periodically.
- Future upgrades may require additional hardware resources

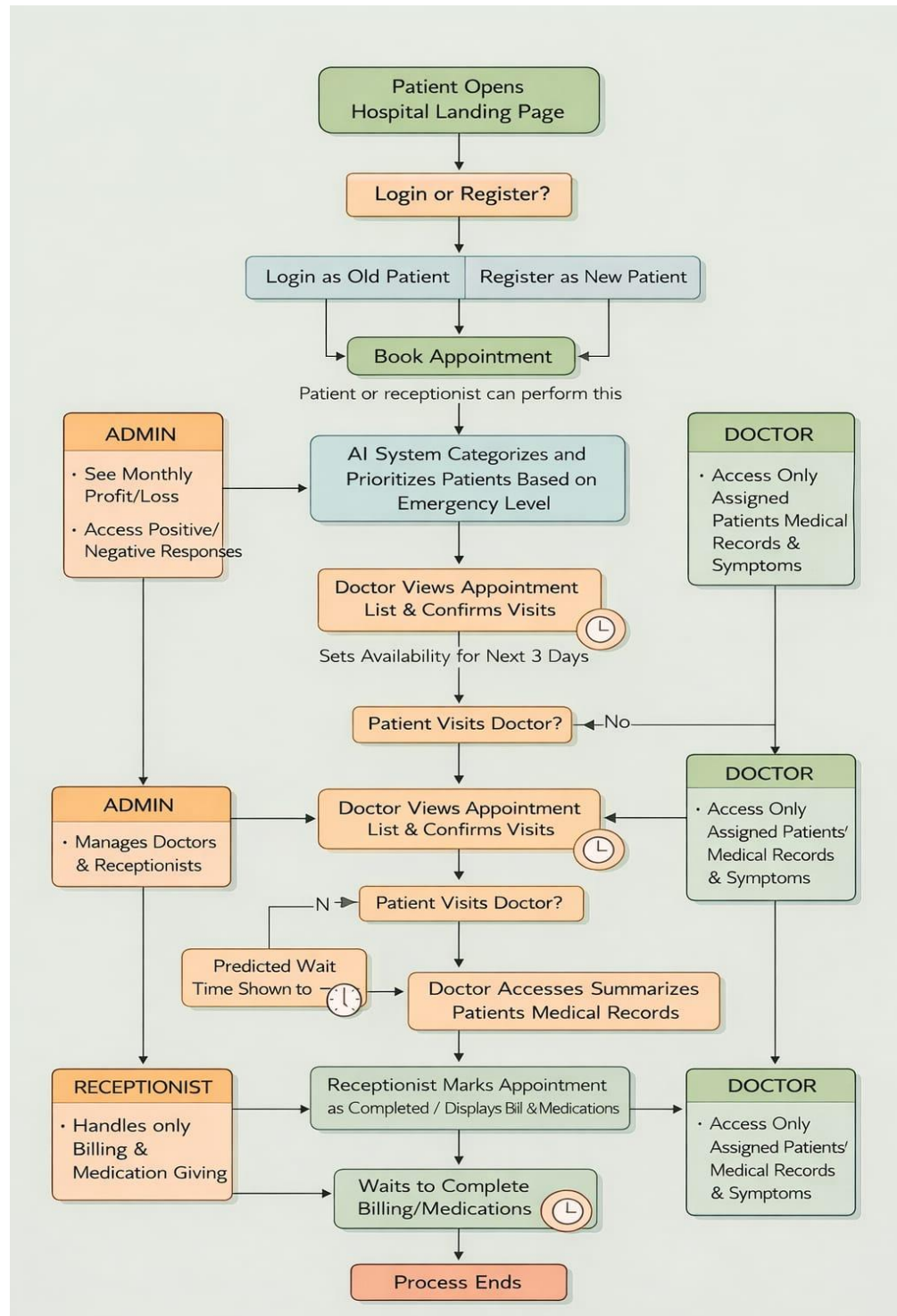
5 . System Design

- The System Design of this project serves as a comprehensive architectural blueprint, focusing on the structured flow of data between the various components of the application. It outlines the essential processes for designing the architecture, interfaces, and data models required to meet specific end-user requirements.
- By managing the critical interactions between the front-end user interface, the back-end logic, and the centralized database, the design ensures a seamless and efficient experience for administrators, doctors, and patients alike. Through the use of modular components and Entity-Relationship (ER) modeling, the system design facilitates a secure environment with role-based access control and high data integrity.
- This structured approach not only optimizes the current clinical and administrative workflows but also provides a scalable foundation for future enhancements, ensuring that the final application is both robust and highly responsive to real-world healthcare demands.

5.1 Decomposition Diagram(Tree Diagram)

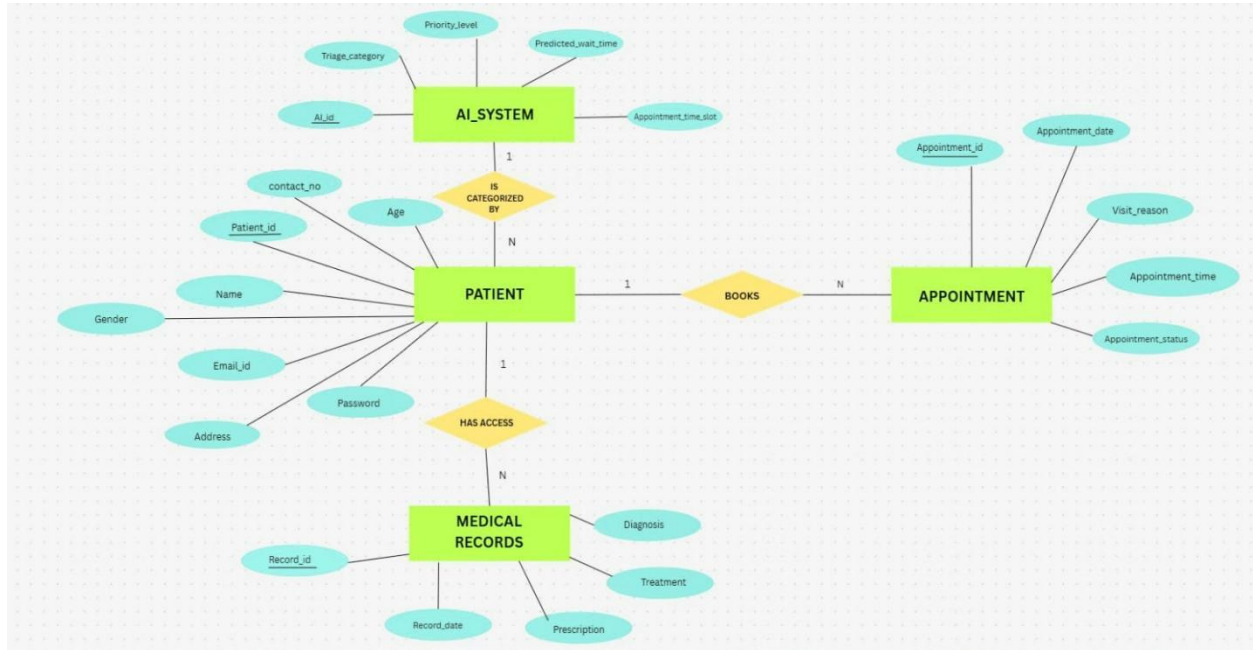


5.2Flow Chart:

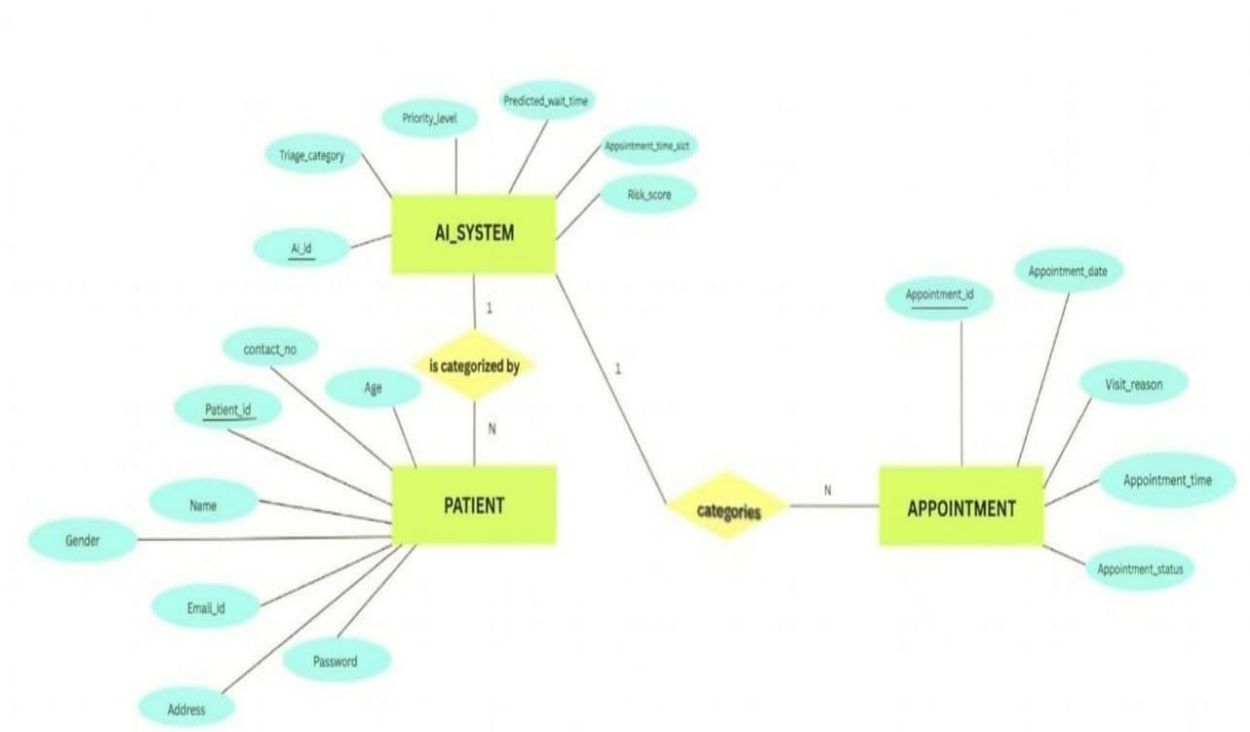


5.3 Entity- Relationship (ER Diagram):

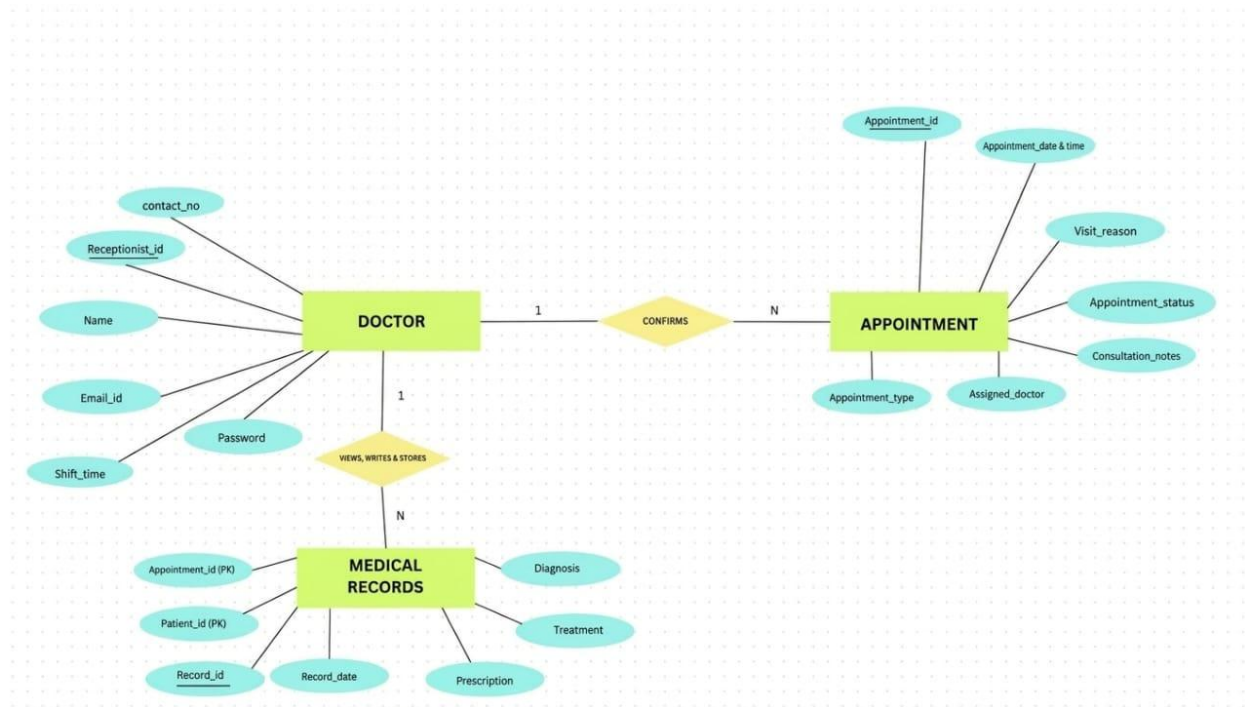
Patient's Module ER Diagram:



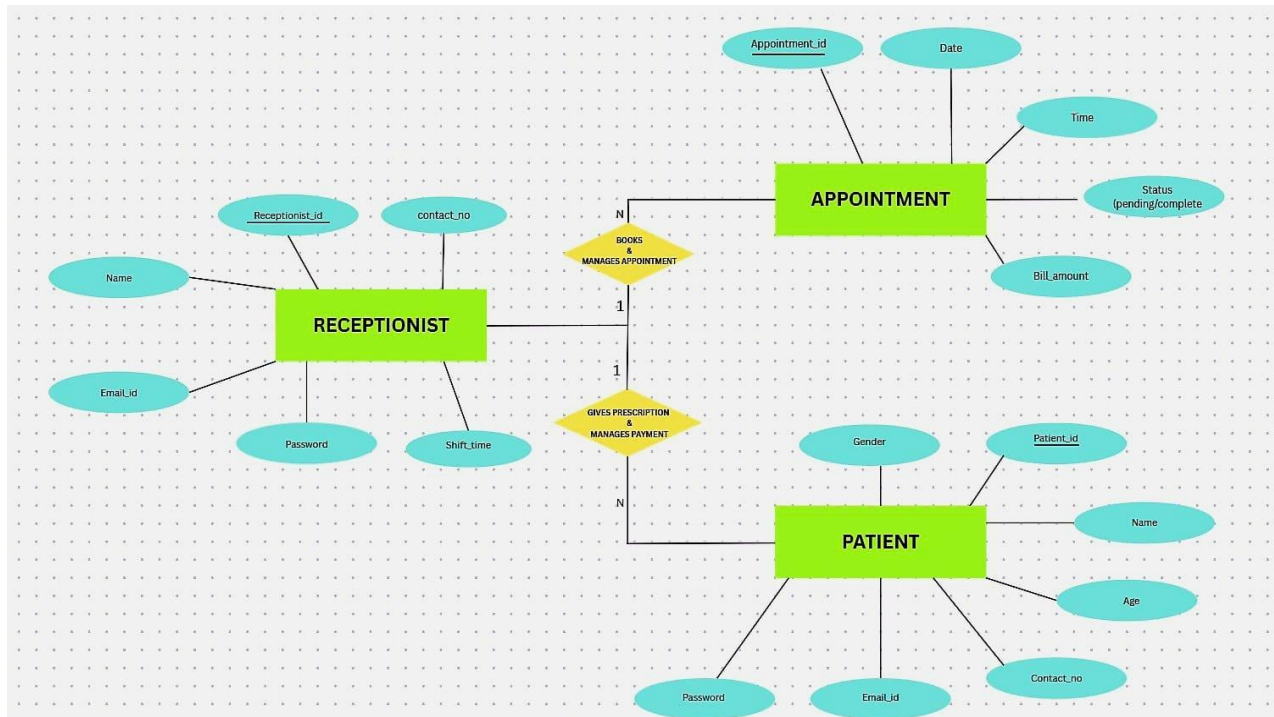
Patient's AI Module:



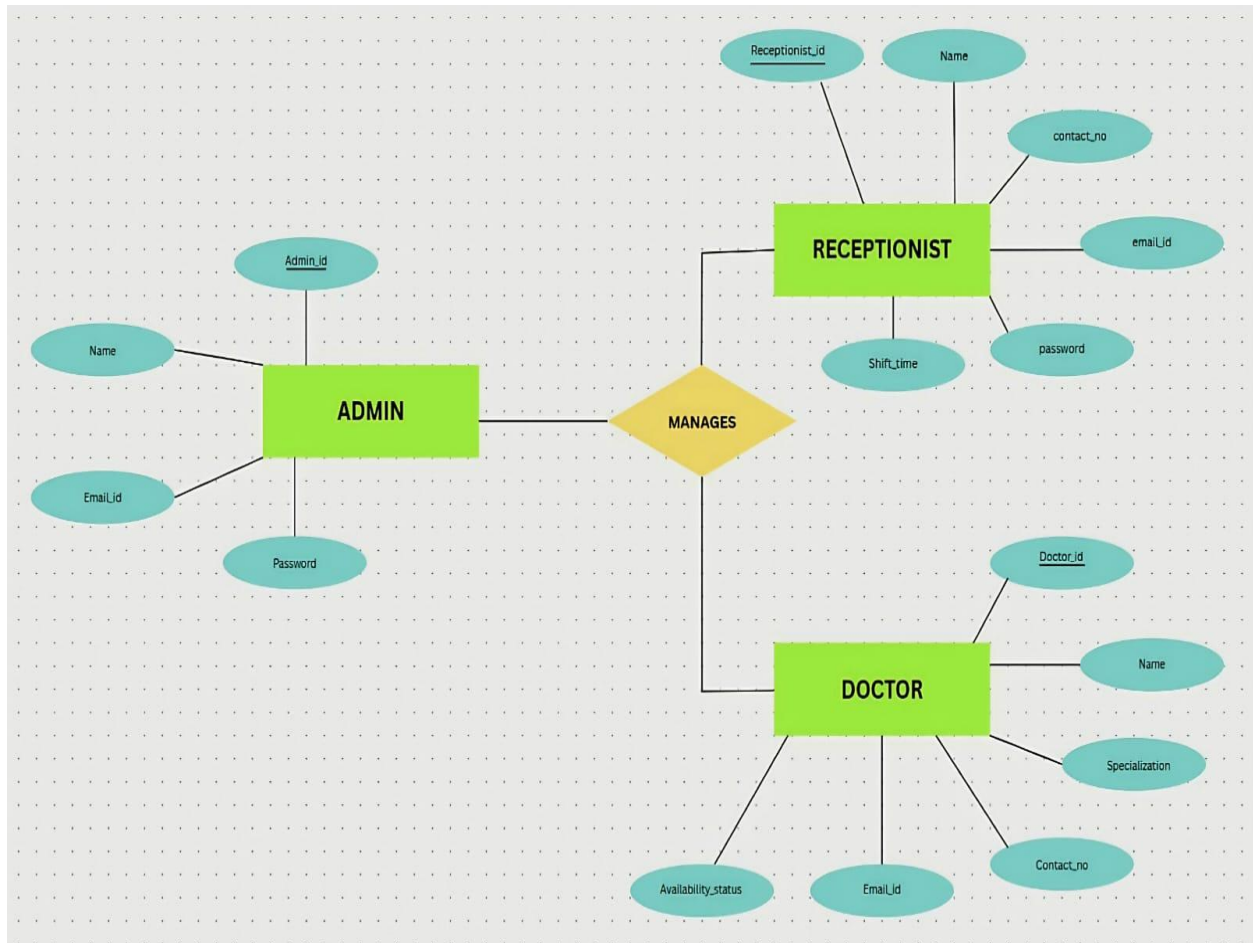
Doctor's Module ER Diagram:



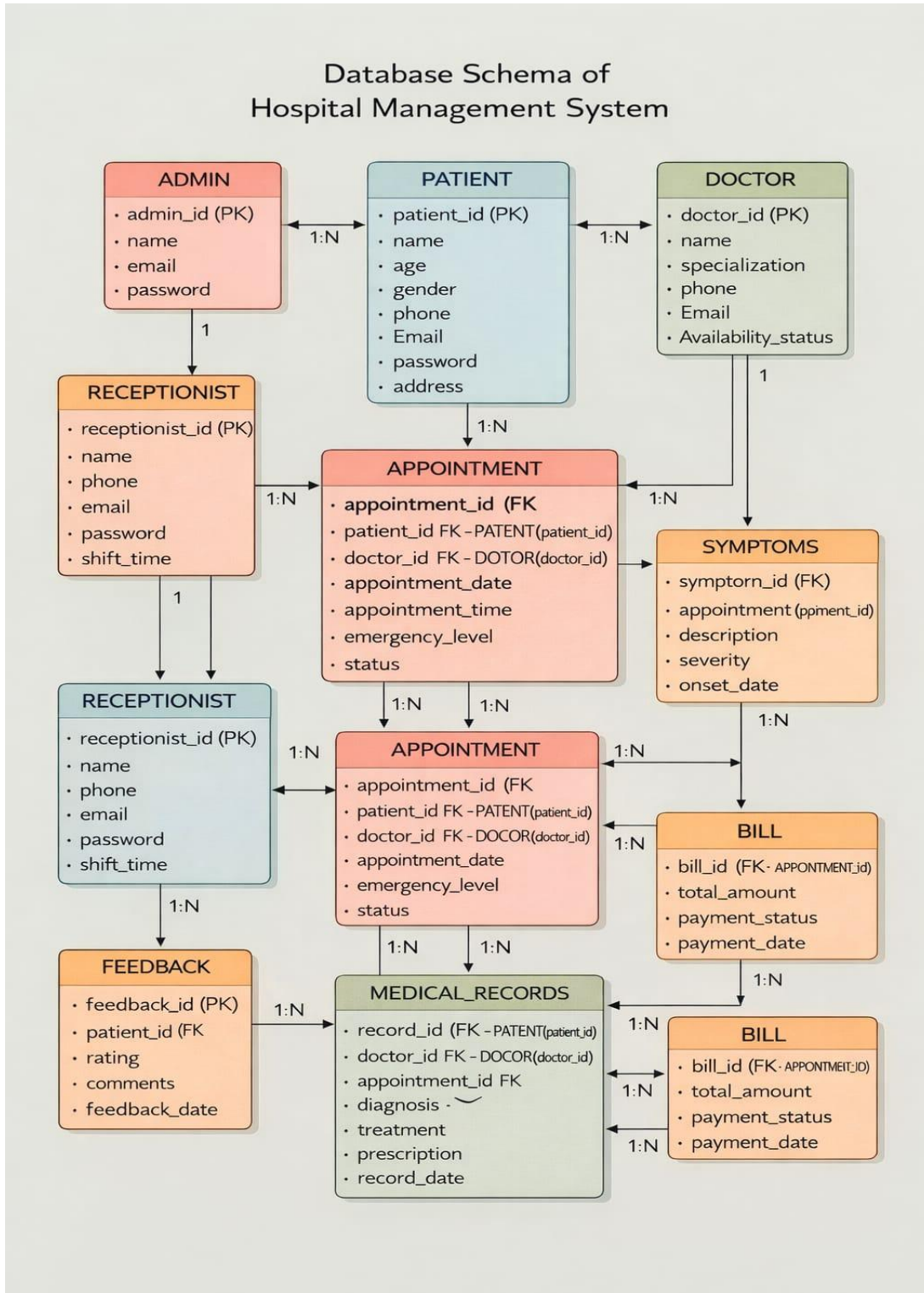
Receptionist Module ER Diagram:



Admin Module ER Diagram:

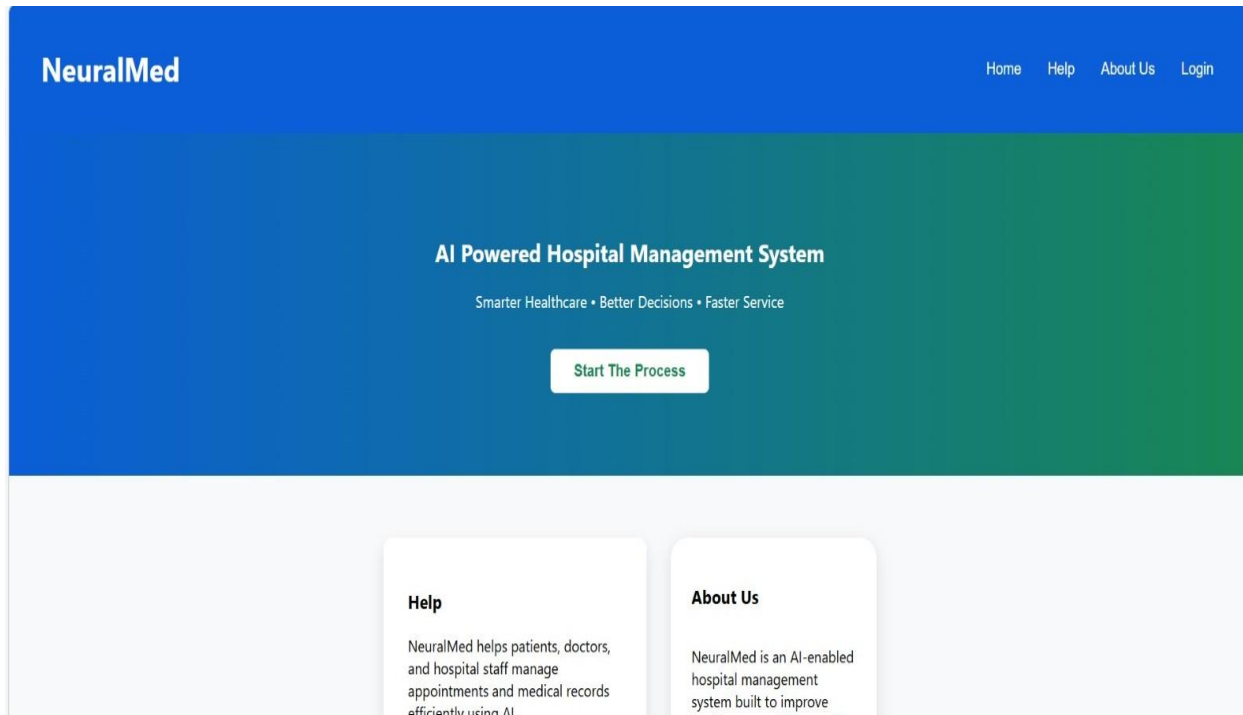


5.4 Database Schema

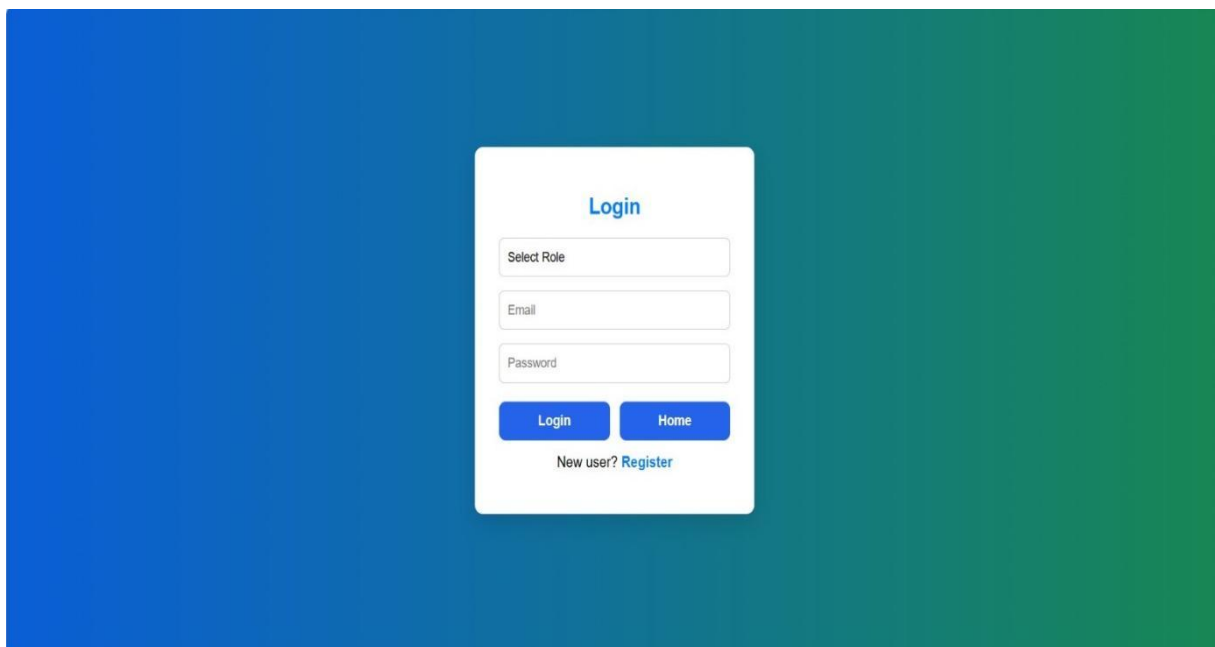


5.5 UI Screen shorts

1. Neural Med:-



2. Login page:-



3. Registration page:

The screenshot shows a web browser window with the address bar displaying 'localhost:5173/register'. The page has a blue header and a green sidebar. The main content area contains a registration form with the following fields:

- Select Role
- First Name
- Last Name
- Email ID
- Contact Number
- Address
- Select Gender
- dd - mm - yyyy
- Password
- Confirm Password

At the bottom of the form, there are two buttons: 'Register' and 'Home'. Below these buttons, there is a link: 'Already have an account? [Login](#)'.

4. Patient Dashboard:

The screenshot shows a Patient Dashboard with a blue header. The header contains the text 'Patient Dashboard' and a user profile 'nidhi vyas' with a 'Logout' button. Below the header, there are five buttons: 'Book Appointment', 'My Appointments', 'Prescriptions', 'Medical Records', and 'Update Profile'. Below these buttons, there is a user profile card for 'nidhi vyas' with a green profile picture and the following details:

- User_id: 40deb874-520a-4ce0-b590-53c32a131893
- Email: vyasnidhi314@gmail.com
- Contact: 9137478886
- Address: 227, Rd Number 14 Jawahar Nagar, Goregaon West
- Gender: female

5. Book Appointment:-

The screenshot shows a web browser at `localhost:5173/book-appointment`. The page has a blue header with the text "BOOK APPOINTMENT" on the left and a "BACK" button on the right. A modal dialog is displayed in the center, titled "localhost:5173 says". The modal contains the following information:

- Appointment booked successfully, Your appointment id is 59253c3e-97c3-4e2a-b3be-e43389e8c44f
- Dr. Shreya sawant (selected from a dropdown)
- 28-03-2026 (selected from a date picker)
- 05 (selected from a time dropdown), 00 (selected from a time dropdown), PM (selected from a time dropdown)
- routine checkup (entered in a text field)
- A "Confirm Appointment" button at the bottom.

The modal also has an "OK" button in the top right corner.

6. My Appointments:-

The screenshot shows a web browser at `localhost:5173/my-appointments`. The page has a blue header with the text "MY APPOINTMENTS" on the left and a "BACK" button on the right. The main content area displays a list of appointments in a light blue box. Each appointment card contains the following information:

- Appointment ID:** 59253c3e-97c3-4e2a-b3be-e43389e8c44f
- Doctor:** Shreya sawant
- Date:** 28-03-2026
- Time:** 05:00 PM
- Reason:** routine checkup

The second appointment card shows:

- Appointment ID:** 14b67b64-7fcd-4941-8513-26bc41eda168
- Doctor:** Shreya sawant
- Date:** 19-03-2026
- Time:** 06:04 PM
- Reason:** haiving milde fever

7. Prescriptions:-

PRESCRIPTIONS

BACK

Date: Sat, 07 Mar 2026 01:33:43 GMT

Doctor: Shreya sawant

Medicines: paracetamol 50mg

Instructions: take after meal

Date: Sat, 07 Mar 2026 00:46:26 GMT

Doctor: Shreya sawant

Medicines: dolo 500mg

Instructions: take three times a day

8. Medical Records:-

MEDICAL RECORDS

BACK

localhost:5173 says

Medical Record Uploaded Successfully!

OK

last vist

body pain and fever

Add Comment

Upload Medical Record

Choose File letter_head_1_1.pdf

Upload Record

Your Uploaded Records

9.Update Profile:-

UPDATE PROFILE

BACK

First Name

Karan

Last Name

Sawant

Date of Birth

01 - 01 - 1970

Gender

Male

Contact Number

1234567890

Address

Andheri

New Password

Confirm New Password

Save Changes

10.Doctor Dashboard:-

HMS

localhost:5173/doctor-dashboard

Gmail YouTube Maps Input and output d... TryHackMe | Defen... Learn from top co... Lenovo Support Lenovo McAfee

Doctor Dashboard

Shreya sawant Logout

Scheduled Appointments

Medical Records of Patients

Provide Prescription

Update Profile



Shreya sawant

User_id: 3facb72f-7e7d-4fde-98f5-b88cb91d9d0e

Email: shreyasawant205@gmail.com

Contact: 1232456789

Address: goregaon

Gender: female

11.Schedule Appointment:-

SCHEDULED APPOINTMENTS

Back

Appointment ID: 59253c3e-97c3-4e2a-b3be-e43389e8c44f

Patient Name: Karan Sawant

Date: 28-03-2026

Time: 05:00 PM

Reason: routine checkup

Status: Booked

Type: Routine

Provide Prescription

Appointment ID: 14b67b64-7fcd-4941-8513-26bc41eda168

Patient Name: Karan Sawant

Date: 19-03-2026

Time: 06:04 PM

Reason: haiving milde fever

Status: Booked

12. Medical Record of Patients :-

Patient Medical Records

Back

Patient Name: Karan Sawant

Title: last vist

Description: body pain and fever

Comment:

Uploaded Date: Sat Mar 07 2026

File Name: letter_head_1_1.pdf

Download File

Patient Name: Karan Sawant

Title: last vist report

Description: body pain and fever

Comment: lab reports

Uploaded Date: Sat Mar 07 2026

13. Provide Prescription:-

localhost:5173/provide-prescription

PROVIDE PRESCRIPTION

Prescription Saved Successfully!

OK

BACK

59253c3e-97c3-4e2a-b3be-e43389e8c44f

Karan Sewant

paracetamol 50mg

take after meal

Save Prescription

14.Update Profile:-

UPDATE PROFILE

BACK

First Name

Shreya

Last Name

sawant

Date of Birth

01 - 01 - 1970

Gender

Female

Contact Number

1232456789

Address

goregaon

New Password

....

Confirm New Password

Save Changes

15.Admin Dashboard:-

Admin Dashboard

Jagdish shahLogout

Doctors List

Receptionist List

Update Profile

ADMIN

Jagdish shah

User_id: 551c28bb-8b3e-4c8b-87f1-cff9b587331c

Email: jagdishvyas88@gmail.com

Contact: 9819129739

Address: 227, Rd Number 14 Jawahar Nagar, Goregaon West

Gender: male

16. Staff Schedule:-

HOSPITAL STAFF SCHEDULE			
Doctor	Specialisation	Availability Status	Shift Timing
Dr. Sharma	Family Medicine Physician	Available	09:00 AM - 12:00 PM
Dr. Mehta	Pediatrician	Unavailable	12:00 PM - 03:00 PM
Dr. Patil	Internist	Available	03:00 PM - 06:00 PM
Dr. Vyas	Obstetrician & Gynecologist	Available	06:00 PM - 09:00 PM

17.Receptionist Staff:-

RECEPTION STAFF SCHEDULE			Logout
Staff Name	Contact Number		Shift Timing
Ms. Pooja	9765341238		09:00 AM - 01:00 PM
Mrs. Chetana	7890654321		01:00 PM - 05:00 PM
Ms. Disha	8675430123		05:00 PM - 09:00 PM

18. Update Profile:-

Update Profile

Back

First Name

Last Name

Date of Birth

dd-mm-yyyy

Gender

Select Gender

Contact Number

3456796435

Address

19. Receptionist Dashboard:-

Receptionist Dashboard

Sneha kaurLogout

View Patients Prescription

Billing

Update Profile



Sneha kaur

User_id: f383e493-477e-49b1-a6be-de0aa4322c1b

Email: sneha@gmail.com

Contact: 2345674431

Address: panvel

Gender:

20. View Patient Prescription:-

Patient Prescriptions					BACK
Patient Name	Doctor	Medicines	Notes	Actions	
Rahul Sharma	Dr. Mehta	Paracetamol, Vitamin C	Take after meals	Prescribed	
Sneha Patil	Dr. Patil	Amoxicillin	Twice daily for 5 days	Prescribed	
Amit Joshi	Dr. Sharma	Cough Syrup	Before sleep	Prescribed	

21. Billing:-

HMS

localhost:5173/billing

GmailYouTubeMapsInput and output d...TryHackMe | Defen...Learn from top co...Lenovo SupportLenovoMcAfee

BACK

Appointment ID	Patient Name	Total Billing Amount	Mode of Payment
APT-1001	Rahul Patil	₹ 1200	Cash ☒
APT-1002	Sneha Mehta	₹ 850	Select Pay
APT-1003	Amit Sharma	₹ 1500	Select Pay

22. Update Profile:-

UPDATE PROFILE

BACK

First Name

Sneha

Last Name

kaur

Date of Birth

01-01-1970

Gender

Female

Contact Number

2345674431

Address

panvel

New Password

Confirm New Password

Save Changes

6. System Coding:-

6.1 Code:

1. Frontend code:

Login code:

```
import React, { useState } from "react";
import "./Login.css";
import { useNavigate } from "react-router-dom";
import axios from "axios";

const Login: React.FC = () => {
  const navigate = useNavigate();
  const [role, setRole] = useState<string>("");
  const [email, setEmail] = useState<string>("");
  const [password, setPassword] = useState<string>("");

  const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();

    if (!role || !email || !password) {
      alert("Please fill all fields");
      return;
    }

    axios.post("http://localhost:5000/api/auth/login", {
      role,
      email,
      password
    })
    .then(response => {
      if(response.data["response"]){
        let user = response.data["user"];
```

```

if(user["role"]?.toLowerCase() === role?.toLowerCase()){
  localStorage.setItem("isLoggedIn", "true");
  localStorage.setItem("userRole", role);
  localStorage.setItem("user", JSON.stringify(response.data["user"]));
  if (role === "patient"){
    navigate("/patient-dashboard");
  }
  else if (role === "doctor") navigate("/doctor-dashboard");
  else if (role === "admin") navigate("/admin-dashboard");
  else if (role === "receptionist") navigate("/receptionist-dashboard");
}
else{
  alert("Please select valid role!!")
}

}
else{
  alert("Invalid user!!")
}
})
.catch(error => {
  console.error(error);
  alert("Something went wrong, please try after sometime!!")
});
};

return (
  <div className="login-container">
    <div className="login-card">
      <h2>Login</h2>
      <form onSubmit={handleSubmit}>
        <select
value={role}
onChange={(e: React.ChangeEvent<HTMLSelectElement>) =>

```



```

    setRole(e.target.value)
  }
  >
    <option value="">Select Role</option>
    <option value="patient">Patient</option>
    <option value="doctor">Doctor</option>
    <option value="admin">Admin</option>
    <option value="receptionist">Receptionist</option>
  </select>
  <input
    type="text"
    placeholder="Email"
    value={email}
    onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
setEmail(e.target.value)
  }
  />
  <input
    type="password"
    placeholder="Password"
    value={password}
    onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
      setPassword(e.target.value)}
  />
  <div className="button-group">
    <button type="submit">Login</button>
    <button type="button" onClick={() => navigate("/")}>Home</button>
  </div>
</form>
<p>
  New user?{" "}
  <span onClick={() => navigate("/register")}>Register</span></p>
</div>
</div>
);};

```

Login CSS code

```
/* Full page center */
.login-container {
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  background: linear-gradient(to right, #0b5ed7, #198754);
  font-family: Arial, sans-serif;
}
/* Card */
.login-card {
  background: white;
  padding: 20px;
  width: 350px;
  border-radius: 10px;
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
}
/* Heading */
.login-card h2 {
  text-align: center;
  color: #007bff;
  margin-bottom: 20px;
}
/* Form elements */
.login-card select,
.login-card input {
  width: 100%;
  height: 42px;          /* □ THIS fixes size mismatch */
  padding: 10px;
  margin-bottom: 15px;
  border-radius: 6px;
  border: 1px solid #ccc;
  font-size: 14px;
```

```
box-sizing: border-box;
}
.login-card select {
  appearance: none;
  -webkit-appearance: none;
  -moz-appearance: none;
}
/* Button */
.login-card button {
  width: 50%;
  padding: 10px;
  background-color: #007bff;
  color: white;
  font-size: 15px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}
.login-card button:hover {
  background-color: #0056b3;
}
.back-btn {
  background-color: #ccc;
  border: none;
  padding: 10px 18px;
  border-radius: 5px;
  cursor: pointer;
}
.back-btn:hover {
  background-color: #b3b3b3;
}
/* Register text */
.login-card p {
  text-align: center;
  margin-top: 15px;
```

```

}
.login-card span {
  color: #007bff;
  cursor: pointer;
  font-weight: bold;
}
.login-card span:hover {
  text-decoration: underline;
}
.button-group {
  display: flex;
  gap: 10px;    /* space between buttons */
}
.button-group button {
  width: 50%;
}

```

Registration page code:

```

import React, { useState } from "react";
import "./Register.css";
import { useNavigate } from "react-router-dom";
import axios from "axios";

const Register: React.FC = () => {
  const navigate = useNavigate();
  const [role, setRole] = useState<string>("");
  const [firstName, setFirstName] = useState<string>("");
  const [lastName, setLastName] = useState<string>("");
  const [email, setEmail] = useState<string>("");
  const [contact, setContact] = useState<string>("");
  const [address, setAddress] = useState<string>("");
  const [gender, setGender] = useState<string>("");

```

```
const [dob, setDob] = useState<string>("");
const [password, setPassword] = useState<string>("");
const [confirmPassword, setConfirmPassword] = useState<string>("");

const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();

  if (!role || !firstName || !lastName || !email || !contact || !address || !gender || !dob
    || !password || !confirmPassword) {
    alert("Please fill all fields");
    return;
  }
  // 10 digit contact validation
  const contactRegex = /^[0-9]{10}$/;
  if (!contactRegex.test(contact)) {
    alert("Contact number must be exactly 10 digits");
    return;
  }
  if (password !== confirmPassword) {
    alert("Passwords do not match");
    return;
  }
  axios.post("http://localhost:5000/api/registration/register", {
    role,
    firstName,
    lastName,
    email,
    contact,
    address,
    gender,
    dob,
    password
  })
  .then(response => {
    alert("Registered successfully! Please login.");
```

```
    navigate("/login");
    return;
  })
  .catch(error => {
    console.error(error);
  });
}
return (
  <div className="register-container">
    <div className="register-card">
      <h2>Register</h2>
      <form onSubmit={handleSubmit}>
        <select
          value={role}
          onChange={(e: React.ChangeEvent<HTMLSelectElement>) =>
            setRole(e.target.value)}
        >
          <option value="">Select Role</option>
          <option value="patient">Patient</option>
          <option value="doctor">Doctor</option>
          <option value="admin">Admin</option>
          <option value="receptionist">Receptionist</option>
        </select>

        <input
          type="text"
          placeholder="First Name"
          value={firstName}
          onChange={(e) => setFirstName(e.target.value)}
        />
        <input
          type="text"
          placeholder="Last Name"
```

```
    value={lastName}
    onChange={(e) => setLastName(e.target.value)}
  />
  <input
    type="email"
    placeholder="Email ID"
    value={email}
    onChange={(e) => setEmail(e.target.value)}
  />
  <input
    type="text"
    placeholder="Contact Number"
    value={contact}
    onChange={(e) => setContact(e.target.value)}
  />
  <textarea
    placeholder="Address"
    value={address}
    onChange={(e) => setAddress(e.target.value)}
  />
  { /* ☐ Gender field */}
  <select
    value={gender}
    onChange={(e: React.ChangeEvent<HTMLSelectElement>) =>
      setGender(e.target.value)}
  }
  >
    <option value="">Select Gender</option>
    <option value="male">Male</option>
    <option value="female">Female</option>
    <option value="other">Other</option>
  </select>
  { /* ☐ DOB field */}
  <input
    type="date"
```

```

placeholder="Date of Birth"
value={dob}
onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
  setDob(e.target.value)
}
/>
<input
  type="password"
  placeholder="Password"
  value={password}
  onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
    setPassword(e.target.value)
  }
/>
<input
  type="password"
  placeholder="Confirm Password"
  value={confirmPassword}
  onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
    setConfirmPassword(e.target.value)
  }
/>
<div className="button-group">
  <button type="submit">Register</button>
  <button type="button" onClick={() => navigate("/")}>Home</button>
</div>
</form>
<p className="login-link">
  Already have an account?{" "}
  <span onClick={() => navigate("/login")}>Login</span> </p>
</div>
</div>
);
}
export default Register; // □ THIS LINE IS REQUIRED

```


Registration CSS code:

```
/* Full page center */
.register-container {
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  background: linear-gradient(to right, #0b5ed7, #198754);
  font-family: Arial, sans-serif;
}

/* Card */
.register-card {
  background: white;
  padding: 30px;
  width: 380px;
  border-radius: 10px;
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
}

/* Heading */
.register-card h2 {
  text-align: center;
  color: #007bff;
  margin-bottom: 20px;
}

/* Inputs & select */
.register-card select,
.register-card input,
.register-card textarea {
  width: 100%;
  height: 42px;          /* □ THIS fixes size */
  padding: 10px;
  margin-bottom: 15px;
  border-radius: 6px;
  border: 1px solid #ccc;
  font-size: 14px;
```

```
box-sizing: border-box;
}
.register-card select {
  appearance: none;
  -webkit-appearance: none;
  -moz-appearance: none;
}
/* Button */
.register-card button {
  width: 100%;
  padding: 10px;
  background-color: #007bff;
  color: white;
  font-size: 15px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}
.register-card button:hover {
  background-color: #0056b3;
}
/* Login redirect text */
.register-card p {
  text-align: center;
  margin-top: 15px;}
.register-card span {
  color: #007bff;
  cursor: pointer;
  font-weight: bold;
}
.register-card span:hover {
  text-decoration: underline;
}
.login-link {
  text-align: center;
```

```
margin-top: 15px;
font-size: 14px;
}
.login-link span {
color: #065cb7;
cursor: pointer;
font-weight: bold;
}
.login-link span:hover {
text-decoration: underline;
}
.username-row {
display: flex;
align-items: center;
gap: 10px;
margin-bottom: 15px;
}
.generate-btn {
width: 130px;
padding: 8px 10px;
font-size: 14px;
border-radius: 6px;
background-color: #2563eb;
color: white;
border: none;
cursor: pointer;
white-space: nowrap;
}
.generate-btn:hover {
background-color: #1e4fd8;
}

.username-row input {
flex: 1;
}
```

- **Patient Module Code:**

```
import React from "react";
import { useNavigate } from "react-router-dom";
import "../PatientDashboard.css";
import ProfileDetails from "../Profile Card/ProfileDetails";
import userImg from "../../src/assets/patient.jpg";
const PatientDashboard: React.FC = () => {
  const navigate = useNavigate();
  const [patient, setPatient] = React.useState<any>({});
  React.useEffect(() => {
    const storedUser = JSON.parse(localStorage.getItem("user") || "{}");
    setPatient(storedUser);
  }, []);
  const handleLogout = (): void => {
    localStorage.clear(); // remove login info
    navigate("/login"); // redirect to login
  };
  return (
    <div className="patient-dashboard">
      { /* Header */ }
      <div className="dashboard-header">
        <h2>Patient Dashboard</h2>
        <div className="header-right">
          <span className="name">`${patient["first_name"]}
            ${patient["last_name"]}`</span>
          <button className="logout-btn" onClick={handleLogout}>
            Logout
          </button>
        </div>
      </div>
      { /* Dashboard Cards */ }
      <div className="dashboard-cards">
        <div
          className="card"

```

```

        onClick={() => navigate("/book-appointment")}
      >
        Book Appointment
      </div>
      <div
        className="card"
        onClick={() => navigate("/my-appointments")}
      >
        My Appointments
      </div>
      <div
        className="card"
        onClick={() => navigate("/prescriptions")}
      >
        Prescriptions
      </div>
      <div
        className="card"
        onClick={() => navigate("/medical-records")}
      >
        Medical Records
      </div>
      <div
        className="card"
        onClick={() => navigate("/update-profile")}
      >Update Profile
    </div>
    </div>
    <ProfileDetails userImg={userImg}></ProfileDetails>
  </div>
);
};

```

```

export default PatientDashboard;

```

```
.patient-dashboard {  
  min-height: 100vh;  
  background: #f4f7fb;  
  padding: 20px;  
}  
.dashboard-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  background: #2563eb;  
  color: white;  
  padding: 15px 25px;  
  border-radius: 10px;  
}  
.dashboard-header button {  
  background: white;  
  color: #2563eb;  
  border: none;  
  padding: 8px 16px;  
  border-radius: 6px;  
  cursor: pointer;  
  font-weight: bold;  
}  
.dashboard-cards {  
  margin-top: 30px;  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));  
  gap: 20px;  
}  
.card {  
  background: white;  
  padding: 30px;  
  text-align: center;
```

```
border-radius: 12px;  
font-weight: 600;  
cursor: pointer;  
box-shadow: 0 4px 10px rgba(0,0,0,0.1);  
transition: transform 0.2s;  
}  
.card:hover {  
  transform: translateY(-5px);  
}  
.dashboard-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  background: #2563eb;  
  padding: 15px 25px;  
  color: white;  
}  
.header-buttons {  
  display: flex;  
  gap: 10px;  
}  
.logout-btn {  
  background: white;  
  color: #2563eb;  
  border: none;  
  padding: 8px 14px;  
  border-radius: 6px;  
  cursor: pointer;  
  font-weight: bold;  
}  
.name {  
  font-weight: bold;  
  padding: 8px 14px;  
}
```

```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "./BookAppointment.css";
import axios from "axios";

interface AppointmentForm {
  doctor: string;
  date: string;
  hour: string;
  minute: string;
  period: "AM" | "PM";
  reason: string;
}

const BookAppointment: React.FC = () => {
  const navigate = useNavigate();

  const [formData, setFormData] = useState<AppointmentForm>({
    doctor: "",
    date: "",
    hour: "",
    minute: "",
    period: "AM",
    reason: "",
  });

  interface User {
    user_id: number;
    first_name: string;
    last_name: string;
  }

  const [doctors, setDoctors] = useState<User[]>([]);
  const patient = JSON.parse(localStorage.getItem("user") || "");
```



```

useEffect(() => {
  if(!doctors.length){
    axios.get("http://localhost:5000/api/registration/fetchdoctors")
    .then(response => {
      setDoctors(response.data);
    })
    .catch(error => {
      console.error(error);
      alert("Something went wrong, please try after sometime!!")
    });
  }
});

const handleChange = (
  e: React.ChangeEvent<
    HTMLInputElement | HTMLSelectElement | HTMLTextAreaElement
  >
): void => {
  const { name, value } = e.target;
  setFormData({ ...formData, [name]: value });
};

const handleSubmit = ( e: React.FormEvent<HTMLFormElement>) =>{
  e.preventDefault();
  axios.post("http://localhost:5000/api/appointments/bookappointment",{
    patientId : patient["user_id"],
    doctorId : formData.doctor,
    appointmentDate:formData.date,
    appointmentTime:`${formData.hour}:${formData.minute}
      ${formData.period}`,
    visitReason:formData.reason
  })
  .then(response => {
    if(response.data["response"]){
      alert(`Appointment booked successfully, Your appointment id is
        ${response.data["

```

```
    setFormData({
      doctor: "",
      date: "",
      hour: "",
      minute: "",
      period: "AM",
      reason: "",
    })
  }
  else{
    alert("Something went wrong, please try after sometime!!")
  }
})
.catch(error => {
  console.error(error);
  alert("Something went wrong, please try after sometime!!")
});
};
```

```
return (
  <div className="book-appointment">
    <div className="page-header">
      <h2>Book Appointment</h2>
      <button
        type="button"
        className="back-btn"
        onClick={() => navigate(-1)}
      >
        Back
      </button>
    </div>
```

```

<form onSubmit={handleSubmit}>
  <select
    name="doctor"
    value={formData.doctor}
    onChange={handleChange}
    required
  >
    <option value="">Select Doctor</option>
    {doctors.length > 0 && doctors.map((doc) => (
      <option key={doc.user_id} value={doc.user_id}>
        Dr. {doc.first_name} {doc.last_name}
      </option>
    ))}
  </select>
  <input
    type="date"
    name="date"
    value={formData.date}
    onChange={handleChange}
    required
  />
  <div className="time-group">
    <select
      name="hour"
      value={formData.hour}
      onChange={handleChange}
      required
    >
      <option value="">HH</option>
      {[...Array(12)].map((_, i) => (
        <option key={i} value={String(i + 1).padStart(2, "0")}>
          {String(i + 1).padStart(2, "0")}
        </option>
      ))}
    </select>
  </div>

```

```

<select
  name="minute"
  value={formData.minute}
  onChange={handleChange}
  required
>
  <option value="">MM</option>
  {[...Array(60)].map((_, i) => (
    <option key={i} value={String(i).padStart(2, "0")}>
      {String(i).padStart(2, "0")}
    </option>
  ))}
</select> <select
  name="period"
  value={formData.period}
  onChange={handleChange}
>
  <option value="AM">AM</option>
  <option value="PM">PM</option>
</select>
</div>
<textarea
  name="reason"
  placeholder="Reason for appointment"
  value={formData.reason}
  onChange={handleChange}
  required
/>
<div className="button-group">
  <button type="submit">Confirm Appointment</button>
</div> </form>
</div>
);
};
export default BookAppointment;

```

```
.book-appointment {  
  min-height: 100vh;  
  background: #f4f7fb;  
  padding: 40px;  
}
```

```
/* --- THE HEADER BAR --- */
```

```
.page-header {  
  display: flex;  
  align-items: center;  
  justify-content: space-between; /* Pushes title left and button right */  
  background: #2563eb;  
  color: white;  
  padding: 0 30px;  
  /* Negative margins reach the very edge of the screen */  
  margin: -40px -40px 30px -40px;  
  height: 60px;  
  border: none;  
}
```

```
.page-header h2 {  
  margin: 0;  
  padding: 0;  
  background: transparent;  
  background-color: #2563eb !important;  
  font-size: 20px;  
  font-weight: bold;  
  color: white !important;  
  text-transform: uppercase;  
}
```

```
.page-header .back-btn {  
  background: white ;  
  color: rgb(17, 89, 213);  
  border: 1px solid white;  
  padding: 6px 14px;  
  font-size: 13px;
```

```
font-weight: bold;
border-radius: 6px;
cursor: pointer;
}
```

```
/* --- THE SMALL BLUE BACK BUTTON --- */
```

```
.back-btn {
  /* REMOVED: position: fixed, right: 0, top: 0 */
  position: static !important; /* Forces it to stay inside the flex header */
```

```
background-color: #2563eb; /* Blue to match the bar */
color: white;
border: 1px solid white;
```

```
/* Size adjustments */
width: 75px;
padding: 5px 0;
```

```
font-size: 11px;
font-weight: bold;
text-transform: uppercase;
cursor: pointer;
transition: all 0.2s ease;
```

```
/* Ensuring it stays far right */
margin-left: auto;
}
```

```
.page-header .back-btn:hover {
  background: white; /* Inverts colors on hover */
  color: #2563eb;
  text-decoration: none;
}
```

```
/* --- FORM AND INPUT STYLING --- */
```

```
.book-appointment form {  
  max-width: 450px;  
  margin: auto;  
  background: white;  
  padding: 30px;  
  border-radius: 12px;  
  box-shadow: 0 4px 10px rgba(0,0,0,0.1);  
}  
.book-appointment select,  
.book-appointment input,  
.book-appointment textarea {  
  width: 100%;  
  padding: 12px;  
  margin-bottom: 15px;  
  border-radius: 6px;  
  border: 1px solid #ccc;  
  box-sizing: border-box;  
}  
.book-appointment textarea {  
  resize: none;  
  height: 90px;  
}
```

```
.time-group {  
  display: flex;  
  gap: 10px;  
  margin-bottom: 15px;
```

```
}  
.button-group button {  
  width: 100%;  
  background: #2563eb;  
  color: white;  
  padding: 12px;  
  border: none;  
  border-radius: 8px;  
  font-weight: bold;  
  cursor: pointer;  
}/* --- ID GROUP STYLING --- */  
.id-group {  
  display: flex;  
  align-items: center;  
  gap: 10px;  
  margin-bottom: 15px;  
}  
.generate-id-btn {  
  width: 30% !important;  
  padding: 8px 12px;  
  font-size: 13px;  
  background-color: #0b5ed7;  
  color: white;  
  border: none;  
  border-radius: 5px;  
  cursor: pointer;  
}
```



```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "./MyAppointments.css";
import axios from "axios";
```

```
interface Appointment {
  appointmentId: string;
  doctor: string;
  date: string;
  time: string;
  reason: string;
  status: string;
}
```

```
const MyAppointments: React.FC = () => {
  const navigate = useNavigate();
  const [appointments, setAppointments] = useState<Appointment[]>([]);
```

```
  const formatDate = (dateString: string) => {
    const date = new Date(dateString);
    const day = String(date.getDate()).padStart(2, "0");
    const month = String(date.getMonth() + 1).padStart(2, "0");
    const year = date.getFullYear();
```

```
    return `${day}-${month}-${year}`;
  };
```

```
  useEffect(() => {
```

```
if(!appointments.length){  
  const patient = JSON.parse(localStorage.getItem("user") || "");
```

```
  const patientId = patient["user_id"];
```

```
    axios.get(`http://localhost:5000/api/appointments/getappointments/${patientId}`)  
  .then(response => {  
    setAppointments(response.data)  
  })  
  .catch(error => {  
    console.error(error);  
    alert("Something went wrong, please try after sometime!!")  
  });  
}  
  
));
```

```
return (  
  <div className="medical-records-container">  
    { /* HEADER WRAPPER - This is the key change */}  
    <div className="medical-header">  
      <h2>My Appointments</h2>  
      <button  
        className="medical-back-btn"  
        onClick={() => navigate(-1)}  
      >  
        Back  
      </button>  
    </div>
```

```
{/* CONTENT SECTION */}
<div className="records-section">
  {appointments.length === 0 ? (
    <p className="no-records">No appointments booked yet.</p>
  ) : (
    <div className="records-list">
      {appointments.map((appt, index) => (
        <div className="record-card" key={index}>
          <p><strong>Appointment ID:</strong> {appt.appointmentId}</p>
          <p><strong>Doctor:</strong> {appt.doctor}</p>
          <p><strong>Date:</strong> {formatDate(appt.date)}</p>
          <p><strong>Time:</strong> {appt.time}</p>
          <p><strong>Reason:</strong> {appt.reason}</p>
        </div>
      ))}
    </div>
  )}
</div>
</div>
);
};

export default MyAppointments;
```

```
.medical-records-container {  
  min-height: 100vh;  
  background: #f4f7fb;  
  padding: 40px;  
}
```

```
.back-btn:hover {  
  background: #1d4ed8;  
}
```

```
.medical-header {  
  display: flex;           /* Activates the alignment engine */  
  align-items: center;     /* Centers items vertically */  
  justify-content: space-between; /* Pushes text to LEFT and button to RIGHT */  
  background: #2563eb;     /* Standard Blue to match other pages */  
  color: white;  
  padding: 0 30px;        /* Side spacing */  
  /* Bleeds the header to the edges of the container padding */  
  margin: -40px -40px 30px -40px;  
  height: 60px;  
}
```

```
.medical-form {  
  max-width: 450px;  
  margin: auto;  
  background: white;  
  padding: 30px;  
  border-radius: 12px;  
  display: flex;  
  flex-direction: column;  
  gap: 15px;  
}
```

```
.medical-form input,  
.medical-form textarea {  
  padding: 10px;  
  border-radius: 8px;  
  border: 1px solid #ccc;  
}
```

```
.file-label {  
  font-weight: 600;  
  color: #333;  
}
```

```
.save-btn {  
  background: #2563eb;  
  color: white;  
  padding: 10px;  
  border: none;  
  border-radius: 8px;  
  cursor: pointer;  
}
```

```
.save-btn:hover {  
  background: #1d4ed8;  
}
```

```
/* BLUE HEADER */  
.medical-header {  
  background: #3566d6;  
  color: white;
```

```
padding: 18px 30px;  
position: relative; /* IMPORTANT */
```

```
.medical-header h2 {  
  margin: 0;  
  padding: 0;  
  background: transparent; /* Removes any grey/box background */  
  font-size: 20px;  
  font-weight: bold; /* Bold white text */  
  color: white;  
  text-transform: uppercase;  
}
```

```
/* TOP RIGHT BACK BUTTON */
```

```
.medical-back-btn {  
  background: #2563eb; /* Blue background matching the header */  
  color: white;  
  border: 1px solid white; /* White pill outline */
```

```
/* The size you wanted */  
width: 75px;  
padding: 5px 0;
```

```
border-radius: 20px; /* Precise rounded pill shape */  
font-size: 11px;  
font-weight: bold;  
text-transform: uppercase;  
cursor: pointer;  
transition: all 0.2s ease;  
}
```

```
.medical-back-btn:hover {
```

```
background: white;      /* Inverts colors on hover */
color: #2563eb;
text-decoration: none;
}
```

```
/* RECORDS SECTION (FULL CENTER FIX) */
.records-section {
  width: 100%;
  display: flex;
  flex-direction: column;
  align-items: center; /* This centers everything */
  margin-top: 40px;
}
```

```
/* HEADING */
.records-section h3 {
  color: #2563eb;      /* Blue heading */
  margin-bottom: 20px;
  text-align: center;
}
```

```
/* LIST */
.records-list {
  display: flex;
  flex-direction: column;
  align-items: flex-start; /* □ left align */
  width: 100%;
  padding: 20px 40px;
}
```

```
/* CARD */
```

```
.record-card {  
  background: white;  
  padding: 20px;  
  border-radius: 12px;  
  box-shadow: 0 4px 10px rgba(0,0,0,0.08);  
  
  width: 100%;      /* □ full width */  
  max-width: 100%; /* □ no shrinking */  
  margin: 10px 0;   /* spacing between cards */  
}
```

```
/* BUTTON */  
.download-btn {  
  margin-top: 12px;  
  background: #2563eb;  
  color: white;  
  border: none;  
  padding: 8px 16px;  
  border-radius: 6px;  
  cursor: pointer;  
}
```

```
/* NO RECORD TEXT */  
.no-records {  
  text-align: center;  
  color: gray;  
}
```



```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "../MedicalRecords.css";
import axios from "axios";
```

```
const MedicalRecords: React.FC = () => {
  const navigate = useNavigate();
```

```
  const [file, setFile] = useState<File | null>(null);
  const [comment, setComment] = useState("");
  const [title, setTitle] = useState("");
  const [description, setDescription] = useState("");
  const [records, setRecords] = useState<any[]>([]);
```

```
  useEffect(()=>{
    if(!records.length){
      getMedicalRecordsByPatientId();
    }
  });
```

```
  const getMedicalRecordsByPatientId = ()=>{
    const patient = JSON.parse(localStorage.getItem("user") || "");
```

```
    const patientId = patient["user_id"];
```

```
    axios.get(`http://localhost:5000/api/medicalrecords/getmedicalrecordsbypatientid/${patientId}`)
```

```
.then(response => {
  setRecords(response.data)
})
.catch(error => {
  console.error(error);
  alert("Something went wrong, please try after sometime!!")
});
}
```

```
const formatDate = (dateString:string) => {
  const date = new Date(dateString);
  return date.toDateString();
}
```

```
const handleFileChange = (e: React.ChangeEvent<HTMLInputElement>)
=> {
  if (e.target.files) {
    setFile(e.target.files[0]);
  }
};
```

```
const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
```

```
  if (!title || !description || !file) {
    alert("Please fill all fields");
    return;
  }
```

```
  const patient = JSON.parse(localStorage.getItem("user") || "");
```

```
const patientId = patient["user_id"];
```

```
const formData = new FormData();  
formData.append("file", file);  
formData.append("patientId", patientId);  
formData.append("description", description);  
formData.append("title", title);  
formData.append("comment", comment);
```

```
    axios.post("http://localhost:5000/api/medicalrecords/savemedicalrecord",f  
    ormData)  
    .then(response => {  
        if(response.data["response"]){  
            getMedicalRecordsByPatientId();  
            alert("Medical Record Uploaded Successfully!");  
            setFile(null);  
            setComment("");  
            setTitle("");  
            setDescription("");  
        }  
        else{  
            alert("Something went wrong, please try after sometime!!")  
        }  
    })  
    .catch(error => {  
        console.error(error);  
        alert("Something went wrong, please try after sometime!!")  
    });  
};
```

```
return (  
  <div className="medical-page">  
    {/* HEADER */}  
    <div className="medical-header">  
      <h2>MEDICAL RECORDS</h2>  
  
      <button  
        className="medical-back-btn"  
        onClick={() => navigate(-1)}  
      >  
        Back  
      </button>  
    </div>  
  
    {/* CONTENT */}  
    <div className="medical-content">  
      <form className="medical-form" onSubmit={handleSubmit}>  
  
        <input  
          type="text"  
          placeholder="Title"  
          value={title}  
          onChange={(e) => setTitle(e.target.value)}  
          required  
        />  
  
        <textarea  
          placeholder="Description"  
          value={description}  
          onChange={(e) => setDescription(e.target.value)}
```

```
    rows={3}
    required/>
<textarea
  placeholder="Add Comment"
  value={comment}
  onChange={(e) => setComment(e.target.value)}
  rows={4}
/>
```

```
<label className="file-label">
  Upload Medical Record
</label>
```

```
<input
  type="file"
  accept=".pdf, .png, .jpg, .jpeg"
  onChange={handleFileChange}
  required
/>
```

```
<button type="submit" className="save-btn">
  Upload Record
</button>
```

```
</form>
<div className="records-section">
  <div className="records-container"></div>
```

```
<h3>Your Uploaded Records</h3>
```

```

{records.length === 0 ? (
  <p className="no-records">No records uploaded yet</p>
) : (
  <div className="records-list">
    {records.map((rec, index) => (
      <div key={index} className="record-card">

        <p><strong>Title:</strong> {rec.title}</p>
        <p><strong>Description:</strong> {rec.description}</p>
        <p><strong>Comment:</strong> {rec.comment}</p>
        <p><strong>Date:</strong> {formatDate(rec.uploadedAt)}</p>

        <p><strong>File Name:</strong> {rec.fileName}</p>

        <a href={rec.fileURL} download={rec.fileName}>
          <button className="download-btn"> Download</button>
        </a>
      </div>
    ))}
  </div>
)}
</div>
</div>
</div>
);
};
export default MedicalRecords;

```

```
.medical-records-container {  
  min-height: 100vh;  
  background: #f4f7fb;  
  padding: 40px;  
}
```

```
.back-btn:hover {  
  background: #3566d6;  
}
```

```
.medical-heading {  
  text-align: center;  
  color: #2563eb;  
  background: white;  
  padding: 20px;  
  border-radius: 10px;  
  margin-bottom: 28px;  
}
```

```
.medical-header {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  background: #2563eb;
```

```
    color: white;
    padding: 0 30px;
    margin: -30px -30px 20px -20px;
    height: 100px;
}
```

```
.medical-form {
    max-width: 450px;
    margin: auto;
    background: white;
    padding: 30px;
    border-radius: 12px;
    display: flex;
    flex-direction: column;
    gap: 15px;
}
```

```
.medical-form input,
.medical-form textarea {
    padding: 10px;
    border-radius: 8px;
    border: 1px solid #ccc;
}
```

```
.file-label {
    font-weight: 600;
    color: #333;
}
```

```
.save-btn {
    background: #2563eb;
    color: white;
    padding: 10px;
    border: none;
    border-radius: 8px;
    cursor: pointer;
}
```



```
.save-btn:hover {  
  background: #1d4ed8;  
}  
/* BLUE HEADER */  
.medical-header {  
  background: #3566d6;  
  color: white;  
  padding: 18px 30px;  
  position: relative; /* IMPORTANT */  
}  
.medical-header h2 {  
  margin: 0;  
}
```

```
/* TOP RIGHT BACK BUTTON */  
.medical-back-btn {  
  position: absolute; /* MAGIC */  
  right: 25px;  
  top: 20px;  
  background: transparent;  
  border: none;  
  color: white;  
  font-size: 11px;  
  cursor: pointer;  
}
```

```
.medical-back-btn {  
  background: #3566d6;  
  color: white;  
  width: 80px;  
  padding: 18px 20px ;  
  margin: 3px 3px 2px 2px;  
  border-radius: 5px;  
  font-size: 15px;  
  font-weight: bold;  
  text-transform: uppercase;  
  cursor: pointer;  
  transition: all 0.2s ease;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}
```

```
import React from "react";
import { useNavigate } from "react-router-dom";
import "./Prescriptions.css";

const Prescriptions: React.FC = () => {
  const navigate = useNavigate();
  return (
    <div className="prescriptions-page">
      <div className="page-header">
        <h2>Prescriptions</h2>
        <h2>THIS IS PRESCRIPTION PAGE</h2>
        <button className="back-btn" onClick={() => navigate(-1)}>
          BACK
        </button>
      </div>

      <div className="prescriptions-content">
        <div style={{ color: "red" }}>TEST</div> { /* □ ADD HERE */}
        <p className="no-records">No prescriptions available.</p>
      </div>
    </div>
  );
};

export default Prescriptions;
```

```
/* MAIN PAGE */  
.prescriptions-page {  
  min-height: 100vh;  
  background-color: #f3f4f6;  
}
```

```
/* HEADER */  
.page-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  background-color: #3f6ccf;  
  padding: 18px 40px;  
}
```

```
.page-header h2 {  
  margin: 0;  
  color: white;  
  font-size: 25px;  
  font-weight: bold;  
  text-transform: uppercase;  
}
```

```
.back-btn {  
  background: transparent;  
  color: white;  
  width: 80px;  
  padding: 6px 20px;  
  font-size: 15px;  
  font-weight: bold;  
  text-transform: uppercase;  
  cursor: pointer;  
  transition: all 0.2s ease;  
  display: flex;  
  justify-content: center;  
  align-items: center;
```

```
.back-btn:hover {  
  opacity: 0.8;  
}
```

```
/* CONTENT */  
.prescriptions-content {  
  padding: 40px 80px;  
}
```

```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "./MyProfile.css";
```

```
const MyProfile: React.FC = () => {
  const navigate = useNavigate();
```

```
  const [username, setUsername] = useState<string>("");
  const [contact, setContact] = useState<string>("");
  const [patientId, setPatientId] = useState<string>("");
  const [currentPassword, setCurrentPassword] = useState<string>("");
  const [newPassword, setNewPassword] = useState<string>("");
  const [confirmPassword, setConfirmPassword] = useState<string>("");
  const [firstName, setFirstName] = useState<string>("");
  const [lastName, setLastName] = useState<string>("");
  const [email, setEmail] = useState<string>("");
  const [address, setAddress] = useState<string>("");
```

```
  useEffect(() => {
    const storedUsername = localStorage.getItem("username");
    const storedContact = localStorage.getItem("contact");
    const storedFirstName = localStorage.getItem("firstName");
    const storedLastName = localStorage.getItem("lastName");
    const storedEmail = localStorage.getItem("email");
    const storedAddress = localStorage.getItem("address");
    let storedPatientId = localStorage.getItem("patientId");
```

```
    if (!storedPatientId) {
```

```
    storedPatientId = "PAT-" + Math.floor(100000 + Math.random() *  
900000);  
    localStorage.setItem("patientId", storedPatientId);  
}
```

```
if (storedUsername) setUsername(storedUsername);  
if (storedContact) setContact(storedContact);  
if (storedPatientId) setPatientId(storedPatientId);  
if (storedFirstName) setFirstName(storedFirstName);  
if (storedLastName) setLastName(storedLastName);  
if (storedEmail) setEmail(storedEmail);  
if (storedAddress) setAddress(storedAddress);  
}, []);
```

```
const handleSave = () => {  
    const storedPassword = localStorage.getItem("password");
```

```
// □ Contact validation
```

```
if (contact.length !== 10) {  
    alert("Contact number must be exactly 10 digits");  
    return;  
}
```

```
// □ Password change validation
```

```
if (newPassword || confirmPassword || currentPassword) {  
    if (currentPassword !== storedPassword) {  
        alert("Current password is incorrect");  
        return;  
    }  
}
```

```
if (newPassword.length < 4) {  
    alert("New password must be at least 4 characters");  
    return;  
}
```

```
if (newPassword !== confirmPassword) {  
    alert("New password and Confirm password do not match");  
    return;  
}
```

```
localStorage.setItem("password", newPassword);  
}  
localStorage.setItem("firstName", firstName);  
localStorage.setItem("lastName", lastName);  
localStorage.setItem("email", email);  
localStorage.setItem("address", address);  
localStorage.setItem("username", username);  
localStorage.setItem("contact", contact);
```

```
alert("Profile Updated Successfully!");
```

```
setCurrentPassword("");  
setNewPassword("");  
setConfirmPassword("");  
};
```

```
return (  
    <div className="my-profile">  
        <div className="page-header">  
            <h2>My Profile</h2>
```



```
<button className="back-btn" onClick={() => navigate(-1)}>
  Back
</button>
</div>
<div className="profile-card">
  {/* Username */}
  <label>Patient ID</label>
  <input
    type="text"
    value={patientId}
    readOnly
  />
  <label>Username</label>
  <input
    type="text"
    value={username}
    onChange={(e) => setUsername(e.target.value)}
  />
  <label>First Name</label>
  <input
    type="text"
    value={firstName}
    onChange={(e) => setFirstName(e.target.value)}
  />
  <label>Last Name</label>
  <input
    type="text"
    value={lastName}
    onChange={(e) => setLastName(e.target.value)}
  />
  <label>Email ID</label>
  <input
    type="email"
```

```

    value={email}
    onChange={(e) => setEmail(e.target.value)}
  />
  {/ * Contact */}
  <label>Contact Number</label>
  <input
    type="text"
    value={contact}
    maxLength={10}
    placeholder="Enter 10-digit number"
    onChange={(e) => {
      const value = e.target.value;
      if (/^\d*$/.test(value)) {
        setContact(value);
      }
    }}
  />
  <label>Address</label>
  <textarea
    value={address}
    onChange={(e) => setAddress(e.target.value)}
  />
  {/ * Current Password */}
  <label>Current Password</label>
  <input
    type="password"
    value={currentPassword}
    onChange={(e) => setCurrentPassword(e.target.value)}
  />

  {/ * New Password */}
  <label>New Password</label>
  <input
    type="password"

```

```
    value={newPassword}  
    onChange={(e) => setNewPassword(e.target.value)}  
  />
```

```
  {  
    /* Confirm Password */  
    <label>Confirm New Password</label>  
    <input  
      type="password"  
      value={confirmPassword}  
      onChange={(e) => setConfirmPassword(e.target.value)}  
    />  
  }
```

```
    <button onClick={handleSave}>Save Changes</button>  
  </div>  
</div>  
);  
};
```

```
export default MyProfile;
```

```
.my-Profile-page {  
  padding: 30px;  
}
```

```
.page-header {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  background: #2563eb;  
  color: white;  
  padding: 18px 40px;  
  margin: -20px -20px 10px -10px;  
  height: 80px;  
  border: none;  
}
```

```
.page-header button {  
  background-color: #2563eb;  
  color: white;  
  border: none;  
  width: 80px;  
  padding: 6px 20px;  
  border-radius: 6px;  
  font-size: 15px;  
  font-weight: bold;  
  text-transform: uppercase;  
  cursor: pointer;  
  font-weight: 500;  
}
```

```
.back-btn {  
  background: rgb(30, 30, 246);
```

```
color: white;
width: 80px;
padding: 6px 20px;
font-size: 15px;
font-weight: bold;
text-transform: uppercase;
cursor: pointer;
transition: all 0.2s ease;
display: flex;
justify-content: center;
align-items: center;
}
```

```
.back-btn:hover {
  background-color: #084298;
}
```

```
.profile-card textarea {
  width: 100%;
  padding: 10px;
  border-radius: 6px;
  border: 1px solid #ccc;
  font-size: 14px;
  resize: none;      /* optional - prevents resizing */
  height: 45px;      /* makes height same as input */
  box-sizing: border-box;
}
```

- Doctor's Module code:

```
import React from "react";
import { useNavigate } from "react-router-dom";
import "../DoctorDashboard.css";
import ProfileDetails from "../Profile Card/ProfileDetails";

import userImg from "../../src/assets/doctor.jpg";

const DoctorDashboard: React.FC = () => {
  const navigate = useNavigate();

  const doctor = JSON.parse(localStorage.getItem("user") || "{}")

  const handleLogout = (): void => {
    localStorage.clear();
    navigate("/login");
  };

  return (
    <div className="doctor-dashboard">
      { /* Header */ }
      <div className="dashboard-header">
        <h2>Doctor Dashboard</h2>
        <div className="header-right">
          <span className="name">{`${doctor["first_name"]}
${doctor["last_name"]}`}</span>
          <button className="logout-btn" onClick={handleLogout}>
            Logout
          </button>
        </div>
      </div>
    </div>
  )
}
```

```

    {/* Dashboard Cards */}
    <div className="dashboard-cards">
      <div
        className="card"
        onClick={() => navigate("/scheduled-appointments")})
      >
        Scheduled Appointments
      </div>
      <div
        className="card"
        onClick={() => navigate("/patient-records")})
      >
        Medical Records of Patients
      </div>
      <div
        className="card"
        onClick={() => navigate("/provide-prescription")})
      >
        Provide Prescription
      </div>

      <div
        className="card"
        onClick={() => navigate("/update-profile")})
      >
        Update Profile
      </div>
    </div>
    <ProfileDetails userImg={userImg}></ProfileDetails>
  </div>
);
};

export default DoctorDashboard

```

```
.doctor-dashboard {  
  min-height: 100vh;  
  background: #f4f7fb;  
  padding: 30px;  
}
```

```
/* Header */  
.dashboard-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  margin-bottom: 30px;  
}  
.dashboard-header h2 {  
  color: #ffffff;    /* pure white */  
  font-weight: 700; /* bold */  
  font-size: 26px;  
}
```

```
/* Logout */  
.logout-btn {  
  padding: 8px 14px;  
  background-color: #0678d6;  
  color: white;  
  border: none;  
  border-radius: 6px;  
  cursor: pointer;  
}
```

```
/* Cards */  
.dashboard-cards {  
  display: grid;
```



```
grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));  
gap: 20px;  
}
```

```
.card {  
  background: white;  
  padding: 25px;  
  text-align: center;  
  border-radius: 10px;  
  font-weight: bold;  
  cursor: pointer;  
  box-shadow: 0 5px 15px rgba(0,0,0,0.1);  
  transition: transform 0.2s ease;  
}
```

```
.card:hover {  
  transform: translateY(-5px);  
}
```

```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "./ScheduledAppointments.css";
import axios from "axios";
```

```
interface Appointment {
  appointmentId: string;
  patientName: string;
  date: string;
  time: string;
  reason: string;
  status:string;
  appointmentType: string;
}
```

```
const ScheduledAppointments: React.FC = () => {
  const navigate = useNavigate();
  const [appointments, setAppointments] = useState<Appointment[]>([]);
```

```
  const formatDate = (dateString: string) => {
    const date = new Date(dateString);
    const day = String(date.getDate()).padStart(2, "0");
    const month = String(date.getMonth() + 1).padStart(2, "0");
    const year = date.getFullYear();
```

```
    return `${day}-${month}-${year}`;
  };
```

```
  useEffect(() => {
    if(!appointments.length){
```

```
const doctor = JSON.parse(localStorage.getItem("user") || "");
```

```
const doctorId = doctor["user_id"];
```

```
axios.get(`http://localhost:5000/api/appointments/getappointmentsbydoctorid/
${doctorId}`)
  .then(response => {
    setAppointments(response.data)
  })
  .catch(error => {
    console.error(error);
    alert("Something went wrong, please try after sometime!!")
  });
}
```

```
return (
  <div className="scheduled-page">
```

```
    {/ * □ HEADER */}
```

```
    <div className="scheduled-header">
```

```
      <h2>SCHEDULED APPOINTMENTS</h2>
```

```
      <button onClick={() => navigate(-1)}>
```

```
        Back
```

```
      </button>
```

```
    </div>
```

```

{ /* □ CONTENT */
<div className="scheduled-content">
  {appointments.length === 0 ? (
    <p className="no-data">No scheduled appointments.</p>
  ) : (
    appointments.map((appt, index) => (
      <div key={index} className="appointment-card">

        <p><strong>Appointment ID:</strong> {appt.appointmentId}</p>
        <p><strong>Patient Name:</strong> {appt.patientName}</p>
        <p><strong>Date:</strong> {formatDate(appt.date)}</p>
        <p><strong>Time:</strong> {appt.time}</p>
        <p><strong>Reason:</strong> {appt.reason}</p>
        <p><strong>Status:</strong> {appt.status}</p>
        <p><strong>Type:</strong> {appt.appointmentType}</p>

        <button
          className="prescription-btn"
          onClick={() =>
            navigate("/provide-prescription", {
              state: { appointmentId: appt.appointmentId }
            })
          }
        >
          Provide Prescription
        </button>
      </div>
    ))
  )}
</div>
</div>
);
}
export default ScheduledAppointments;

```

```
/* PAGE */
```

```
.scheduled-page {  
  background: #f4f7fb;  
  min-height: 100vh;  
}  
* {  
  box-sizing: border-box;  
}
```

```
body {  
  overflow-x: hidden;  
}
```

```
/* □ HEADER (Same as My Prescriptions) */
```

```
.scheduled-header {  
  width: 100%;  
  background: #2563eb;  
  color: white;  
  padding: 16px 24px;  
  box-sizing: border-box;  
  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

```
.scheduled-header h2 {  
  margin: 0;
```

```
    font-size: 22px;
}
.scheduled-header button {
    background: transparent;
    color: white;
    border: none;
    font-size: 15px;
    cursor: pointer;
}
/* CONTENT AREA */
.scheduled-content {
    padding: 20px;
}
/* APPOINTMENT CARD */
.appointment-card {
    background: white;
    padding: 16px;
    border-radius: 10px;
    margin-bottom: 15px;
    box-shadow: 0 2px 6px rgba(0,0,0,0.08);
}
/* PRESCRIPTION BUTTON */
.prescription-btn {
    margin-top: 10px;
    background: #2563eb;
    color: white;
    border: none;
    padding: 6px 14px;
    border-radius: 6px;
    cursor: pointer;
}
/* NO DATA */
.no-data {
    padding: 20px;
```

```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "../PatientRecords.css";
import axios from "axios";
```

```
interface MedicalRecord {
  recordId:number;
  patientId:string;
  patientName: string;
  title: string;
  description: string;
  comment: string;
  fileName: string;
  fileURL: string;
  uploadedAt:string;
}
```

```
const PatientRecords: React.FC = () => {
  const navigate = useNavigate();
  const [records, setRecords] = useState<MedicalRecord[]>([]);
```

```
  useEffect(() => {
    if(!records.length){
      const doctor = JSON.parse(localStorage.getItem("user") || "");
```

```
      const doctorId = doctor["user_id"];
```

```
      axios.get(`http://localhost:5000/api/medicalrecords/getmedicalrecordsbydoctor/${doctorId}`)
        .then(response => {
          setRecords(response.data)
        })
    }
  })
}
```

```

    })
    .catch(error => {
        console.error(error);
        alert("Something went wrong, please try after sometime!!")
    });
}
});

const handleDownload = (url: string, name: string) => {
    const link = document.createElement("a");
    link.href = url;
    link.download = name;
    link.click();
};

const formatDate = (dateString:string) => {
    const date = new Date(dateString);
    return date.toDateString();
}

return (
    <div className="doctor-medical-page">
        /* HEADER */
        <div className="doctor-medical-header">
            <h2>Patient Medical Records</h2>
            <button onClick={() => navigate(-1)}>
                Back
            </button>
        </div>
        /* RECORD LIST */
        {records.length === 0 ? (
            <p className="no-record">No medical records available</p>
        ) : (
            <div className="record-list">

```



```

{records.map((rec, index) => (
  <div className="record-card" key={index}>

    <p><strong>Patient Name:</strong> {rec.patientName}</p>
    <p><strong>Title:</strong> {rec.title}</p>
    <p><strong>Description:</strong> {rec.description}</p>
    <p><strong>Comment:</strong> {rec.comment}</p>
    <p><strong>Uploaded Date:</strong>
{formatDate(rec.uploadedAt)}</p>
    <p><strong>File Name:</strong> {rec.fileName}</p>
    <button
      className="download-btn"
      onClick={() => handleDownload(rec.fileURL, rec.fileName)}
    >
      Download File
    </button>
  </div>
  )})
</div>
)}
</div>
);
};

export default PatientRecords;

```

```
.doctor-medical-page {  
  padding: 20px;  
  background: #f4f7fb;  
  min-height: 100vh;  
}
```

```
.doctor-medical-header {  
  width: 100%;  
  background: #2563eb;  
  color: white;  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 14px 20px;  
  border-radius: 8px;  
  margin-bottom: 20px;  
}
```

```
.doctor-medical-header button {  
  background: white;  
  color: #2563eb;  
  border: none;  
  padding: 6px 16px;  
  border-radius: 6px;  
  cursor: pointer;  
}
```

```
.record-list {  
  display: flex;  
  flex-direction: column;  
  gap: 15px;  
}
```

```
.record-card {  
  background: white;  
  padding: 15px;  
  border-radius: 10px;  
  box-shadow: 0 2px 6px rgba(0,0,0,0.1);  
}
```

```
.download-btn {  
  margin-top: 10px;  
  background: #2563eb;  
  color: white;  
  border: none;  
  padding: 6px 14px;  
  border-radius: 6px;  
  cursor: pointer;  
}
```

```
.no-record {  
  text-align: center;  
  margin-top: 30px;  
}
```

```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "./ProvidePrescription.css";
import axios from "axios";
import { useLocation } from "react-router-dom";
```

```
const ProvidePrescription: React.FC = () => {
```

```
  const location = useLocation();
  const data = location.state;
  const navigate = useNavigate();
```

```
  const [appointmentId, setAppointmentId] = useState(data?.appointmentId);
  const [patientName, setPatientName] = useState("");
  const [patientId, setPatientId] = useState("");
  const [medicines, setMedicines] = useState("");
  const [instructions, setInstructions] = useState("");
```

```
  useEffect(()=>{
    getPatientDataByAppointmentId(appointmentId);
  })
  const getPatientDataByAppointmentId = (appointmentId:string)=>{
```

```
axios.get(`http://localhost:5000/api/appointments/getpatientdatabyappointme  
ntid/${appointmentId}`)  
  .then(response => {  
    setPatientName(response.data["patientName"]);  
    setPatientId(response.data["patientId"])  
  })  
  .catch(error => {  
    console.error(error);  
    alert("Something went wrong, please try after sometime!!")  
  });  
}
```

```
const handleSubmit = (e:any) => {
```

```
  e.preventDefault();
```

```
  if (!appointmentId || !medicines || !instructions) {  
    alert("Please fill all fields");  
    return;  
  }
```

```
  const doctor = JSON.parse(localStorage.getItem("user") || "");
```

```
  const doctorId = doctor["user_id"];
```

```
  axios.post("http://localhost:5000/api/prescriptions/saveprescription",{
```

```

    appointmentId,
    doctorId,
    patientId,
    medicines,
    instructions
  })
  .then(response => {
    if(response.data["response"]){
      alert("Prescription Saved Successfully!");
      setAppointmentId("");
      setPatientName("");
      setMedicines("");
      setInstructions("");
    }
    else{
      alert("Something went wrong, please try after sometime!!")
    }
  })
  .catch(error => {
    console.error(error);
    alert("Something went wrong, please try after sometime!!")
  });
};

```

```

return (
  <div className="provide-prescription">
    <div className="page-header">
      <h2>Provide Prescription</h2>
      <button onClick={() => navigate(-1)}>Back</button>
    </div>

    <div className="prescription-card">
      <input

```

```
        type="text"
        placeholder="Enter Appointment ID"
        value={appointmentId}
    />
    <input
        type="text"
        placeholder="Patient Name"
        value={patientName}
        readOnly
    />

    <textarea
        placeholder="Medicines"
        value={medicines}
        onChange={(e) => setMedicines(e.target.value)}
    />

    <textarea
        placeholder="Instructions"
        value={instructions}
        onChange={(e) => setInstructions(e.target.value)}
    />

    <button onClick={handleSubmit}>Save Prescription</button>
</div>
</div>
);
};

export default ProvidePrescription;
```

```
.provide-prescription {  
  min-height: 100vh;  
  background: #f4f7fb;  
  padding: 40px;  
}  
.page-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  background-color: #2563eb;  
  padding: 15px 20px;  
  border-radius: 10px;  
  margin-bottom: 20px;  
}  
.page-header h2 {  
  background-color: #e5e7eb; /* light grey box */  
  color: #1e40af;           /* dark blue text */  
  padding: 8px 20px;  
  border-radius: 8px;  
  margin: 0;  
  font-weight: 600;  
}  
.page-header button {  
  background: white;  
  color: #2563eb;  
  border: none;  
  padding: 6px 14px;  
  border-radius: 6px;  
  cursor: pointer;  
  font-weight: 500;  
}  
  
.prescription-card {  
  max-width: 500px;
```



```
margin: 30px auto;
background: white;
padding: 25px;
border-radius: 10px;
box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
}
```

```
.prescription-card input,
.prescription-card textarea {
width: 100%;
padding: 10px;
margin-bottom: 15px;
border-radius: 6px;
border: 1px solid #ccc;
}
```

```
.prescription-card button {
width: 100%;
padding: 10px;
background-color: #0b5ed7;
color: white;
border: none;
border-radius: 6px;
cursor: pointer;
}
```

```
.prescription-card button:hover {
background-color: #084298;
}
```

```
import React, { useState, useEffect } from "react";  
import { useNavigate } from "react-router-dom";  
import "../DoctorProfile.css";
```

```
const DoctorProfile: React.FC = () => {  
  const navigate = useNavigate();  
  const [contactNo, setContactNo] = useState("");  
  const [dob, setDob] = useState("");  
  const [currentPassword, setCurrentPassword] = useState("");  
  const [newPassword, setNewPassword] = useState("");  
  const [confirmPassword, setConfirmPassword] = useState("");  
  const [firstName, setFirstName] = useState("");  
  const [lastName, setLastName] = useState("");  
  const [address, setAddress] = useState("");  
  const [gender, setGender] = useState("");
```

```
  useEffect(() => {  
    const storedContact = localStorage.getItem("contactNo");  
    const storedDob = localStorage.getItem("dob");  
    const storedFirstName = localStorage.getItem("firstName");  
    const storedLastName = localStorage.getItem("lastName");  
    const storedAddress = localStorage.getItem("address");  
    const storedGender = localStorage.getItem("gender");
```

```
    if (storedContact) setContactNo(storedContact);  
    if (storedFirstName) setFirstName(storedFirstName);  
    if (storedLastName) setLastName(storedLastName);  
    if (storedAddress) setAddress(storedAddress);
```

```
    if (storedDob) setDob(storedDob);  
    if (storedGender) setGender(storedGender);  
  }, []);
```

```
const handleSave = () => {  
  // Contact validation  
  if (contactNo.length !== 10) {  
    alert("Contact number must be exactly 10 digits");  
    return;  
  }  
}
```

```
const storedPassword = localStorage.getItem("password");
```

```
// ☐ Check current password  
if (newPassword || confirmPassword) {  
  if (currentPassword !== storedPassword) {  
    alert("Current password is incorrect!");  
    return;  
  }  
}
```

```
if (newPassword !== confirmPassword) {  
  alert("New password and confirm password do not match!");  
  return;  
}
```

```
localStorage.setItem("password", newPassword);  
}
```

```
// Save other details
```

```
localStorage.setItem("contactNo", contactNo);
localStorage.setItem("firstName", firstName);
localStorage.setItem("lastName", lastName);
localStorage.setItem("address", address);
localStorage.setItem("dob", dob);
localStorage.setItem("gender", gender);
```

```
alert("Profile Updated Successfully!");
```

```
// Clear password fields
setCurrentPassword("");
setNewPassword("");
setConfirmPassword("");
};
```

```
return (
  <div className="doctor-profile">
    <div className="profile-header">
      <h2>Update Profile</h2>
      <button className="back-btn" onClick={() => navigate(-1)}>
        Back
      </button>
    </div>

    <div className="profile-card">
      <label>First Name</label>
      <input
        type="text"
        value={firstName}
```

```
    onChange={(e) => setFirstName(e.target.value)}  
  />
```

```
<label>Last Name</label>  
<input  
  type="text"  
  value={lastName}  
  onChange={(e) => setLastName(e.target.value)}  
  />
```

```
<label>Date of Birth</label>  
<input  
  type="date"  
  placeholder="Date of Birth"  
  value={dob}  
  onChange={(e: React.ChangeEvent<HTMLInputElement>) =>  
    setDob(e.target.value)}  
  />
```

```
<label>Gender</label>  
<select  
  className="form-input"  
  value={gender}  
  onChange={(e: React.ChangeEvent<HTMLSelectElement>) =>  
    setGender(e.target.value)}  
  />  
>
```

```
  <option value="">Select Gender</option>  
  <option value="male">Male</option>  
  <option value="female">Female</option>  
  <option value="other">Other</option>  
</select>
```

```
<label>Contact Number</label>
```

```
<input
  type="text"
  value={contactNo}
  onChange={(e) => {
    const value = e.target.value;

    // Allow only numbers
    if (/^\d*$/.test(value)) {
      setContactNo(value);
    }
  }}
  maxLength={10}
  placeholder="Enter 10-digit number"
/>
<hr style={{ margin: "20px 0" }} />
```

```
<label>Address</label>
<textarea
  value={address}
  onChange={(e) => setAddress(e.target.value)}
/>
```

```
<label>Current Password</label>
<input
  type="password"
  value={currentPassword}
  onChange={(e) => setCurrentPassword(e.target.value)}
/>
```

```
<label>New Password</label>
<input
```

```
    type="password"  
    value={newPassword}  
    onChange={(e) => setNewPassword(e.target.value)}  
  />
```

```
<label>Confirm New Password</label>  
<input  
  type="password"  
  value={confirmPassword}  
  onChange={(e) => setConfirmPassword(e.target.value)}  
/>
```

```
  <button onClick={handleSave}>Save Changes</button>  
</div>  
</div>  
);  
};
```

```
export default DoctorProfile;
```

```
.doctor-profile-page {  
  background: #f4f7fb;  
  min-height: 100vh;  
}  
* {  
  box-sizing: border-box;  
}
```

```
body {  
  overflow-x: hidden;  
}  
.profile-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  background: #2563eb;  
  padding: 15px 25px;  
  border-radius: 12px;  
  width: 100%;  
}
```

```
.profile-header h2 {  
  color: white;  
  margin: 0;  
  font-size: 22px;  
}
```

```
.profile-card {  
  max-width: 450px;  
  margin: 30px auto;  
  background: white;  
  padding: 25px;
```



```
border-radius: 10px;  
box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);  
}
```

```
.profile-card label {  
  font-weight: 600;  
  margin-bottom: 5px;  
  display: block;  
}  
.profile-card input {  
  width: 100%;  
  padding: 10px;  
  margin-bottom: 15px;  
  border-radius: 6px;  
  border: 1px solid #ccc;  
}
```

```
.profile-card button {  
  width: 100%;  
  padding: 10px;  
  background-color: #0b5ed7;  
  color: white;  
  border: none;  
  border-radius: 6px;  
  cursor: pointer;  
}
```

```
.profile-card button:hover {  
  background-color: #084298;  
}  
.profile-header button {  
  background-color: white;
```

```
color: #2563eb;
border: none;
padding: 6px 14px;
border-radius: 6px;
cursor: pointer;
font-weight: 500;
}
.profile-header button {
margin-left: auto;
}

.back-btn:hover {
background-color: #166eb2;
}
.profile-card textarea {
width: 100%;
padding: 10px;
border-radius: 6px;
border: 1px solid #ccc;
font-size: 14px;
resize: none; /* optional - prevents resizing */
height: 45px; /* makes height same as input */
box-sizing: border-box;
}.form-input {
width: 100%;
padding: 12px;
border-radius: 10px;
border: 1px solid #ccc;
font-size: 15px;
box-sizing: border-box;
background: white;
}
Select .form-input {
height: 45px;
}
```

- Admin's Module code:

```
import React from "react";
import { useNavigate } from "react-router-dom";
import "../AdminDashboard.css";
import ProfileDetails from "../Profile Card/ProfileDetails";
import userImg from "../../src/assets/admin.jpg";

const AdminDashboard: React.FC = () => {
  const navigate = useNavigate();

  const admin = JSON.parse(localStorage.getItem("user") || "{}");
  const handleLogout = (): void => {
    localStorage.clear(); // remove login info
    navigate("/login"); // redirect to login
  };

  return (
    <div className="admin-dashboard">
      {/* Header */}
      <div className="dashboard-header">
        <h2>Admin Dashboard</h2>
        <div className="header-right">
          <span className="name">`${admin["first_name"]}
${admin["last_name"]}`</span>
          <button className="logout-btn" onClick={handleLogout}>
            Logout
          </button>
        </div>
      </div>
      {/* Dashboard Cards */}
      <div className="dashboard-cards">
```

```
    <div
      className="card"
      onClick={() => navigate("/doctors-list")}
    >
      Doctors List
    </div>
    <div
      className="card"
      onClick={() => navigate("/reseptionist-list")}
    >
      Receptionist List
    </div>
    <div
      className="card"
      onClick={() => navigate("/update-profile")}
    >
      Update Profile
    </div>
    </div>
    <ProfileDetails userImg={userImg}></ProfileDetails>
  </div>
);
};

export default AdminDashboard;
```

```
.admin-dashboard {  
  min-height: 100vh;  
  background: #f4f7fb;  
  padding: 20px;  
}
```

```
.dashboard-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  background: #2563eb;  
  color: white;  
  padding: 15px 25px;  
  border-radius: 10px;  
}
```

```
.dashboard-header button {  
  background: white;  
  color: #2563eb;  
  border: none;  
  padding: 8px 16px;  
  border-radius: 6px;  
  cursor: pointer;  
  font-weight: bold;  
}
```

```
.dashboard-cards {  
  margin-top: 30px;  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr))  
  gap: 20px;  
}
```

```
.card {
```

```
background: white;  
padding: 30px;  
text-align: center;  
border-radius: 12px;  
font-weight: 600;  
cursor: pointer;  
box-shadow: 0 4px 10px rgba(0,0,0,0.1);  
transition: transform 0.2s;  
}  
.card:hover {  
  transform: translateY(-5px);  
}  
.dashboard-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  background: #2563eb;  
  padding: 15px 25px;  
  color: white;  
}  
.header-buttons {  
  display: flex;  
  gap: 10px;  
}  
.logout-btn {  
  background: white;  
  color: #2563eb;  
  border: none;  
  padding: 8px 14px;  
  border-radius: 6px;  
  cursor: pointer;  
  font-weight: bold;}
```

```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "../DoctorsList.css";
import axios from "axios";
```

```
interface Doctor {
  user_id:string;
  name: string;
  email:string;
  contact:string;
  specialization: string;
  shift_start:string;
  shift_end:string;
}
```

```
const DoctorsList: React.FC = () => {
  const navigate = useNavigate();
  const [doctors, setDoctors] = useState<Doctor[]>([])
```

```
  useEffect(()=>{

    axios.get("http://localhost:5000/api/doctors/getdoctors")
      .then(response => {
        setDoctors(response.data)
      })
      .catch(error => {
        console.error(error);
        alert("Something went wrong, please try after sometime!!")
      });
  })
}
```

)

```
return (  
  <div className="staff-container">  
    <div className="staff-header">  
      <h2>HOSPITAL STAFF SCHEDULE</h2>
```

```
      <button className="back-btn" onClick={() => navigate(-1)}>  
        Back  
      </button>  
    </div>
```

```
    <div className="staff-table">
```

```
      { /* Table Heading */}  
      <div className="staff-row staff-header">  
        <span>Doctor Name</span>  
        <span>Email</span>  
        <span>Contact</span>  
        <span>Specialization</span>  
        <span>Shift Timing</span>  
      </div>
```

```
      { /* Data Rows */}  
      {doctors.map((doc, index) => (  
        <div className="staff-row" key={index}>  
          <span>{doc.name}</span>
```



```
        <span>{doc.email}</span>
        <span>{doc.contact}</span>
        <span>{doc.specialization}</span>
        <span>{doc.shift_start ? (doc.shift_start + " - " + doc.shift_end) :
""}</span>
    </div>
    )}}
}
```

```
</div>
```

```
</div>
);
};
```

```
export default DoctorsList;
```

/* PAGE CONTAINER */

```
.staff-container {  
  background: white;  
  min-height: 100vh;  
  width: 100%;  
  padding: 40px 60px;  
  box-sizing: border-box;  
  font-family: Arial, sans-serif;  
}
```

/* BLUE HEADER BAR */

```
.staff-container > .staff-header {  
  background-color: #2563eb;  
  color: white;  
  padding: 22px 30px;  
  border-radius: 10px;  
  position: relative;  
  margin-bottom: 30px;  
}
```

/* HEADING */

```
.staff-container > .staff-header h2 {  
  margin: 0;  
  font-weight: 600;  
  letter-spacing: 1px;  
  text-align: left;  
  color: white;  
}
```

/* BACK BUTTON */

```
.back-btn {  
  position: absolute;  
  right: 30px;  
  top: 50%;  
  transform: translateY(-50%);  
  background: white;  
  border: none;  
  color: #2563eb;  
  padding: 6px 18px;  
  border-radius: 20px;  
  cursor: pointer;  
  font-weight: bold;  
}
```

```
.back-btn:hover {  
  background: #f3f4f6;  
}
```

/* TABLE */

```
.staff-table {  
  width: 100%;  
}
```

```
/* ROW STYLE */  
.staff-row {  
  display: flex;  
  justify-content: space-between;  
  padding: 14px 10px;  
  border-bottom: 1px solid #e5e7eb;  
  background: white;  
}
```

```
/* TABLE HEADER ROW */  
.staff-table .staff-header {  
  background: white;  
  color: #2563eb;    /* Blue text */  
  font-weight: bold;  
  border-bottom: 2px solid #2563eb;  
}
```

```
/* TABLE DATA */  
.staff-row span {  
  flex: 1;  
  color: black;  
}
```

```
/* EMAIL COLUMN WIDER */  
.staff-row span:nth-child(2) {  
  flex: 2;  
}
```

```
/* HOVER EFFECT */  
.staff-row:hover {  
  background: #f5f7ff;  
}
```

```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "../ReceptionistList.css";
import axios from "axios";
```

```
interface Receptionist {
  user_id:string;
  name: string;
  contact: string;
  shift_start: string;
  shift_end: string;
}
```

```
const ReceptionistList: React.FC = () => {
```

```
  const navigate = useNavigate();
  const [Receptionist, setReceptiont] = useState<Receptionist[]>([])
```

```
  useEffect(()=>{
    if(!Receptionist.length){
      axios.get("http://localhost:5000/api/receptionists/getreceptionists")
        .then(response => {
          setReceptiont(response.data)
        })
        .catch(error => {
          console.error(error);
          alert("Something went wrong, please try after sometime!!")
        });
    }
  })
```

```
})
```

```
return (
```

```
  <div className="staff-container">
```

```
    {/* Back Button */}
```

```
    <button className="back-btn" onClick={() => navigate(-1)}>
```

```
      Back
```

```
    </button>
```

```
  {/* Heading */}
```

```
  <h2 className="staff-title">RECEPTION STAFF SCHEDULE</h2>
```

```
  <div className="staff-table">
```

```
    {/* Header */}
```

```
    <div className="staff-row staff-header">
```

```
      <span>Staff Name</span>
```

```
      <span>Contact Number</span>
```

```
      <span>Shift Timing</span>
```

```
    </div>
```

```
    {/* Rows */}
```

```
    {Receptionist.map((receptionist, index) => (
```

```
      <div className="staff-row" key={index}>
```

```
        <span>{receptionist.name}</span>
```

```
        <span>{receptionist.contact}</span>
```

```
        <span>{receptionist.shift_start ? (receptionist.shift_start + " - " +
```

```
receptionist.shift_end) : ""
```

```
      </div>
```

```
    )))
```

```
  </div>
```

```
</div>
```

```
);
```

```
};
```

```
export default ReceptionistList;
```

```
.staff-container {  
  background: white;  
  min-height: 100vh;  
  width: 100%;  
  padding: 40px 60px;  
  box-sizing: border-box;  
  position: relative;  
}
```

```
/* BLUE HEADER BAR */  
.staff-header-bar {  
  background-color: #2563eb;  
  color: white;  
  padding: 20px 30px;  
  margin: -40px -60px 40px -60px;  
  border-radius: 12px;  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

```
.staff-header-bar h2{  
  margin: 0;  
  letter-spacing: 1px;  
}
```

```
/* TABLE */  
.staff-table {  
  width: 100%;  
}
```

/* ROWS */

```
.staff-row {  
  display: flex;  
  justify-content: space-between;  
  padding: 14px 10px;  
  border-bottom: 1px solid #e5e7eb;  
  background: white;  
  color: rgb(25, 21, 233);  
}
```

```
.staff-row:hover{  
  background:#f8fafc;  
}
```

/* HEADER ROW */

```
.staff-header {  
  
  font-weight: 600;  
  border-bottom: 2px solid #2563eb;  
}
```

/* COLUMN WIDTH */

```
.staff-row span {  
  flex: 1;  
}
```

```
/* .staff-row span:nth-child(2) {  
  flex: 2;  
}
```



```
.staff-row span:nth-child(3) {  
  text-align: right;  
} */
```

```
/* BACK BUTTON */  
.back-btn {  
  background: transparent;  
  color: white;  
  border: none;  
  font-size: 14px;  
  font-weight: 600;  
  cursor: pointer;  
}
```

```
.back-btn:hover {  
  background-color: #1e40af; /* Darker blue on hover */  
  color: #ffffff; /* Keep text white */  
}
```

```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "./AdminProfile.css"; // SAME CSS
```

```
const AdminProfile: React.FC = () => {
  const navigate = useNavigate();
```

```
  const [username, setUsername] = useState("");
  const [contact, setContact] = useState("");
  const [adminId, setAdminId] = useState("");
  const [currentPassword, setCurrentPassword] = useState("");
  const [newPassword, setNewPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [firstName, setFirstName] = useState("");
  const [lastName, setLastName] = useState("");
  const [email, setEmail] = useState("");
  const [address, setAddress] = useState("");
```

```
  useEffect(() => {
    const storedUsername = localStorage.getItem("adminUsername");
    const storedContact = localStorage.getItem("adminContact");
    const storedFirstName = localStorage.getItem("adminFirstName");
    const storedLastName = localStorage.getItem("adminLastName");
    const storedEmail = localStorage.getItem("adminEmail");
    const storedAddress = localStorage.getItem("adminAddress");
```

```
    let storedAdminId = localStorage.getItem("adminId");
```

```
    if (!storedAdminId) {
```

```
    storedAdminId = "ADM-" + Math.floor(100000 + Math.random() *  
900000);  
    localStorage.setItem("adminId", storedAdminId);  
}
```

```
if (storedUsername) setUsername(storedUsername);  
if (storedContact) setContact(storedContact);  
if (storedAdminId) setAdminId(storedAdminId);  
if (storedFirstName) setFirstName(storedFirstName);  
if (storedLastName) setLastName(storedLastName);  
if (storedEmail) setEmail(storedEmail);  
if (storedAddress) setAddress(storedAddress);
```

```
}, []);
```

```
const handleSave = () => {  
    const storedPassword = localStorage.getItem("adminPassword");
```

```
    if (contact.length !== 10) {  
        alert("Contact number must be exactly 10 digits");  
        return;  
    }
```

```
    if (newPassword || confirmPassword || currentPassword) {
```

```
        if (currentPassword !== storedPassword) {  
            alert("Current password is incorrect");  
            return;  
        }
```

```
if (newPassword.length < 4) {  
    alert("New password must be at least 4 characters");  
    return;  
}
```

```
if (newPassword !== confirmPassword) {  
    alert("New password & Confirm password do not match");  
    return;  
}
```

```
localStorage.setItem("adminPassword", newPassword);  
}
```

```
localStorage.setItem("adminUsername", username);  
localStorage.setItem("adminContact", contact);  
localStorage.setItem("adminFirstName", firstName);  
localStorage.setItem("adminLastName", lastName);  
localStorage.setItem("adminEmail", email);  
localStorage.setItem("adminAddress", address);
```

```
alert("Admin Profile Updated Successfully!");
```

```
setCurrentPassword("");  
setNewPassword("");  
setConfirmPassword("");  
};
```

```
return (  
  <div className="my-profile">  
  
    <div className="page-header">  
      <h2>My Profile</h2>  
  
      <button className="back-btn" onClick={() => navigate(-1)}>  
        Back  
      </button>  
    </div>  
  
    <div className="profile-card">  
  
      <label>Admin ID</label>  
      <input type="text" value={adminId} readOnly />  
  
      <label>Username</label>  
      <input  
        type="text"  
        value={username}  
        onChange={(e) => setUsername(e.target.value)}  
      />  
  
      <label>First Name</label>  
      <input  
        type="text"  
        value={firstName}  
        onChange={(e) => setFirstName(e.target.value)}  
      />  
    </div>  
  </div>  
)
```

```
<label>Last Name</label>
<input
  type="text"
  value={lastName}
  onChange={(e) => setLastName(e.target.value)}
/>
```

```
<label>Email</label>
<input
  type="email"
  value={email}
  onChange={(e) => setEmail(e.target.value)}
/>
```

```
<label>Contact Number</label>
<input
  type="text"
  value={contact}
  maxLength={10}
  onChange={(e) => {
    const value = e.target.value;
    if (/^\d*$/.test(value)) {
      setContact(value);
    }
  }}
/>
```

```
<label>Address</label>
<textarea
  value={address}
  onChange={(e) => setAddress(e.target.value)}
```

```
    />
    <label>Current Password</label>
    <input
      type="password"
      value={currentPassword}
      onChange={(e) => setCurrentPassword(e.target.value)}
    />
    <label>New Password</label>
    <input
      type="password"
      value={newPassword}
      onChange={(e) => setNewPassword(e.target.value)}
    />

    <label>Confirm New Password</label>
    <input
      type="password"
      value={confirmPassword}
      onChange={(e) => setConfirmPassword(e.target.value)}
    />

    <button onClick={handleSave}>Save Changes</button>
  </div>
</div>
);
};

export default AdminProfile;
```

`/* Admin Profile Page */`

**`.my-profile {
padding: 30px;
min-height: 100vh;
background: #f4f7fb;
}`**

**`.page-header {
display: flex;
justify-content: space-between;
align-items: center;
margin-bottom: 25px;
}`**

**`.page-header h2 {
color: white;
margin: 0;
}`**

**`.back-btn {
background-color: #0b5ed7;
color: white;
border: none;
padding: 8px 16px;
border-radius: 6px;
cursor: pointer;
font-weight: 500;
}`**


```
.back-btn:hover {  
  background-color: #084298;  
}
```

```
/* Profile Card */
```

```
.profile-card {  
  max-width: 500px;  
  margin: auto;  
  background: white;  
  padding: 30px;  
  border-radius: 12px;  
  box-shadow: 0 4px 12px rgba(0,0,0,0.1);  
}
```

```
.profile-card label {  
  font-weight: 600;  
  display: block;  
  margin-top: 15px;  
  margin-bottom: 5px;  
}
```

```
.profile-card input,  
.profile-card textarea {  
  width: 100%;  
  padding: 10px;  
  border-radius: 6px;  
  border: 1px solid #ccc;  
  font-size: 14px;  
  box-sizing: border-box;  
}
```

```
.profile-card textarea {  
  resize: none;  
  height: 60px;  
}
```

```
/* Save Button */
```

```
.profile-card button {  
  width: 100%;  
  margin-top: 20px;  
  background: #2563eb;  
  color: white;  
  padding: 12px;  
  border: none;  
  border-radius: 8px;  
  font-weight: bold;  
  cursor: pointer;  
}
```

```
.profile-card button:hover {  
  background: #1d4ed8;  
}
```

- Receptionist Module code:

```
import React from "react";
import { useNavigate } from "react-router-dom";
import "../ReceptionistDashboard.css";
import ProfileDetails from "../Profile Card/ProfileDetails";

import userImg from "../../src/assets/receptionist.jpg";

const receptionistDashboard: React.FC = () => {
  const navigate = useNavigate();

  const receptionist = JSON.parse(localStorage.getItem("user") || "{}")

  const handleLogout = (): void => {
    localStorage.clear(); // remove login info
    navigate("/login"); // redirect to login
  };

  return (
    <div className="receptionist-dashboard">
      { /* Header */ }
      <div className="dashboard-header">
        <h2>Receptionist Dashboard</h2>
        <div className="header-right">
          <span className="name">`${receptionist["first_name"]}
${receptionist["last_name"]}`</span>
          <button className="logout-btn" onClick={handleLogout}>
            Logout
          </button>
        </div>
      </div>
    </div>
  )
}
```

```
    {/* Dashboard Cards */}
    <div className="dashboard-cards">
      <div
        className="card"
        onClick={() => navigate("/View-Patients-Prescription")}
      >
        View Patients Prescription
      </div>
      <div
        className="card"
        onClick={() => navigate("/billing")}
      >
        Billing
      </div>
      <div
        className="card"
        onClick={() => navigate("/update-profile")}
      >
        Update Profile
      </div>
    </div>
    <ProfileDetails userImg={userImg}></ProfileDetails>
  </div>
);
};
```

```
export default receptionistDashboard;
```

```
.receptionist-dashboard {  
  min-height: 100vh;  
  background: #f4f7fb;  
  padding: 20px;  
}  
.dashboard-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  background: #2563eb;  
  color: white;  
  padding: 15px 25px;  
  border-radius: 10px;  
}  
.dashboard-header button {  
  background: white;  
  color: #2563eb;  
  border: none;  
  padding: 8px 16px;  
  border-radius: 6px;  
  cursor: pointer;  
  font-weight: bold;  
}  
.dashboard-cards {  
  margin-top: 30px;  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));  
  gap: 20px;  
}  
  
.card {  
  background: white;  
  padding: 30px;  
  text-align: center;
```

```
border-radius: 12px;  
font-weight: 600;  
cursor: pointer;  
box-shadow: 0 4px 10px rgba(0,0,0,0.1);  
transition: transform 0.2s;  
}
```

```
.card:hover {  
  transform: translateY(-5px);  
}  
.dashboard-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  background: #2563eb;  
  padding: 15px 25px;  
  color: white;  
}
```

```
.header-buttons {  
  display: flex;  
  gap: 10px;  
}
```

```
.logout-btn {  
  background: white;  
  color: #2563eb;  
  border: none;  
  padding: 8px 14px;  
  border-radius: 6px;  
  cursor: pointer;  
  font-weight: bold;}
```

```
import React, { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import "./ViewPrescription.css";
import axios from "axios";
```

```
interface Prescription {
  patient_name: string;
  doctor_name: string;
  medicines: string;
  instructions: string;
}
```

```
const ViewPrescription: React.FC = () => {
  const navigate = useNavigate();
  const [showPopup, setShowPopup] = useState(false);
  const [prescriptions, setPrescriptions] = useState<Prescription[]>([]);
```

```
  const handlePrescribeClick = () => {
    setShowPopup(true);
```

```
    setTimeout(() => {
      setShowPopup(false);
    }, 2000);
  };
};
```

```
useEffect(() => {
  axios
    .get("http://localhost:5000/api/prescriptions/getallprescriptions")
    .then((response) => {
      setPrescriptions(response.data);
    });
}, []);
```

```
    })  
    .catch((error) => {  
        console.error(error);  
    });  
}, []);
```

```
return (  
    <div className="view-prescription">  
  
        <div className="prescription-header">  
            <h2>Patient Prescriptions</h2>  
  
            <button className="back-btn" onClick={() => navigate(-1)}>  
                Back  
            </button>  
        </div>  
  
        <div className="prescription-table">  
  
            <div className="table-header">  
                <span>Patient Name</span>  
                <span>Doctor</span>  
                <span>Medicines</span>  
                <span>Notes</span>  
                <span>Actions</span>  
            </div>  
  
            {prescriptions.map((p, index) => (
```



```
<div className="table-row" key={index}>
  <span>{p.patient_name}</span>
  <span>{p.doctor_name}</span>
  <span>{p.medicines}</span>
  <span>{p.instructions}</span>
  <span>
    <button
      className="prescribed-btn"
      onClick={handlePrescribeClick}
    >
      Prescribed
    </button>
  </span>
</div>
```

```
    </span>
  </div>
  )})
```

```
</div>
{showPopup && (
  <div className="popup">
    Prescription Prescribed Successfully ☐
  </div>
)}
</div>
);
};
```

```
export default ViewPrescription;
```

```
.view-prescription {  
  min-height: 100vh;  
  background: #f4f7fb;  
  padding: 30px;  
}
```

```
/* Header */  
.prescription-header {  
  background: #2f64d6;  
  padding: 18px 25px;  
  border-radius: 12px;  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  margin-bottom: 20px;  
}
```

```
.prescription-header h2 {  
  color: white;  
  margin: 0;  
}
```

```
/* Back button */  
.back-btn {  
  background: white;  
  color: #2f64d6;  
  border: none;  
  padding: 8px 16px;  
  border-radius: 8px;  
  cursor: pointer;  
  font-weight: 500;
```

```
.back-btn:hover {  
  background: #f0f4ff;  
}
```

```
/* Table */  
.prescription-table {  
  background: white;  
  border-radius: 10px;  
  overflow: hidden;  
}
```

```
/* Header Row */  
.table-header {  
  display: flex;  
  background: #2563eb;  
  color: white;  
  padding: 12px;  
  font-weight: bold;  
}
```

```
.table-header span {  
  flex: 1;  
}
```

```
/* Data Rows */  
.table-row {  
  display: flex;  
  padding: 12px;  
  border-bottom: 1px solid #eee;  
}
```

```
.table-row span {  
  flex: 1;  
  color: black;  
}
```

```
.prescribed-btn {  
  background: #2f64d6;  
  color: white;  
  border: none;  
  padding: 6px 14px;  
  border-radius: 6px;  
  cursor: pointer;  
  font-size: 13px;  
}
```

```
.prescribed-btn:hover {  
  background: #1e4db7;  
}
```

```
.table-header,  
.table-row {  
  display: grid;  
  grid-template-columns: 1.5fr 1.2fr 2fr 2fr 1fr;  
  align-items: center;  
}
```

```
.popup {  
  position: fixed;  
  top: 20px;  
  right: 20px;  
  background: #22c55e;
```

```
color: white;  
padding: 14px 22px;  
border-radius: 8px;  
font-weight: 500;  
box-shadow: 0 4px 12px rgba(0,0,0,0.2);  
animation: fadeInOut 2s ease-in-out;  
}
```

```
@keyframes fadeInOut {  
  0% { opacity: 0; transform: translateY(-20px); }  
  20% { opacity: 1; transform: translateY(0); }  
  80% { opacity: 1; }  
  100% { opacity: 0; transform: translateY(-20px); }  
}
```

```
import React, { useState } from "react";
import { useNavigate } from "react-router-dom";
import "./Billing.css";
```

```
interface BillingRecord {
  appointmentId: string;
  patientName: string;
  amount: number;
  paymentMode: string;
  paymentDone: boolean; // ☐ ADD THIS
}
```

```
const Billing: React.FC = () => {
  const navigate = useNavigate();
  const [showPopup, setShowPopup] = useState(false);
  const [billingData, setBillingData] = useState<BillingRecord[]>([
    { appointmentId: "APT-1001", patientName: "Rahul Patil", amount: 1200,
      paymentMode: "", paymentDone: false },
    { appointmentId: "APT-1002", patientName: "Sneha Mehta", amount: 850,
      paymentMode: "", paymentDone: false },
    { appointmentId: "APT-1003", patientName: "Amit Sharma", amount:
      1500, paymentMode: "", paymentDone: false },
  ]);
```

```
  const handlePaymentChange = (index: number, mode: string) => {
    const updatedData = [...billingData];
    updatedData[index].paymentMode = mode;
    setBillingData(updatedData);
  };
```

```
  const handlePayClick = (index: number) => {
    if (!billingData[index].paymentMode) {
```

```
    alert("Please select payment mode first");  
    return;  
}
```

```
const updatedData = [...billingData];  
updatedData[index].paymentDone = true; // ☐ mark payment done  
setBillingData(updatedData);
```

```
setShowPopup(true);  
};
```

```
return (  
  <div className="billing-page">  
    <div className="billing-header">  
      <h2>Patients Billing</h2>
```

```
      <button className="back-btn" onClick={() => navigate(-1)}>  
        Back  
      </button>  
    </div>
```

```
    <table className="billing-table">  
      <thead>  
        <tr>  
          <th>Appointment ID</th>  
          <th>Patient Name</th>  
          <th>Total Billing Amount</th>  
          <th>Mode of Payment</th>  
        </tr>  
      </thead>
```

```

<tbody>
  {billingData.map((bill, index) => (
    <tr key={index}>
      <td>{bill.appointmentId}</td>
      <td>{bill.patientName}</td>
      <td>₹ {bill.amount}</td>

      <td>
        {bill.paymentDone ? (
          <span className="selected-payment">
            {bill.paymentMode} ☐
          </span>
        ) : (
          <div className="payment-action">
            <select
              value={bill.paymentMode}
              onChange={(e) =>
                handlePaymentChange(index, e.target.value)
              }
            >
              <option value="">Select</option>
              <option>Cash</option>
              <option>Cheque</option>
              <option>UPI</option>
              <option>GPay</option>
              <option>PhonePe</option>
            </select>

            <button
              className="pay-btn"
              onClick={() => handlePayClick(index)}
            >
              Pay
            </button>
          </div>
        )}
      </td>
    </tr>
  )}
</tbody>

```



```

        </button>
      </div>
    )}
  </td>

  </tr>
  )})
</tbody>
</table>
{showPopup && (
<div className="popup-overlay">
  <div className="popup-box">
    <h3>Payment Successful ☐ </h3>
    <button onClick={() => setShowPopup(false)}>OK</button>
  </div>
</div>
)}
</div>
);
};

```

```

export default Billing;

```

/* PAGE */

**.billing-page {
padding: 30px;
}**

/* BLUE HEADER BAR */

**.billing-header {
background-color: #2563eb;
color: white;
padding: 18px 25px;
border-radius: 8px;**

**display: flex;
justify-content: space-between;
align-items: center;**

**margin-bottom: 25px;
}**

/* HEADING */

**.billing-header h2 {
margin: 0;
color: white;
font-weight: 600;
}**

**.billing-table {
width: 100%;
border-collapse: collapse;
background: white;**

```
    box-shadow: 0 4px 10px rgba(0,0,0,0.1);  
}
```

```
.billing-table th {  
    background: #2563eb;  
    color: white;  
    padding: 12px;  
    text-align: left;  
}
```

```
.billing-table td {  
    padding: 12px;  
    border-bottom: 1px solid #ddd;  
}
```

```
.billing-table select {  
    padding: 6px;  
    border-radius: 5px;  
    border: 1px solid #ccc;  
}
```

```
.selected-payment {  
    font-weight: bold;  
    color: green;  
}
```

```
/* BACK BUTTON */
```

```
.back-btn {  
    background: white;  
    color: #2563eb;  
    border: none;  
    padding: 6px 16px;
```

```
border-radius: 6px;  
cursor: pointer;  
font-weight: 500;  
}
```

```
.back-btn:hover {  
  background: #f1f5ff;  
}
```

```
.payment-action {  
  display: flex;  
  gap: 10px;  
  align-items: center;  
}
```

```
.pay-btn {  
  background: #2563eb;  
  color: white;  
  border: none;  
  padding: 6px 14px;  
  border-radius: 6px;  
  cursor: pointer;  
}
```

```
.pay-btn:hover {  
  background: #1e4ed8;  
}
```

```
/* Popup Overlay */
```

```
.popup-overlay {  
  position: fixed;  
  top: 0;
```

```
left: 0;
width: 100%;
height: 100%;
background: rgba(0,0,0,0.4);
display: flex;
justify-content: center;
align-items: center;
```

```
}
```

```
/* Popup Box */
```

```
.popup-box {
  background: white;
  padding: 30px 40px;
  border-radius: 10px;
  text-align: center;
}
```

```
.popup-box button {
  margin-top: 15px;
  padding: 6px 18px;
  background: #2563eb;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
}
```

```
import React, { useState } from "react";
import { useNavigate } from "react-router-dom";
import "../ReceptionistProfile.css";

const ReceptionistProfile: React.FC = () => {
  const navigate = useNavigate();

  const [contact, setContact] = useState("");

  const handleContactChange = (e:
    React.ChangeEvent<HTMLInputElement>) => {
    const value = e.target.value;

    // Allow only numbers & max 10 digits
    if (/^\d{0,10}$/.test(value)) {
      setContact(value);
    }
  };

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();

    if (contact.length !== 10) {
      alert("Contact number must be exactly 10 digits");
      return;
    }

    alert("Profile Updated Successfully ☐");
  };
};
```

```
};
```

```
return (  
  <div className="receptionist-profile-page">
```

```
    {/* Back Button */}  
    <button className="back-btn" onClick={() => navigate(-1)}>  
      Back  
    </button>
```

```
    <h2>My Profile</h2>
```

```
    <form onSubmit={handleSubmit} className="profile-form">
```

```
      <label>Receptionist ID</label>  
      <input type="text" value="REC-1001" readOnly />
```

```
      <label>Username</label>  
      <input type="text" defaultValue="reception1" />
```

```
      <label>First Name</label>  
      <input type="text" />
```

```
      <label>Last Name</label>  
      <input type="text" />
```

```
<label>Email ID</label>
<input type="email" />
```

```
<label>Contact Number</label>
<input
  type="text"
  value={contact}
  onChange={handleContactChange}
  placeholder="Enter 10 digit number"
/>
```

```
<label>Address</label>
<textarea rows={3}></textarea>
```

```
<label>Current Password</label>
<input type="password" />
```

```
<label>New Password</label>
<input type="password" />
```

```
<label>Confirm New Password</label>
<input type="password" />
<button type="submit" className="save-btn">
  Save Changes
</button>
</form>
</div>
);
};
export default ReceptionistProfile;
```



```
.receptionist-profile-page {  
  min-height: 100vh;  
  background: #f4f7fb;  
  padding: 40px;  
}
```

```
.back-btn {  
  background: #2563eb;  
  color: white;  
  border: none;  
  padding: 8px 18px;  
  border-radius: 8px;  
  cursor: pointer;  
  margin-bottom: 15px;  
}
```

```
.back-btn:hover {  
  background: #1e40af;  
}
```

```
.receptionist-profile-page h2 {  
  text-align: center;  
  color: #2563eb;  
  margin-bottom: 25px;  
}
```

```
.profile-form {  
  max-width: 500px;  
  margin: auto;  
  background: white;  
  padding: 30px;
```

```
border-radius: 12px;  
box-shadow: 0px 4px 12px rgba(0,0,0,0.1);  
}
```

```
.profile-form label {  
  display: block;  
  margin-top: 15px;  
  font-weight: 600;  
}
```

```
.profile-form input,  
.profile-form textarea {  
  width: 100%;  
  padding: 10px;  
  margin-top: 5px;  
  border-radius: 8px;  
  border: 1px solid #ccc;  
}
```

```
.save-btn {  
  width: 100%;  
  margin-top: 25px;  
  padding: 12px;  
  background: #2563eb;  
  color: white;  
  border: none;  
  border-radius: 8px;  
  font-size: 16px;  
  cursor: pointer;  
}
```

```
.save-btn:hover {  
  background: #1e40af;}
```

Profile Card code:

```
import React, { useEffect, useState } from "react";
import "./ProfileDetails.css";
import axios from "axios";
interface Props {
  userImg: string;
}
interface Profile {
  user_id: string;
  first_name: string;
  last_name: string;
  role: string;
  email: string;
  contact:string;
  address:string;
  gender:string;
  dob:Date
}

const ProfileDetails: React.FC<Props> = ({userImg}) => {

  const [profile, setProfile] = useState<Profile>()

  const user = JSON.parse(localStorage.getItem("user") || "{}")

  useEffect(()=>{

    if(!profile){
```

```
axios.get(`http://localhost:5000/api/auth/getuserdetails/${user["user_id"]}`).then((response)=>{
    setProfile(response.data);
})
.catch((e)=>{
    console.log(e);
})
}
})
```

```
return (
<div className="profile-section">
  <div className="patient-profile-card">
    <img src={userImg} alt="User" className="profile-img" />
    <div className="profile-info">
      <h3>{profile?.first_name} {profile?.last_name}</h3>
      <p><strong>User_id:</strong> {profile?.user_id}</p>
      <p><strong>Email:</strong> {profile?.email}</p>
      <p><strong>Contact:</strong> {profile?.contact}</p>
      <p><strong>Address:</strong> {profile?.address}</p>
      <p><strong>Gender:</strong> {profile?.gender}</p>
      { /* <p><strong>DOB:</strong> {profile?.dob}</p> */ }
    </div>

  </div>
</div>

);
};
```

```
export default ProfileDetails;
```

```
.profile-section {  
  margin-top: 30px;  
  width: 100%;  
  padding: 0 20px; /* small left-right spacing */  
}
```

```
/* PROFILE CARD FULL WIDTH */  
.patient-profile-card {  
  width: 100%;  
  background: #f4f7fb;  
  border-radius: 16px;  
  padding: 25px;  
  display: flex;  
  align-items: center;  
  gap: 25px;  
  box-shadow: 0 4px 10px rgba(0,0,0,0.08);  
}
```

```
/* PROFILE IMAGE */  
.profile-img {  
  width: 100px;  
  height: 100px;  
  border-radius: 50%;  
  object-fit: cover;  
}
```

```
/* PROFILE INFO */  
.profile-info h3 {  
  color: #2563eb;  
  margin-bottom: 10px;  
}
```

Update Profile code:

```
import React, { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import "../UpdateProfile.css";
import axios from "axios";

const UpdateProfile: React.FC = () => {
  const navigate = useNavigate();
  const [contactNo, setContactNo] = useState("");
  const [dob, setDob] = useState("");
  const [newPassword, setNewPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [firstName, setFirstName] = useState("");
  const [lastName, setLastName] = useState("");
  const [address, setAddress] = useState("");
  const [gender, setGender] = useState("");

  const user = JSON.parse(localStorage.getItem("user") || "{}")

  const formatDateForInput = (dateString: string) => {
    const date = new Date(dateString);
    return date.toISOString().split("T")[0];
  };

  useEffect(() => {
    if(!firstName){
      getProfile()
    }
  });
};
```

```
const getProfile = () => {
```

```
  axios.get(`http://localhost:5000/api/auth/getuserdetails/${user["user_id"]}`).then(response=>{  
    setFirstName(response.data["first_name"]);  
    setLastName(response.data["last_name"]);  
    setContactNo(response.data["contact"]);  
    setAddress(response.data["address"]);  
    setGender(response.data["gender"]);  
    setDob(formatDateForInput(response.data["dob"]));  
  })  
  .catch((e)=>{  
    console.log(e);  
  })  
}
```

```
const handleSave = () => {
```

```
  // Contact validation
```

```
  if (contactNo.length !== 10) {  
    alert("Contact number must be exactly 10 digits");  
    return;  
}
```

```
// ☐ Check current password
```

```
if (newPassword || confirmPassword) {
```

```
  if (newPassword !== confirmPassword) {  
    alert("New password and confirm password do not match!");  
    return;  
  }  
}
```

```
axios.post("http://localhost:5000/api/registration/updateprofile",{
  userId : user["user_id"],
  firstName,
  lastName,
  contactNo,
  address,
  gender,
  dob,
  newPassword
}).then(response=>{
  if(response.data["response"]){
    // 1 Update localStorage manually
    const updatedUser = {
      ...user,
      first_name: firstName,
      last_name: lastName,
      contact: contactNo,
      address: address,
      gender: gender,
      dob: dob
    };

    localStorage.setItem("user", JSON.stringify(updatedUser));

    alert(response.data["message"])

    setNewPassword("")
    setConfirmPassword("")

    // 2 Go back to dashboard
```



```
navigate(-1);
  }
  else{
    alert("Something went wrong!!")
  }
}).catch()=>{
  alert("Something went wrong!!")
})
};
```

```
return (
  <div className="doctor-profile">
    <div className="page-header">
      <h2>Update Profile</h2>
      <button onClick={() => navigate(-1)}>Back</button>
    </div>

    <div className="profile-card">
      <label>First Name</label>
      <input
        type="text"
        value={firstName}
        onChange={(e) => setFirstName(e.target.value)}
      />
      <label>Last Name</label>
      <input
        type="text"
        value={lastName}
        onChange={(e) => setLastName(e.target.value)}
      />
      <label>Date of Birth</label>
      <input
        type="date"
```

```

placeholder="Date of Birth"
value={dob}
onChange={(e: React.ChangeEvent<HTMLInputElement>) =>
  setDob(e.target.value)
}
/>
<label>Gender</label>
<select
  className="form-input"
  value={gender}
  onChange={(e: React.ChangeEvent<HTMLSelectElement>) =>
    setGender(e.target.value)
  }
>
  <option value="">Select Gender</option>
  <option value="male">Male</option>
  <option value="female">Female</option>
  <option value="other">Other</option>
</select>
<label>Contact Number</label>
<input
  type="text"
  value={contactNo}
  onChange={(e) => {
    const value = e.target.value;

    // Allow only numbers
    if (/^\d*$/.test(value)) {
      setContactNo(value);
    }
  }}
  maxLength={10}
  placeholder="Enter 10-digit number"
/>

```

```
<hr style={{ margin: "20px 0" }} />
```

```
<label>Address</label>
```

```
<textarea
```

```
  value={address}
```

```
  onChange={(e) => setAddress(e.target.value)}
```

```
/>
```

```
<label>New Password</label>
```

```
<input
```

```
  type="password"
```

```
  value={newPassword}
```

```
  onChange={(e) => setNewPassword(e.target.value)}
```

```
/>
```

```
<label>Confirm New Password</label>
```

```
<input
```

```
  type="password"
```

```
  value={confirmPassword}
```

```
  onChange={(e) => setConfirmPassword(e.target.value)}
```

```
/>
```

```
<button onClick={handleSave}>Save Changes</button>
```

```
</div>
```

```
</div>
```

```
);
```

```
};
```

```
export default UpdateProfile;
```

```
.doctor-profile-page {  
  background: #f4f7fb;  
  min-height: 100vh;  
}  
* {  
  box-sizing: border-box;  
}
```

```
body {  
  overflow-x: hidden;  
}  
.profile-header {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  background: #2563eb;  
  padding: 15px 25px;  
  border-radius: 12px;  
  width: 100%;  
}
```

```
.profile-header h2 {  
  color: white;  
  margin: 0;  
  font-size: 22px;  
}  
.profile-card {  
  max-width: 450px;  
  margin: 30px auto;  
  background: white;  
  padding: 25px;  
  border-radius: 10px;  
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);  
}
```

```
}
```

```
.profile-card label {  
  font-weight: 600;  
  margin-bottom: 5px;  
  display: block;  
}  
.profile-card input {  
  width: 100%;  
  padding: 10px;  
  margin-bottom: 15px;  
  border-radius: 6px;  
  border: 1px solid #ccc;  
}
```

```
.profile-card button {  
  width: 100%;  
  padding: 10px;  
  background-color: #0b5ed7;  
  color: white;  
  border: none;  
  border-radius: 6px;  
  cursor: pointer;  
}
```

```
.profile-card button:hover {  
  background-color: #084298;  
}  
.profile-header button {  
  background-color: white;  
  color: #2563eb;  
  border: none;
```

```
padding: 6px 14px;
border-radius: 6px;
cursor: pointer;
font-weight: 500;
}
.profile-header button {
margin-left: auto;
}
```

```
.back-btn:hover {
background-color: #5a6268;
}
.profile-card textarea {
width: 100%;
padding: 10px;
border-radius: 6px;
border: 1px solid #ccc;
font-size: 14px;
resize: none; /* optional - prevents resizing */
height: 45px; /* makes height same as input */
box-sizing: border-box;
}
.form-input {
width: 100%;
padding: 12px;
border-radius: 10px;
border: 1px solid #ccc;
font-size: 15px;
box-sizing: border-box;
background: white;
}
select.form-input {
height: 45px;
}
```

Protected Route code:

```
import React from "react";
import { Navigate } from "react-router-dom";

interface ProtectedRouteProps {
  children: React.ReactElement;
  allowedRole?: string;
}

const ProtectedRoute: React.FC<ProtectedRouteProps> = ({
  children,
  allowedRole,
}) => {
  const isLoggedIn = localStorage.getItem("isLoggedIn");
  const userRole = localStorage.getItem("userRole");

  if (!isLoggedIn) {
    return <Navigate to="/login" replace />;
  }

  // □ ROLE-BASED RESTRICTION
  if (allowedRole && userRole !== allowedRole) {
    return <Navigate to="/login" replace />;
  }

  return children;
};
export default ProtectedRoute;
```

App.TSX code:

```
import React from "react";
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Homepage from "./components/Home page files/Homepage";
import Login from "./components/Login page file/Login";
import Register from "./components/Registration page files/Register";
import PatientDashboard from "./components/Patient's
files/PatientDashboard";
import DoctorDashboard from "./components/Doctor's
files/DoctorDashboard";
import AdminDashboard from "./components/Admin's
file/AdminDashboard";
import ReceptionistDashboard from "./components/Receptionist's
file/ReceptionistDashboard";
import ProtectedRoute from "./components/ProtectedRoute";

import BookAppointment from "./components/Patient's
files/BookAppointment";
import MyAppointments from "./components/Patient's
files/MyAppointments";
import Prescriptions from "./components/Prescription's file/Prescriptions";
import MyProfile from "./components/Patient's files/MyProfile";
import ScheduledAppointments from "./components/Doctor's
files/ScheduledAppointments";
import PatientRecords from "./components/Doctor's files/PatientRecords";
import ProvidePrescription from "./components/Doctor's
files/ProvidePrescription";
import DoctorsList from "./components/Admin's file/DoctorsList";
import ReceptionistList from "./components/Admin's file/ReceptionistList";
import AdminProfile from "./components/Admin's file/AdminProfile";
import ViewPrescription from "./components/Receptionist's
file/ViewPrescription";
import Billing from "./components/Receptionist's file/Billing";
```



```
import ReceptionistProfile from "../components/Receptionist's
file/ReceptionistProfile";
import MedicalRecords from "../components/Patient's files/MedicalRecords";
import UpdateProfile from "../components/Profile Card/UpdateProfile";
import Help from "../components/Home page files/Help";
import AboutUs from "../components/Home page files/AboutUs";
```

```
function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Homepage />} />
        <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Register />} />
      <Route path="/about" element={<AboutUs />} />
      <Route path="/help" element={<Help />} />
```

```
    <Route
      path="/patient-dashboard"
      element={
        <ProtectedRoute allowedRole="patient">
          <PatientDashboard />
        </ProtectedRoute>
      }
    />
```

```
    <Route
      path="/book-appointment"
      element={
        <ProtectedRoute>
          <BookAppointment />
        </ProtectedRoute>
      }
    />
```

```
    <Route
```

```
    path="/my-appointments"
    element={
      <ProtectedRoute>
        <MyAppointments />
      </ProtectedRoute>
    }
  />
<Route
  path="/Prescriptions"
  element={
    <ProtectedRoute allowedRole="patient">
      <Prescriptions />
    </ProtectedRoute>
  }
/>
<Route
  path="/medical-records"
  element={
    <ProtectedRoute allowedRole="patient">
      <MedicalRecords />
    </ProtectedRoute>
  }
/>
<Route
  path="/my-profile"
  element={
    <ProtectedRoute allowedRole="patient">
      <MyProfile />
    </ProtectedRoute>
  }
/>
<Route
  path="/doctor-dashboard"
  element={
    <ProtectedRoute allowedRole="doctor">
```

```
        <DoctorDashboard />
    </ProtectedRoute>
  }
/>
<Route
  path="/scheduled-appointments"
  element={
    <ProtectedRoute allowedRole="doctor">
      <ScheduledAppointments />
    </ProtectedRoute>
  }
/>
<Route
  path="/patient-records"
  element={
    <ProtectedRoute allowedRole="doctor">
      <PatientRecords />
    </ProtectedRoute>
  }
/>
<Route
  path="/provide-prescription"
  element={
    <ProtectedRoute allowedRole="doctor">
      <ProvidePrescription />
    </ProtectedRoute>
  }
/>
<Route
  path="/update-profile"
  element={
    <ProtectedRoute allowedRole="">
      <UpdateProfile />
    </ProtectedRoute>
  }
}
```

```
/>
<Route
  path="/admin-dashboard"
  element={
    <ProtectedRoute allowedRole="admin">
      <AdminDashboard />
    </ProtectedRoute>
  }
/>
<Route
  path="/doctors-list"
  element={
    <ProtectedRoute allowedRole="admin">
      <DoctorsList />
    </ProtectedRoute>
  }
/>
<Route
  path="/receptionist-list"
  element={
    <ProtectedRoute allowedRole="admin">
      <ReceptionistList />
    </ProtectedRoute>
  }
/>
<Route
  path="/admin-profile"
  element={
    <ProtectedRoute allowedRole="admin">
      <AdminProfile />
    </ProtectedRoute>
  }
/>
<Route
  path="/receptionist-dashboard"
```

```

    element={
      <ProtectedRoute allowedRole="receptionist">
        <ReceptionistDashboard />
      </ProtectedRoute>
    }
  />
<Route
  path="/View-Patients-Prescription"
  element={
    <ProtectedRoute allowedRole="receptionist">
      <ViewPrescription />
    </ProtectedRoute>
  }
/>
<Route
  path="/billing"
  element={
    <ProtectedRoute allowedRole="receptionist">
      <Billing />
    </ProtectedRoute>
  }
/>
<Route
  path="/receptionist-profile"
  element={
    <ProtectedRoute allowedRole="receptionist">
      <ReceptionistProfile />
    </ProtectedRoute>
  }
/>
</Routes>
</BrowserRouter>
);
}
export default App;

```

App.CSS code:

```
.auth-container {  
  min-height: 100vh;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  background: linear-gradient(135deg, #0f62fe, #2bb673);  
}
```

```
.auth-card {  
  background: #ffffff;  
  padding: 35px;  
  width: 360px;  
  border-radius: 14px;  
  box-shadow: 0 15px 40px rgba(0, 0, 0, 0.25);  
  text-align: center;  
}
```

```
.auth-card h2 {  
  margin-bottom: 25px;  
}
```

```
.auth-form {  
  display: flex;  
  flex-direction: column; /* □ THIS IS THE KEY */  
  gap: 15px;  
}
```

```
.auth-form select,  
.auth-form input {
```

```
padding: 12px;  
border-radius: 6px;  
border: 1px solid #ccc;  
font-size: 14px;  
}
```

```
.auth-form button {  
padding: 12px;  
background: #0f62fe;  
color: white;  
border: none;  
border-radius: 6px;  
font-weight: bold;  
cursor: pointer;  
}
```

```
.auth-form button:hover {  
background: #084cdf;  
}
```

```
.switch-text {  
margin-top: 18px;  
font-size: 14px;  
}
```

```
.switch-text span {  
color: #0f62fe;  
cursor: pointer;  
font-weight: bold;  
}
```

Main. TSX code:

```
import React from 'react'  
import ReactDOM from "react-dom/client";  
import './index.css'  
import App from './App.tsx'
```

```
const container = document.getElementById("root");
```

```
if (!container) throw new Error("Root container missing");  
const root = ReactDOM.createRoot(container);  
root.render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```


Index . CSS code:

```
body {  
  margin: 0;  
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto',  
  'Oxygen',  
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',  
  sans-serif;  
  -webkit-font-smoothing: antialiased;  
  -moz-osx-font-smoothing: grayscale;  
}
```

```
code {  
  font-family: source-code-pro, Menlo, Monaco, Consolas, 'Courier New',  
  monospace;  
}
```

Backend Code:

Routes code:

Appointment Routes code:

```
from flask import Blueprint, request, jsonify
import app.services.appointment_services as appointment_services

appointments_bp = Blueprint("appointments", __name__)

@appointments_bp.route("/bookappointment", methods=["POST"])
def book_appointment():

    data = request.json

    appointment = {
        "patientId" : data["patientId"],
        "doctorId" : data["doctorId"],
        "appointmentDate" : data["appointmentDate"],
        "appointmentTime" : data["appointmentTime"],
        "visitReason" : data["visitReason"]
    }

    result = appointment_services.book_appointment_service(appointment)

    return jsonify(result)
```

```
@appointments_bp.route("/getappointments/<patient_id>",methods=["GET  
"])
```

```
def get_appointments(patient_id):
```

```
    result = appointment_services.get_appointments_service(patient_id)
```

```
    return jsonify(result),200
```

```
@appointments_bp.route("/getappointmentsbydoctorid/<doctor_id>",method  
s=["GET"])
```

```
def get_appointments_by_doctor_id(doctor_id):
```

```
    result =  
    appointment_services.get_appointments_by_doctorid_service(doctor_id)
```

```
    return jsonify(result),200
```

```
@appointments_bp.route("/getpatientdatabyappointmentid/<appointment_id  
>",methods=["GET"])
```

```
def get_patient_data_by_appointment_id(appointment_id):
```

```
    result =  
    appointment_services.get_patient_data_by_appointment_id_service(appointm  
ent_id)
```

```
    return jsonify(result),200
```

Auth Routes code:

```
from flask import Blueprint, request, jsonify  
import app.services.auth_services as auth_services
```

```
auth_bp = Blueprint("auth", __name__)
```

```
@auth_bp.route("/login", methods=["POST"])
```

```
def login():
```

```
    data = request.json
```

```
    email = data["email"]
```

```
    password = data["password"]
```

```
    result = auth_services.login_user(email,password)
```

```
    return jsonify(result)
```

```
@auth_bp.route("/getuserdetails/<user_id>", methods=["GET"])
```

```
def get_user_details(user_id):
```

```
    result = auth_services.get_user_details(user_id)
```

```
    return jsonify(result)
```

Doctor Routes code:

```
from flask import Blueprint, jsonify  
import app.services.doctor_services as doctor_services  
  
doctor_bp = Blueprint("doctor_bp", __name__)  
  
@doctor_bp.route("/getdoctors", methods=["GET"])  
def get_doctors():  
    try:  
  
        doctors = doctor_services.get_doctors()  
        return jsonify(doctors), 200  
  
    except Exception as e:  
        return jsonify({"error": "error, something went wrong"}), 500
```

Medical Records code:

```
from flask import Blueprint, request, jsonify  
import app.services.medical_record_services as medical_record_services
```

```
medical_records_bp = Blueprint("medical_records_bp", __name__)
```

```
@medical_records_bp.route("/savemedicalrecord", methods=["POST"])  
def save_medical_record():
```

```
    result = medical_record_services.save_medical_record(request)
```

```
    if "error" in result:  
        return jsonify(result), 400
```

```
    return jsonify(result), 200
```

```
@medical_records_bp.route("/getmedicalrecordsbydoctorid/<doctor_id>",  
methods=["GET"])
```

```
def get_medical_records_by_doctor_id(doctor_id):
```

```
    result =  
    medical_record_services.get_medical_records_by_doctor_id(doctor_id)
```

```
return jsonify(result), 200
```

```
@medical_records_bp.route("/getmedicalrecordsbypatientid/<patient_id>",  
methods=["GET"])
```

```
def get_medical_records_by_patient_id(patient_id):
```

```
    result =
```

```
    medical_record_services.get_medical_records_by_patient_id(patient_id)
```

```
    return jsonify(result), 200
```

Prescription Route code:

```
from flask import Blueprint, request, jsonify
```

```
import app.services.prescription_services as prescription_services
```

```
prescriptions_bp = Blueprint("prescription", __name__)
```

```
@prescriptions_bp.route("/saveprescription", methods=["POST"])  
def save_prescription():
```

```
    data = request.json
```

```
    result = prescription_services.save_prescription(data)  
    return jsonify(result), 200
```

```
@prescriptions_bp.route("/getprescriptions/<patient_id>",  
methods=["GET"])  
def get_prescriptions(patient_id):
```

```
    result = prescription_services.get_prescriptions(patient_id)  
    return jsonify(result), 200
```

```
@prescriptions_bp.route("/getallprescriptions", methods=["GET"])  
def get_all_prescriptions():
```

```
    result = prescription_services.get_all_prescriptions()
```

```
    return jsonify(result), 200
```


Receptionist Route code:

```
from flask import Blueprint, jsonify  
import app.services.receptionist_services as receptionist_services  
  
receptionist_bp = Blueprint("receptionist_bp", __name__)  
  
@receptionist_bp.route("/getreceptionists", methods=["GET"])  
  
def get_receptionists():  
    try:  
  
        receptionists = receptionist_services.get_receptionist()  
        return jsonify(receptionists), 200  
  
    except Exception as e:  
        return jsonify({"error": "error, something went wrong!"}), 500
```

Registration Route code:

```
from flask import Blueprint, request, jsonify  
import app.services.registration_services as registration_services
```

```
registration_bp = Blueprint("registration", __name__)
```

```
@registration_bp.route("/register", methods=["POST"])
```

```
def register():
```

```
    data = request.json  
    role = data["role"]  
    email = data["email"]  
    password = data["password"]  
    firstName = data["firstName"]  
    lastName = data["lastName"]  
    gender = data["gender"]  
    dob = data["dob"]  
    contact = data["contact"]  
    address = data["address"]
```

```
user = {  
    "role":role,  
    "email":email,  
    "password":password,  
    "firstName":firstName,  
    "lastName":lastName,  
    "gender":gender,
```

```
        "dob":dob,  
        "contact":contact,  
        "address":address  
    }
```

```
    result = registration_services.register_user(user)
```

```
    return jsonify(result)
```

```
@registration_bp.route("/fetchdoctors", methods=["GET"])  
def fetchDoctors():
```

```
    result = registration_services.fetch_doctors()
```

```
    return jsonify(result)
```

```
@registration_bp.route("/updateprofile", methods=["POST"])  
def update_profile():
```

```
    user = request.json
```

```
    result = registration_services.update_profile(user)
```

```
    return jsonify(result)
```

AI Route code:

```
from flask import Blueprint, request, jsonify  
import app.services.ai_services as ai_services
```

```
ai_bp = Blueprint("ai_bp", __name__)
```

```
@ai_bp.route("/predictappointmentcategory", methods=["POST"])  
def predict_category():  
    data = request.get_json()  
    result = ai_services.predict_category(data)  
    return jsonify(result)
```

Services Codes:

Appointment Services code:

```
from app.utils.db import get_db_connection  
import uuid  
from app.services.ai_services import predict_category
```

```
def book_appointment_service(data):
```

```
    appointmentId = str(uuid.uuid4())
```

```
    predicted_category = predict_category({"symptoms":data["visitReason"]})
```

```
    patientId = data["patientId"]  
    doctorId = data["doctorId"]  
    appointmentDate = data["appointmentDate"]  
    appointmentTime = data["appointmentTime"]  
    appointmentType = predicted_category["category"]  
    visitReason = data["visitReason"]  
    appointmentStatus = "Booked"  
    consultationNotes = None
```

```

conn = get_db_connection()
if conn is None:
    return {"error": "Database connection failed"}

cursor = conn.cursor()
cursor.execute(
    """
    INSERT INTO appointments
    (appointment_id, patient_id, doctor_id,
    appointment_date, appointment_time, appointment_type,
    visit_reason, appointment_status, consultation_notes)
    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
    """,
    (
        appointmentId,
        patientId,
        doctorId,
        appointmentDate,
        appointmentTime,
        appointmentType,
        visitReason,
        appointmentStatus,
        consultationNotes
    )
)
conn.commit()
cursor.close()
conn.close()
return {
    "response": True,
    "appointmentId": appointmentId
}

```

```
def get_appointments_service(patient_id):

    conn = get_db_connection()
    if conn is None:
        return {"error": "Database connection failed"}

    cursor = conn.cursor()

    cursor.execute("""
        SELECT a.appointment_id, u.first_name,u.last_name,
               a.appointment_date, a.appointment_time,
               a.visit_reason, a.appointment_status
        FROM appointments a
        inner join users u on u.user_id=a.doctor_id
        WHERE patient_id = %s
        ORDER BY appointment_date DESC
    """, (patient_id,))

    appointments = cursor.fetchall()

    result = []

    for appt in appointments:
        result.append({
            "appointmentId": appt[0],
            "doctor": appt[1] + " " + appt[2],
            "date": appt[3],
            "time": appt[4],
```

```
        "reason": appt[5],  
        "status": appt[6]  
    })
```

```
cursor.close()  
conn.close()
```

```
return result
```

```
def get_appointments_by_doctorid_service(doctor_id):
```

```
    conn = get_db_connection()  
    if conn is None:  
        return {"error": "Database connection failed"}
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("""  
        SELECT a.appointment_id, u.first_name,u.last_name,  
               a.appointment_date, a.appointment_time,  
               a.visit_reason, a.appointment_status, a.appointment_type  
  
        FROM appointments a  
        inner join users u on u.user_id=a.patient_id  
        WHERE doctor_id = %s  
        ORDER BY appointment_date DESC  
    """, (doctor_id,))
```



```
appointments = cursor.fetchall()
```

```
result = []
```

```
for appt in appointments:  
    result.append({  
        "appointmentId": appt[0],  
        "patientName": appt[1] + " " + appt[2],  
        "date": appt[3],  
        "time": appt[4],  
        "reason": appt[5],  
        "status": appt[6],  
        "appointmentType": appt[7]  
    })
```

```
cursor.close()
```

```
conn.close()
```

```
return result
```

```
def get_patient_data_by_appointment_id_service(appointment_id):
```

```
    conn = get_db_connection()
```

```
if conn is None:
    return {"error": "Database connection failed"}

cursor = conn.cursor(dictionary=True)

cursor.execute("""
    SELECT concat(u.first_name, ' ', u.last_name) as name,a.patient_id
    FROM appointments a
    inner join users u on u.user_id=a.patient_id
    WHERE a.appointment_id = %s
    """, (appointment_id,))

result = cursor.fetchone()

cursor.close()
conn.close()

if result:
    return {"patientName": result["name"], "patientId":result["patient_id"]}
else:
    return {"message": "Appointment not found"}
```

Auth Services code;

```
from werkzeug.security import check_password_hash  
from app.utils.db import get_db_connection
```

```
def login_user(email,password):
```

```
    conn = get_db_connection()  
    if conn is None:  
        return {"error": "Database connection failed"}
```

```
    cursor = conn.cursor(dictionary=True)
```

```
    cursor.execute("SELECT user_id, first_name, last_name, password,role  
FROM users WHERE email=%s",(email,))  
    user = cursor.fetchone()
```

```
    if not user:  
        cursor.close()  
        conn.close()  
        return {"response": False, "message": "User not found"}
```

```
# Step 3: Check password
```

```
if not check_password_hash(user["password"], password):  
    cursor.close()  
    conn.close()  
    return {"response": False, "message": "Incorrect password"}
```

```
resp = {  
    "user_id":user["user_id"],  
    "first_name":user["first_name"],  
    "last_name":user["last_name"],  
    "role":user["role"],  
}
```

```
cursor.close()  
conn.close()
```

```
if user and check_password_hash(user["password"], password):  
    return {"response": True, "user":resp}  
else:  
    return {"response": False}
```

```
def get_user_details(user_id):
```

```
    conn = get_db_connection()  
    if conn is None:  
        return {"error": "Database connection failed"}
```

```
    cursor = conn.cursor(dictionary=True)
```

```
    cursor.execute("SELECT user_id, first_name, last_name,role  
,email,contact,address,gender,dob FROM users WHERE  
user_id=%s",(user_id,))
```

```
    user = cursor.fetchone()
```

```
resp = {  
    "user_id":user["user_id"],  
    "first_name":user["first_name"],  
    "last_name":user["last_name"],  
    "role":user["role"],  
    "email":user["email"],  
    "contact":user["contact"],  
    "address":user["address"],  
    "gender":user["gender"],  
    "dob":user["dob"],  
}
```

```
cursor.close()  
conn.close()
```

```
return resp
```

Doctor Services code:

```
from app.utils.db import get_db_connection
```

```
def get_doctors():
```

```
    conn = get_db_connection()
```

```
    cursor = conn.cursor(dictionary=True)
```

```
    query = """
```

```
        SELECT
```

```
            u.user_id,
```

```
            concat(u.first_name, ' ', u.last_name) as name,
```

```
            u.email,
```

```
            u.contact,
```

```
            d.specialization,
```

```
            d.shift_start,
```

```
            d.shift_end,
```

```
            d.availability_status
```

```
        FROM users u
```

```
        LEFT JOIN doctors_detail d ON u.user_id = d.doctor_id
```

```
        WHERE u.role = 'doctor'
```

```
    """
```

```
    cursor.execute(query)
```

```
    doctors = cursor.fetchall()
```

```
    cursor.close()
```

```
    conn.close()
```

```
return doctors
```

Medical Records Code:

```
from app.utils.db import get_db_connection  
import os  
from app.config import Config  
from werkzeug.utils import secure_filename
```

```
def save_medical_record(data):
```

```
    patient_id = data.form["patientId"]  
    title = data.form["title"]  
    description = data.form["description"]  
    comment = data.form["comment"]  
    file = data.files["file"]
```

```
    if not file:  
        return {"error": "No file uploaded"}
```

```
    if not os.path.exists(Config.UPLOAD_FOLDER):  
        os.makedirs(Config.UPLOAD_FOLDER)
```

```
    filename = secure_filename(file.filename)  
    filepath = os.path.join(Config.UPLOAD_FOLDER, filename)  
    file.save(filepath)
```

```
    conn = get_db_connection()  
    if conn is None:
```

```

        return {"error": "Database connection failed"}

    cursor = conn.cursor()
    cursor.execute (
        """
        INSERT INTO medical_records (patient_id, title, description, comment,
file_path,uploaded_at)
        VALUES (%s,%s,%s, %s,%s,now())
        """,
        (patient_id,title,description,comment,filepath)
    )
    conn.commit()
    cursor.close()
    conn.close()

    return {
        "response": True,
        "message": "Medical records saved successfully"}

def get_medical_records_by_doctor_id(doctor_id):

    conn = get_db_connection()
    if conn is None:
        return {"error": "Database connection failed"}

    cursor = conn.cursor()

    cursor.execute (
        """select md.*, u.first_name, u.last_name from medical_records md
inner join users u on u.user_id=md.patient_id
where patient_id in (select patient_id from appointments where
doctor_id=%s)

```



```
        order by md.uploaded_at desc""",(doctor_id,)
    )
    medical_records = cursor.fetchall()
```

```
result = []
```

```
for mr in medical_records:
    result.append(
        {
            "recordId":mr[0],
            "patientId":mr[1],
            "title":mr[2],
            "description":mr[3],
            "comment":mr[4],
            "fileURL":mr[5],
            "fileName":mr[5].split("\\")[1],
            "uploadedAt":mr[6],
            "patientName": mr[7] + " " + mr[8]
        }
    )
cursor.close()
conn.close()
```

```
return result
```

```
def get_medical_records_by_patient_id(patient_id):
```

```
    conn = get_db_connection()
    if conn is None:
```

```
        return {"error": "Database connection failed"}

    cursor = conn.cursor()

    cursor.execute (
        """select title,description,comment,file_path,uploaded_at from
medical_records
    where patient_id=%s order by uploaded_at desc""",(patient_id,)
    )
    medical_records = cursor.fetchall()

    result = []

    for mr in medical_records:
        result.append(
            {
                "title":mr[0],
                "description":mr[1],
                "comment":mr[2],
                "fileURL":mr[3],
                "fileName":mr[3].split("\\")[1],
                "uploadedAt":mr[4]
            }
        )
    cursor.close()
    conn.close()

    return result
```

Prescription Services code:

```
from app.utils.db import get_db_connection
```

```
def save_prescription(data):
```

```
    patientId = data["patientId"]  
    doctorId = data["doctorId"]  
    appointmentId = data["appointmentId"]  
    medicines = data["medicines"]  
    instructions = data["instructions"]
```

```
    conn = get_db_connection()  
    if conn is None:  
        return {"error": "Database connection failed"}
```

```
    cursor = conn.cursor()  
    cursor.execute(  
        """  
        INSERT INTO prescriptions  
        (appointment_id,patient_id,doctor_id,  
        medicines,instructions,date)  
        VALUES (%s, %s, %s, %s, %s, now())  
        """,  
        (  
            appointmentId,
```

```
        patientId,  
        doctorId,  
        medicines,  
        instructions  
    )  
)  
conn.commit()  
cursor.close()  
conn.close()  
return{  
    "response": True  
}
```

def get_prescriptions(patient_id):

```
    conn = get_db_connection()  
    if conn is None:  
        return {"error": "Database connection failed"}
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("""  
        SELECT p.prescription_id,  
               p.medicines,  
               p.instructions,  
               p.date,  
               concat(u.first_name, ' ', u.last_name) as doctor_name  
        FROM prescriptions p  
        JOIN users u ON p.doctor_id = u.user_id
```

```
WHERE p.patient_id = %s
ORDER BY p.date DESC
"", (patient_id,))
```

```
prescriptions = cursor.fetchall()
```

```
prescription_list = []
```

```
for p in prescriptions:
    prescription_list.append({
        "prescription_id": p[0],
        "medicines": p[1],
        "instructions": p[2],
        "date": p[3],
        "doctor_name": p[4]
    })
```

```
cursor.close()
conn.close()
return prescription_list
```

```
def get_all_prescriptions():
```

```
    conn = get_db_connection()
    if conn is None:
        return {"error": "Database connection failed"}
```

```
cursor = conn.cursor()
```

```
cursor.execute("""  
    select p.prescription_id, concat(pu.first_name, ' ', pu.last_name) as patien  
    concat(du.first_name, ' ', du.last_name) as doctor_name,  
    p.medicines, p.instructions, p.date  
    from prescriptions p  
    left join users pu on pu.user_id=p.patient_id  
    left join users du on du.user_id = p.doctor_id  
    order by p.date desc  
""")
```

```
prescriptions = cursor.fetchall()
```

```
prescription_list = []
```

```
for p in prescriptions:  
    prescription_list.append({  
        "prescription_id": p[0],  
        "patient_name": p[1],  
        "doctor_name": p[2],  
        "medicines": p[3],  
        "instructions":p[4],  
        "date": p[5]  
    })
```

```
cursor.close()  
conn.close()
```

```
return prescription_list
```

Receptionist Services code:

```
from app.utils.db import get_db_connection
```

```
def get_receptionist():
```

```
    conn = get_db_connection()
```

```
    cursor = conn.cursor(dictionary=True)
```

```
    query = """
```

```
        SELECT
```

```
            u.user_id,
```

```
            concat(u.first_name, ' ', u.last_name) as name,
```

```
            u.email,
```

```
            u.contact,
```

```
            r.shift_start,
```

```
            r.shift_end
```

```
        FROM users u
```

```
        LEFT JOIN receptionists_detail r ON u.user_id = r.receptionist_id
```

```
        WHERE u.role = 'receptionist'
```

```
    """
```

```
    cursor.execute(query)
```

```
    receptionists = cursor.fetchall()
```

```
    cursor.close()
```

```
    conn.close()
```

```
    return receptionists
```

Registration Services code:

```
from werkzeug.security import generate_password_hash
from app.utils.db import get_db_connection
import uuid
```

```
def register_user(user):
```

```
    conn = get_db_connection()
    if conn is None:
        return {"error": "Database connection failed"}

    cursor = conn.cursor()
    cursor.execute (
        """
        INSERT INTO users (user_id,role, first_name, last_name,gender,dob,
email,contact,address,password)
        VALUES (%s,%s,%s,%s,%s, %s, %s,%s,%s,%s)
        """,
        (str(uuid.uuid4()),user["role"], user["firstName"],
user["lastName"],user["gender"],user["dob"],user["email"],user["contact"],
user["address"],generate_password_hash(user["password"]))
        conn.commit()
        cursor.close()
        conn.close()

    return {
        "response": True,
        "message": "User registered successfully"}
```



```
def fetch_doctors():
    conn = get_db_connection()
    if conn is None:
        return {"error": "Database connection failed"}

    cursor = conn.cursor()
    cursor.execute(
        "SELECT user_id, first_name, last_name FROM users WHERE
role='doctor' order by first_name"
    )

    doctors = cursor.fetchall()

    doctor_list = []

    for doctor in doctors:
        doctor_list.append({
            "user_id": doctor[0],
            "first_name": doctor[1],
            "last_name": doctor[2]
        })

    cursor.close()
    conn.close()
    return doctor_list


def update_profile(user):

    conn = get_db_connection()
    if conn is None:
```

```
        return {"error": "Database connection failed"}

    cursor = conn.cursor()
    cursor.execute (
        """
        UPDATE users set first_name=%s, last_name=%s,
contact=%s,address=%s,gender=%s,dob=%s
        WHERE user_id=%s
        """,
        (user["firstName"], user["lastName"],user["contactNo"],
user["address"],user["gender"],user["dob"],user["userId"])
    )

    if user["newPassword"] :
        cursor.execute (
            """
            UPDATE users set password=%s
            WHERE user_id=%s
            """,
            (generate_password_hash(user["newPassword"]), user["userId"])
        )

    conn.commit()
    cursor.close()
    conn.close()

    return {
        "response": True,
        "message": "Profile updated successfully"}
```

AI Services code:

```
import pickle
```

```
def predict_category(data):
```

```
    model = pickle.load(open("appointment_model.pkl", "rb"))  
    vectorizer = pickle.load(open("vectorizer.pkl", "rb"))
```

```
    if not data or "symptoms" not in data:  
        return {"error": "Symptoms required"}
```

```
    symptoms = data["symptoms"]
```

```
    transformed = vectorizer.transform([symptoms])  
    prediction = model.predict(transformed)[0]  
    confidence = round(max(model.predict_proba(transformed)[0]) * 100, 2)
```

```
    return {  
        "category": prediction,  
        "confidence": confidence  
    }
```

Utils

Database connection code:

Db.py code:

```
import mysql.connector
from mysql.connector import Error
from app.config import Config

def get_db_connection():
    """Establish and return a MySQL database connection."""
    try:
        conn = mysql.connector.connect(
            host=Config.MYSQL_HOST,
            user=Config.MYSQL_USER,
            password=Config.MYSQL_PASSWORD,
            database=Config.MYSQL_DB,
            port=Config.MYSQL_PORT
        )
        if conn.is_connected():
            return conn
    except Error as e:
        print(f"Database connection failed: {e}")
        return None
```

```

def create_schema():
    try:
        conn = get_db_connection()
        if conn is None:
            return {"error": "Database connection failed"}

        cursor = conn.cursor()
        cursor.execute (
            """
            CREATE TABLE IF NOT EXISTS `users` (
                `user_id` varchar(50) NOT NULL,
                `role` varchar(50) NOT NULL,
                `first_name` varchar(100) NOT NULL,
                `last_name` varchar(100) NOT NULL,
                `gender` varchar(25) NOT NULL,
                `dob` date DEFAULT NULL,
                `email` varchar(150) NOT NULL,
                `contact` varchar(15) DEFAULT NULL,
                `address` text,
                `password` varchar(255) NOT NULL,
                PRIMARY KEY (`user_id`),
                UNIQUE KEY `email` (`email`)
            )
            """
        )
        cursor.execute (
            """
            CREATE TABLE IF NOT EXISTS `appointments` (
                `appointment_id` varchar(50) NOT NULL,
                `patient_id` varchar(50) NOT NULL,
                `doctor_id` varchar(50) NOT NULL,
                `appointment_date` date NOT NULL,
                `appointment_time` varchar(50) NOT NULL,
                `appointment_type` varchar(50) DEFAULT NULL,
            )
            """
        )

```

```

        `visit_reason` text,
        `appointment_status` varchar(50) NOT NULL,
        `consultation_notes` text,
        PRIMARY KEY (`appointment_id`)
    )
    """
)
cursor.execute (
    """
CREATE TABLE IF NOT EXISTS `prescriptions` (
    `prescription_id` int NOT NULL AUTO_INCREMENT,
    `appointment_id` varchar(50) NOT NULL,
    `patient_id` varchar(50) NOT NULL,
    `doctor_id` varchar(50) NOT NULL,
    `medicines` text,
    `instructions` text,
    `date` datetime DEFAULT NULL,
    PRIMARY KEY (`prescription_id`)
)
    """
)
cursor.execute (
    """
CREATE TABLE IF NOT EXISTS `medical_records` (
    `record_id` int NOT NULL AUTO_INCREMENT,
    `patient_id` varchar(50) NOT NULL,
    `title` varchar(250) NOT NULL,
    `description` text,
    `comment` text,
    `file_path` varchar(250) DEFAULT NULL,
    `uploaded_at` timestamp NULL DEFAULT NULL,
    PRIMARY KEY (`record_id`)
)
    """
)

```

```

cursor.execute (
    """
    CREATE TABLE IF NOT EXISTS doctors_detail (
        doctor_id VARCHAR(50) PRIMARY KEY,
        availability_status varchar(50)NOT NULL,
        shift_start VARCHAR(50) NOT NULL,
        shift_end VARCHAR(50) NOT NULL,
        specialization VARCHAR(100) NOT NULL,
        salary DECIMAL(10,2) DEFAULT NULL
    )
    """
)
cursor.execute (
    """
    CREATE TABLE IF NOT EXISTS receptionists_detail (
        receptionist_id VARCHAR(50) PRIMARY KEY,
        shift_start VARCHAR(50) NOT NULL,
        shift_end VARCHAR(50) NOT NULL,
        salary DECIMAL(10,2) DEFAULT NULL
    )
    """
)
conn.commit()
cursor.close()
conn.close()
print("Database created successfully!!")
except Error as e:
    print(f"Database creation failed: {e}")

finally:
    cursor.close()
    conn.close()

```

Helpers.py code:

```
import re  
from io import BytesIO  
from PyPDF2 import PdfReader
```

```
from app.utils.medical_summarizer_model import load_summarizer
```

```
from app.config import Config
```

```
summarizer, tokenizer, model = load_summarizer(Config.MODEL_DIR)
```

```
# Determine model input token limit (fallback to a safe default if "very large")  
try:
```

```
    MODEL_MAX_INPUT_TOKENS = int(getattr(tokenizer,  
"model_max_length", 1024))  
    if MODEL_MAX_INPUT_TOKENS > 100_000_000: # transformers uses  
big sentinel for "infinite"
```

```
        MODEL_MAX_INPUT_TOKENS = 1024  
except Exception:  
    MODEL_MAX_INPUT_TOKENS = 1024
```

```
# -----
```

```
# Helpers
```



```
# -----
```

```
_whitespace_re = re.compile(r"\s+")
```

```
def clean_text(text: str) -> str:
```

```
    # Normalize whitespace and strip
```

```
    return _whitespace_re.sub(" ", text).strip()
```

```
def tokenize_len(text: str) -> int:
```

```
    # Count tokens without truncation, excluding special tokens
```

```
    return len(tokenizer.encode(text, add_special_tokens=False))
```

```
def split_into_token_chunks(text: str, max_tokens: int, overlap_tokens: int = 64):
```

```
    """
```

```
    Splits text into chunks not exceeding max_tokens tokens, with overlaps  
between chunks.
```

```
    """
```

```
    # Tokenize upfront to avoid repeatedly encoding substrings
```

```
    input_ids = tokenizer.encode(text, add_special_tokens=False)
```

```
    if len(input_ids) <= max_tokens:
```

```
        yield text
```

```
    return
```

```
    step = max_tokens - overlap_tokens
```

```
    if step <= 0:
```

```
        step = max_tokens # fallback
```

```
    for start in range(0, len(input_ids), step):
```

```
        end = min(start + max_tokens, len(input_ids))
```

```
        chunk_ids = input_ids[start:end]
```

```
chunk_text = tokenizer.decode(chunk_ids, skip_special_tokens=True)
yield chunk_text
if end == len(input_ids):
    break
```

```
def summarize_text(
    text: str,
    max_input_tokens: int = None,
    summary_max_new_tokens: int = None,
    summary_min_new_tokens: int = None,
):
    """
    Hierarchical summarization:
    1) Chunk input to fit model context
    2) Summarize each chunk
    3) Concatenate chunk summaries and do a second-pass summary
    """
    text = clean_text(text)

    max_input_tokens = max_input_tokens or
min(Config.DEFAULT_MAX_INPUT_TOKENS,
MODEL_MAX_INPUT_TOKENS)
    summary_max_new_tokens = summary_max_new_tokens or
Config.DEFAULT_SUMMARY_MAX_NEW_TOKENS
    summary_min_new_tokens = summary_min_new_tokens or
Config.DEFAULT_SUMMARY_MIN_NEW_TOKENS

    token_count = tokenize_len(text)
    if token_count == 0:
        return ""
    # Step 1 & 2: Chunk + summarize per chunk
    per_chunk_summaries = []
```

```

for chunk in split_into_token_chunks(text, max_tokens=max_input_tokens,
    overlap_tokens=Config.CHUNK_
    out = summarizer(
        chunk,
        max_new_tokens=summary_max_new_tokens,
        min_new_tokens=summary_min_new_tokens,
        do_sample=False
    )
    per_chunk_summaries.append(out[0]["summary_text"].strip())
if len(per_chunk_summaries) == 1:
    return per_chunk_summaries[0]
# Step 3: Second-pass summary over concatenated chunk summaries
joined = " ".join(per_chunk_summaries)
# Keep second pass a bit tighter
second_pass = summarizer(
    joined,
    max_new_tokens=max(128, summary_max_new_tokens // 2),
    min_new_tokens=min(60, summary_min_new_tokens),
    do_sample=False
)
return second_pass[0]["summary_text"].strip()
def extract_text_from_pdf(pdf_bytes: bytes) -> str:
    """
    Extracts textual content from a PDF using PyPDF2.
    """
    reader = PdfReader(BytesIO(pdf_bytes))
    texts = []
    for page in reader.pages:
        try:
            txt = page.extract_text() or ""
        except Exception:
            txt = ""
        if txt:
            texts.append(txt)
    return "\n".join(texts)

```

INIT.Py code:

```
from flask import Flask  
from flask_cors import CORS  
from app.config import Config  
# from dotenv import load_dotenv
```

```
def create_app():
```

```
# load_dotenv()
```

```
app = Flask(__name__)  
app.config.from_object(Config)
```

```
CORS(app)
```

```
from app.routes.auth_routes import auth_bp  
from app.routes.registration_routes import registration_bp  
from app.routes.appointment_routes import appointments_bp  
from app.routes.prescriptions_routes import prescriptions_bp  
from app.routes.medical_records_routes import medical_records_bp  
from app.routes.doctor_routes import doctor_bp  
from app.routes.receptionist_routes import receptionist_bp  
from app.routes.ai_routes import ai_bp  
# from app.routes.medical_summarizer_routes import  
medical_summarizer_bp
```

```
from app.utils.db import create_schema

# Create database tables
create_schema()

# Register blueprints
app.register_blueprint(auth_bp, url_prefix="/api/auth")
app.register_blueprint(registration_bp, url_prefix="/api/registration")
app.register_blueprint(appointments_bp, url_prefix="/api/appointments")
app.register_blueprint(prescriptions_bp, url_prefix="/api/prescriptions")
app.register_blueprint(medical_records_bp,
url_prefix="/api/medicalrecords")
app.register_blueprint(doctor_bp, url_prefix="/api/doctors")
app.register_blueprint(receptionist_bp, url_prefix="/api/receptionists")
app.register_blueprint(ai_bp, url_prefix="/api/ai")
# app.register_blueprint(medical_summarizer_bp,
url_prefix="/api/medicalsummarizer")

return app
```

INT.Py code:

```
from flask import Flask  
from flask_cors import CORS  
from .config import Config  
from .routes.auth_routes import auth_bp  
from .routes.registration_routes import registration_bp  
  
def create_app():  
    app = Flask(__name__)  
    app.config.from_object(Config)  
    CORS(app)  
    app.register_blueprint(auth_bp, url_prefix="/api/auth")  
    app.register_blueprint(registration_bp, url_prefix="/api/registration")  
    return app
```

Config.py

```
import os
```

```
class Config:
```

```
    # MySQL Database Configuration
```

```
    MYSQL_HOST = os.getenv("MYSQL_HOST", "localhost")
```

```
    MYSQL_USER = os.getenv("MYSQL_USER", "root")
```

```
    MYSQL_PASSWORD = os.getenv("MYSQL_PASSWORD", "root")
```

```
    MYSQL_DB = os.getenv("MYSQL_DB", "hospital_management_system")
```

```
    MYSQL_PORT = int(os.getenv("MYSQL_PORT", 3306))
```

```
    UPLOAD_FOLDER = "files"
```

```
    # MAX_CONTENT_LENGTH = int(os.getenv("MAX_CONTENT_MB",  
    "20")) * 1024 * 1024 # MB
```

```
    # BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
    # MODEL_DIR = os.getenv("MODEL_DIR", os.path.join(BASE_DIR,  
    "models", "medical_summarization"))
```

```
    # CHUNK_OVERLAP_TOKENS =
```

```
    int(os.getenv("CHUNK_OVERLAP_TOKENS", "64"))
```

```
    # DEFAULT_MAX_INPUT_TOKENS =
```

```
    int(os.getenv("DEFAULT_MAX_INPUT_TOKENS", "1500")) # soft cap
```

```
    # DEFAULT_SUMMARY_MAX_NEW_TOKENS =
```

```
    int(os.getenv("DEFAULT_SUMMARY_MAX_NEW_TOKENS", "256"))
```

```
    # DEFAULT_SUMMARY_MIN_NEW_TOKENS =
```

```
    int(os.getenv("DEFAULT_SUMMARY_MIN_NEW_TOKENS", "50"))
```

```
    ## Enforce offline if environment indicates
```

```
    # TRANSFORMERS_OFFLINE =
```

```
    os.getenv("TRANSFORMERS_OFFLINE", "0") == "1"
```

```
    # HF_HUB_OFFLINE = os.getenv("HF_HUB_OFFLINE", "0") == "1"
```

Dataset

symptoms,category

severe chest pain and breathing difficulty,Emergency

unconscious after road accident,Emergency

heavy bleeding from injury,Emergency

sudden stroke symptoms and numbness,Emergency

heart attack signs and sweating,Emergency

severe abdominal pain with vomiting blood,Emergency

difficulty breathing and chest tightness,Emergency

high fever with seizures,Emergency

head injury after fall,Emergency

severe allergic reaction with swelling,Emergency

annual health checkup,Routine

blood pressure monitoring visit,Routine

diabetes follow up appointment,Routine

pregnancy regular checkup,Routine

thyroid routine testing,Routine

cholesterol test appointment,Routine

post surgery follow up,Routine

vaccination appointment,Routine

regular pediatric checkup,Routine

routine eye examination,Routine

mild fever and cough,Normal

common cold and sore throat,Normal

slight headache since morning,Normal

body pain and weakness,Normal

minor skin rash and itching,Normal

stomach pain after eating,Normal

mild back pain,Normal

seasonal flu symptoms,Normal

mild allergy symptoms,Normal

light dizziness occasionally,Normal

Server.py

```
from app import create_app
```

```
app = create_app()
```

```
if __name__ == "__main__":  
    app.run(debug=True)
```

Train Model.py code:

```
import pandas as pd  
import pickle
```

```
from sklearn.model_selection import train_test_split  
from sklearn.feature_extraction.text import TfidfVectorizer  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import accuracy_score
```

```
# Load dataset  
data = pd.read_csv("dataset/appointment_category.csv")
```

```
X = data["symptoms"]  
y = data["category"]
```

```
# Convert text to numeric features  
vectorizer = TfidfVectorizer(stop_words="english")
```

```
X_vectorized = vectorizer.fit_transform(X)
```

```
# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_vectorized, y, test_size=0.2, random_state=42  
)
```

```
# Train model
```

```
model = LogisticRegression()  
model.fit(X_train, y_train)
```

```
# Test accuracy
```

```
y_pred = model.predict(X_test)  
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Model Accuracy:", accuracy)
```

```
# Save model
```

```
pickle.dump(model, open("appointment_model.pkl", "wb"))  
pickle.dump(vectorizer, open("vectorizer.pkl", "wb"))
```

Database Query:

```
CREATE DATABASE IF NOT EXIST hospital_management_system;  
use hospital_management_system;
```

```
CREATE TABLE IF NOT EXISTS users (  
    user_id VARCHAR(50) NOT NULL,  
    role VARCHAR(50) NOT NULL,  
    first_name VARCHAR(100) NOT NULL,  
    last_name VARCHAR(100) NOT NULL,  
    gender VARCHAR(25) NOT NULL,  
    dob DATE DEFAULT NULL,  
    email VARCHAR(150) NOT NULL,  
    contact VARCHAR(15) DEFAULT NULL,  
    address TEXT,  
    password VARCHAR(255) NOT NULL,  
    PRIMARY KEY (user_id),  
    UNIQUE KEY email (email)  
);
```

```
CREATE TABLE IF NOT EXISTS appointments (  
    appointment_id VARCHAR(50) NOT NULL,  
    patient_id VARCHAR(50) NOT NULL,  
    doctor_id VARCHAR(50) NOT NULL,  
    appointment_date DATE NOT NULL,  
    appointment_time VARCHAR(50) NOT NULL,  
    appointment_type VARCHAR(50) DEFAULT NULL,  
    visit_reason TEXT,  
    appointment_status VARCHAR(50) NOT NULL,  
    consultation_notes TEXT,  
    PRIMARY KEY (appointment_id)  
);
```

```
CREATE TABLE IF NOT EXISTS prescriptions (  
    prescription_id INT NOT NULL AUTO_INCREMENT,  
    appointment_id VARCHAR(50) NOT NULL,  
    patient_id VARCHAR(50) NOT NULL,  
    doctor_id VARCHAR(50) NOT NULL,  
    medicines TEXT,  
    instructions TEXT,  
    date DATETIME DEFAULT NULL,  
    PRIMARY KEY (prescription_id)  
);
```

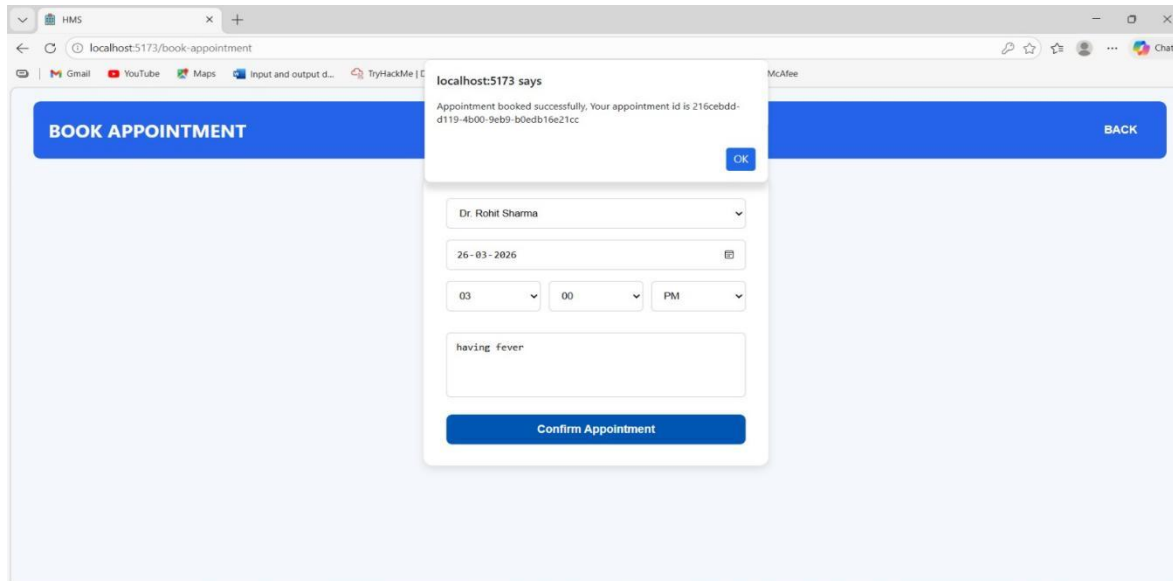
```
CREATE TABLE IF NOT EXISTS medical_records (  
    record_id INT NOT NULL AUTO_INCREMENT,  
    patient_id VARCHAR(50) NOT NULL,  
    title VARCHAR(250) NOT NULL,  
    description TEXT,  
    comment TEXT,  
    file_path VARCHAR(250) DEFAULT NULL,  
    uploaded_at TIMESTAMP NULL DEFAULT NULL,  
    PRIMARY KEY (record_id)  
);
```

```
CREATE TABLE IF NOT EXISTS doctors_detail (  
    doctor_id VARCHAR(50) PRIMARY KEY,  
    availability_status VARCHAR(50) NOT NULL,  
    shift_start VARCHAR(50) NOT NULL,  
    shift_end VARCHAR(50) NOT NULL,  
    specialization VARCHAR(100) NOT NULL,  
    salary DECIMAL(10,2) DEFAULT NULL  
);
```

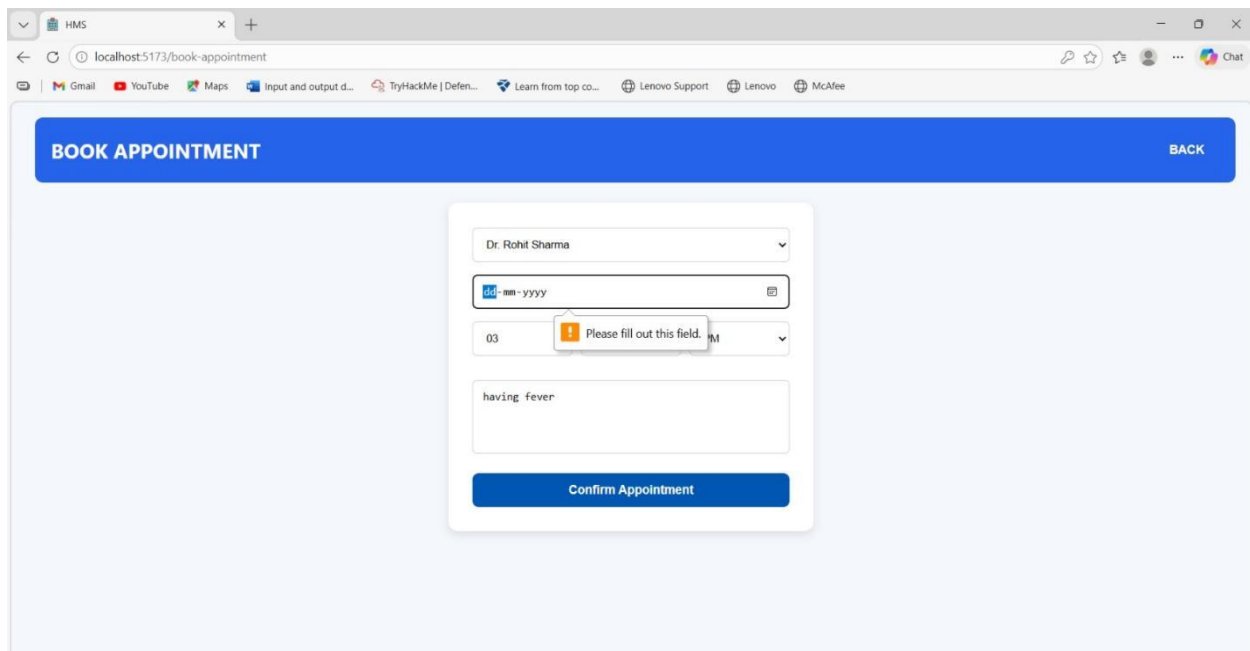
```
CREATE TABLE IF NOT EXISTS receptionists_detail (  
    receptionist_id VARCHAR(50) PRIMARY KEY,  
    shift_start VARCHAR(50) NOT NULL,  
    shift_end VARCHAR(50) NOT NULL,  
    salary DECIMAL(10,2) DEFAULT NULL  
);
```

6.2 Validation Screenshots

1. Appointment Booked Successfully :-



2. Date Validation Check :-



3. Prescription Details Validation :-

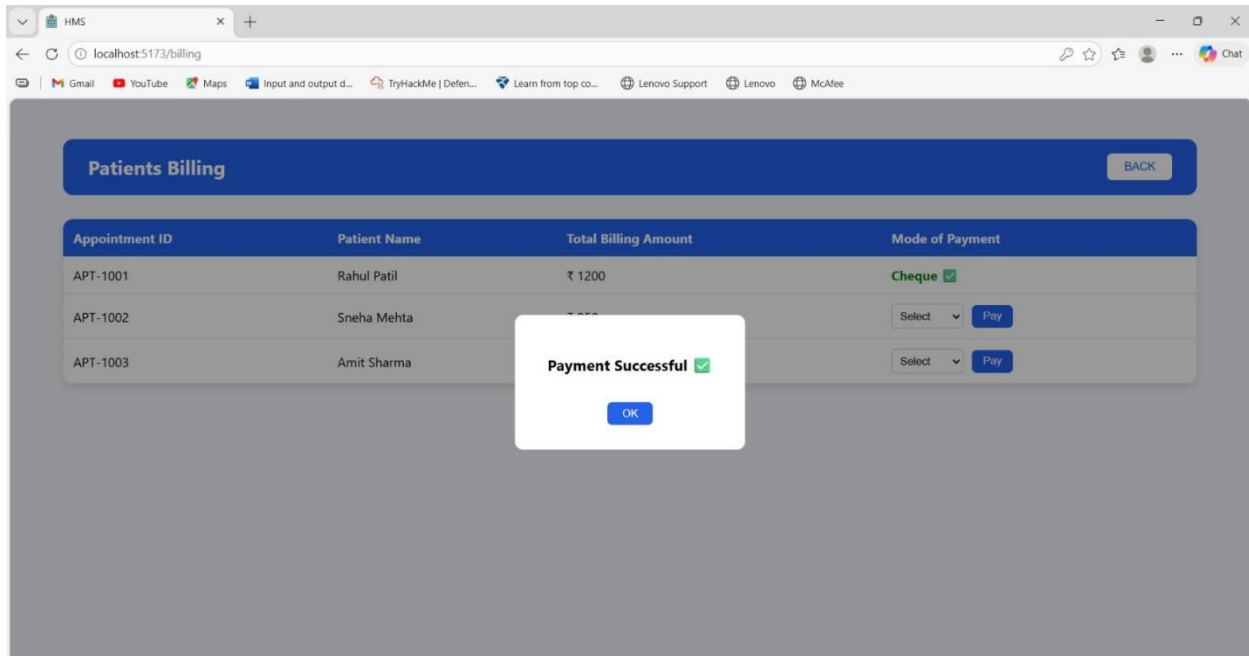
The screenshot shows a web browser window with the URL `localhost:5173/provide-prescription`. The page has a blue header with the text "PROVIDE PRESCRIPTION" and a "BACK" button. A modal dialog box is open in the center, titled "localhost:5173 says" with the message "Please fill all fields". The modal contains four input fields: a text field with the value "14b67b64-7fcd-4941-8513-26bc41eda168", a text field with the value "Karan Sawant", a text field with the value "Medicines", and a text field with the value "Instructions". Below these fields is a blue button labeled "Save Prescription".

4. Medical Records Updated Successfully :-

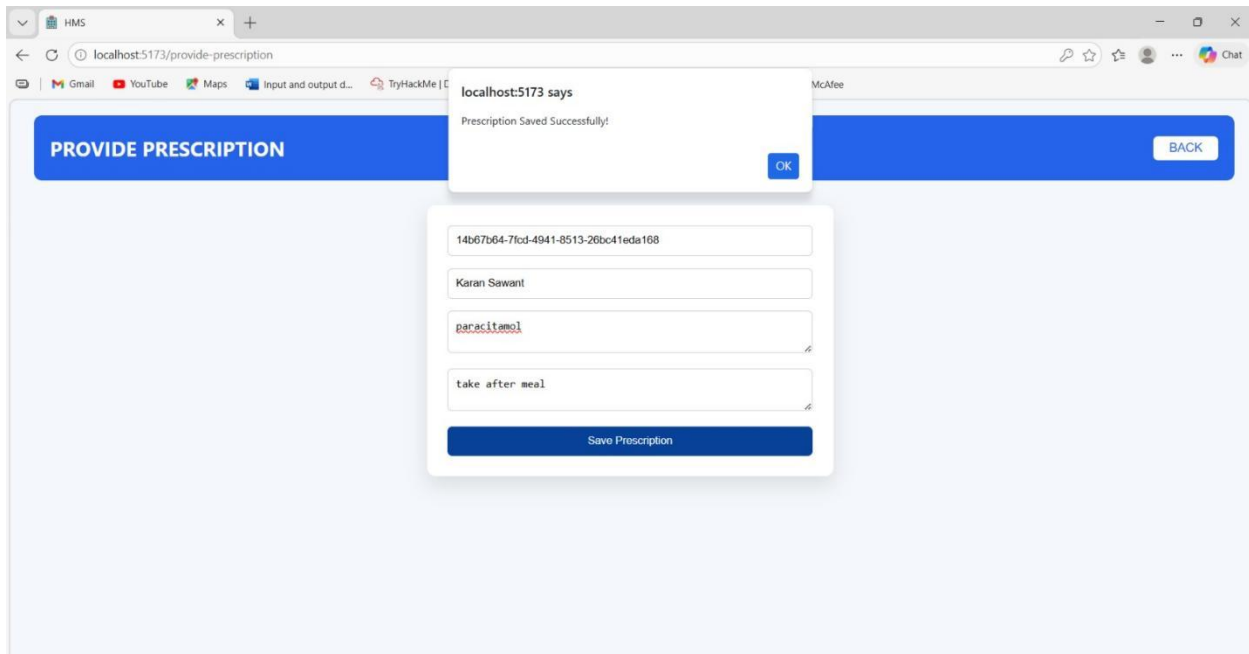
The screenshot shows a web browser window with the URL `localhost:5173/medical-records`. The page has a blue header with the text "MEDICAL RECORDS" and a "BACK" button. A modal dialog box is open in the center, titled "localhost:5173 says" with the message "Medical Record Uploaded Successfully!". The modal contains three input fields: a text field with the value "heart cancer", a text field with the value "first stage", and a text field with the value "Add Comment". Below these fields is a blue button labeled "Upload Record".

Below the modal, the page displays the text "Your Uploaded Records" and "No records uploaded yet".

5. Payment Successful :-



6. Prescription Saved Successfully :-



7. Scheduled Appointments :-

The screenshot shows a web browser window with the URL `localhost:5173/scheduled-appointments`. The page has a blue header with the text "SCHEDULED APPOINTMENTS" and a "Back" button. Below the header, there are two appointment cards. Each card displays the following information:

- Appointment ID:** 14b67b64-7fcd-4941-8513-26bc41eda168 (for the first card) and 21d3276a-2469-4000-b402-8c5ceca57ab4 (for the second card)
- Patient Name:** Karan Sawant
- Date:** 19-03-2026
- Time:** 06:04 PM
- Reason:** haiving milde fever
- Status:** Booked
- Type:** Emergency

Below the information in each card is a blue button labeled "Provide Prescription".

8. Uploaded Records :-

The screenshot shows a web browser window with the URL `localhost:5173/medical-records`. The page has a white background. At the top, there is a text input field. Below it, the heading "Upload Medical Record" is displayed. Under the heading, there is a "Choose File" button and a text input field containing the filename "seo assignments.pdf". Below this is a large blue button labeled "Upload Record".

Below the upload section, the heading "Your Uploaded Records" is displayed. Under this heading, there is a card containing the following information:

- Title:** heart cancer
- Description:** first stage
- Comment:**
- Date:** Thu Mar 05 2026
- File Name:** seo_assignments.pdf

At the bottom of the card is a blue button labeled "Download".

9. Prescription Prescribed Successfully :-

The screenshot shows a web browser window with the URL `localhost:5173/View-Patients-Prescription`. The page title is "Patient Prescriptions". A green notification banner at the top right displays the message "Prescription Prescribed Successfully" with a checkmark icon. Below the banner is a table with the following data:

Patient Name	Doctor	Medicines	Notes	Actions
Rahul Sharma	Dr. Mehta	Paracetamol, Vitamin C	Take after meals	<button>Prescribed</button>
Sneha Patil	Dr. Patil	Amoxicillin	Twice daily for 5 days	<button>Prescribed</button>
Amit Joshi	Dr. Sharma	Cough Syrup	Before sleep	<button>Prescribed</button>

7. System Testing:

System Testing – Definition (for your Hospital Management System project):

System Testing is the process of testing the complete Hospital Management System as a whole to ensure that all modules work together correctly and meet the specified requirements. It is performed after integration testing to verify that the entire system functions properly in a real-world environment.

In this project, system testing checks whether different modules such as Admin, Doctor, Receptionist, Patient Registration, Appointment Management, and Billing interact correctly with each other. The main goal is to ensure that the system performs all operations smoothly, without errors, and provides the expected results to the users.

1. Black Box Testing

Black Box Testing is a software testing method in which the tester checks the functionality of the system without knowing the internal code or program structure. The tester only provides input to the system and verifies whether the output is correct according to the requirements.

In this testing method, the internal working of the application is not considered. The main focus is on testing the system's features, user interface, and overall functionality.

Example (Hospital Management System):

In the Hospital Management System, black box testing can be performed by checking whether a receptionist can successfully register a patient, whether an appointment can be booked, or whether the billing amount is calculated correctly. The tester only checks the input and output without looking at the program code.

2. White Box Testing

White Box Testing is a software testing method in which the tester has knowledge of the internal structure, code, and logic of the application. In this testing, the internal working of the program such as loops, conditions, and code paths are tested. The main objective of white box testing is to ensure that all the internal operations of the system work correctly and that there are no logical errors in the program.

Example (Hospital Management System):

In the Hospital Management System, white box testing may involve checking the program code that handles patient registration, verifying the logic used for appointment scheduling, or testing the code responsible for billing calculations to ensure that it executes correctly.

7.1 Quality Assurance:

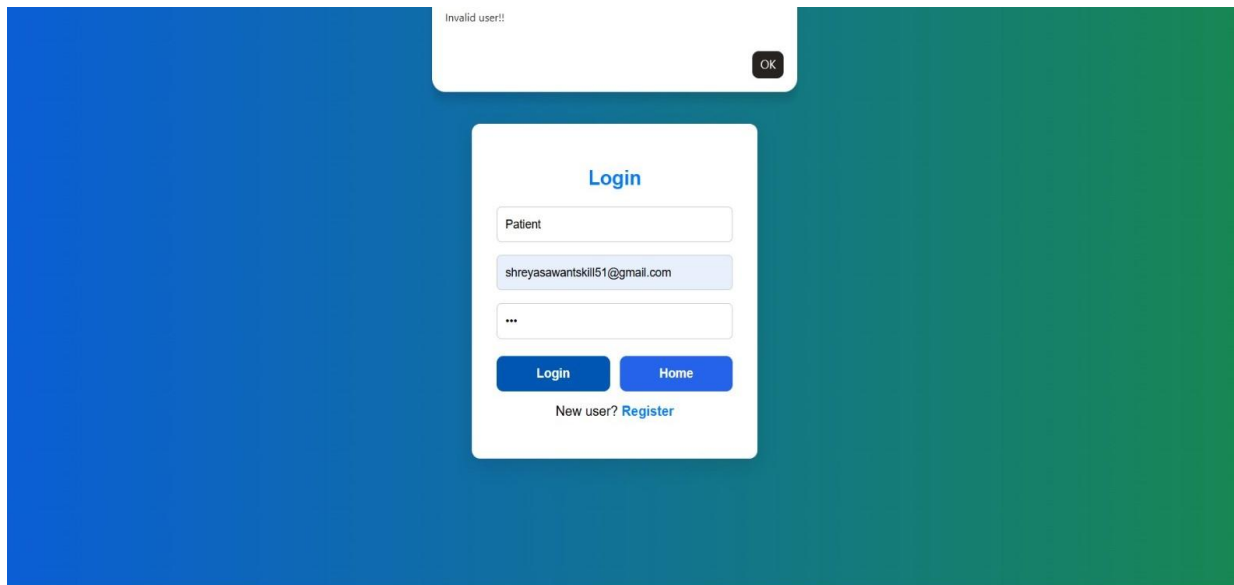
Quality Assurance (QA) is the process of ensuring that a software system meets the required standards of quality, reliability, and performance. It involves checking whether the developed system works correctly according to the specified requirements and is free from major errors or defects. The main objective of quality assurance is to improve the overall quality of the software by identifying and fixing issues during the development process.

In our **Hospital Management AI System** project, quality assurance was used to verify that all modules of the system function properly and provide accurate results. The Quality Assurance role was responsible for testing the different features of the system such as **patient registration, doctor management, appointment scheduling, billing, and user login.**

During the development process, each module was tested to ensure that the inputs, outputs, and system responses were correct. If any errors or bugs were found, they were reported and corrected to improve the system's performance. Testing methods like **black box testing and system testing** were used to check whether the application behaves as expected from the user's perspective.

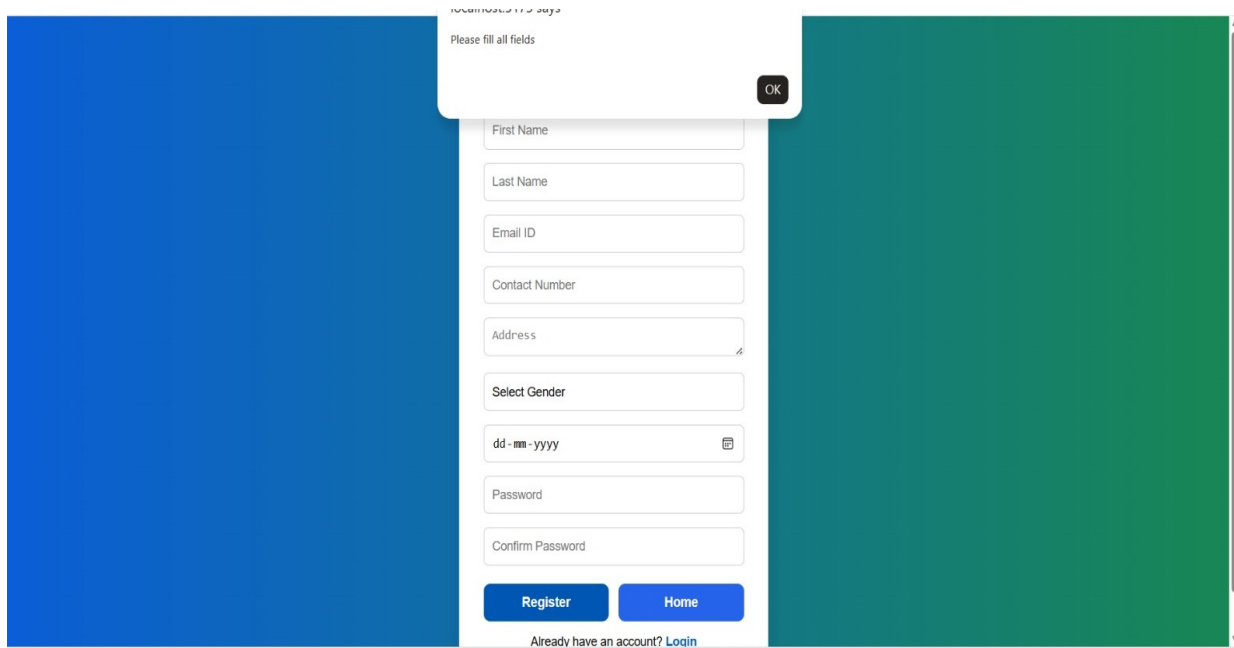
7.2 Screenshots of Testing:

1.Login page:-



The screenshot displays a login page with a blue and green gradient background. A white modal is centered on the screen. At the top of the modal, the title "Login" is displayed in blue. Below the title, there are three input fields: "Patient" (containing the text "shreyasawantkill51@gmail.com"), a password field (masked with "***"), and a "Login" button. To the right of the "Login" button is a "Home" button. Below the buttons, there is a link "New user? Register". An error message "Invalid user!!" is shown in a small white box at the top left of the modal, with an "OK" button next to it.

2.Register Page:-



The screenshot displays a registration page with a blue and green gradient background. A white modal is centered on the screen. At the top of the modal, the title "Register" is displayed in blue. Below the title, there are several input fields: "First Name", "Last Name", "Email ID", "Contact Number", "Address", "Select Gender", "dd-mm-yyyy" (date field), "Password", and "Confirm Password". At the bottom of the modal, there are two buttons: "Register" and "Home". Below the buttons, there is a link "Already have an account? Login". An error message "Please fill all fields" is shown in a small white box at the top left of the modal, with an "OK" button next to it.

3. Book Appointment :-

The screenshot shows a web browser window with a 'BOOK APPOINTMENT' form. The form has a blue header bar with the text 'BOOK APPOINTMENT' on the left and a 'BACK' button on the right. The form fields include a dropdown for 'Dr. Rohit Sharma', a date field for '26-03-2026', time dropdowns for '03' and '00', and a PM/AM selector. A text area contains 'having fever'. A blue 'Confirm Appointment' button is at the bottom. A modal window from 'localhost:5173' is open, displaying a success message: 'Appointment booked successfully, Your appointment id is 216cebddd119-4b00-9eb9-b0edb16e21cc'. The modal has an 'OK' button.

4. Medical Records :-

The screenshot shows a web browser window with a 'MEDICAL RECORDS' form. The form has a blue header bar with the text 'MEDICAL RECORDS' on the left and a 'BACK' button on the right. The form fields include text inputs for 'heart cancer' and 'first stage', a text area for 'Add Comment', and a file upload section with a 'Choose File' button and the filename 'seo assignments.pdf'. A blue 'Upload Record' button is at the bottom. A modal window from 'localhost:5173' is open, displaying a success message: 'Medical Record Uploaded Successfully!'. The modal has an 'OK' button.

5. Provide Prescription :-

The screenshot shows a web browser window with a 'Provide Prescription' form. A modal dialog box is displayed in the center, showing a success message: 'localhost:5173 says Prescription Saved Successfully!'. The form contains the following fields:

- Appointment ID: 14b67b64-7fcd-4941-8513-26bc41eda168
- Patient Name: Karan Sawant
- Medicine: paracetamol
- Instructions: take after meal

A 'Save Prescription' button is at the bottom of the form. A 'BACK' button is located in the top right corner of the form area.

6. Patients Billing :-

The screenshot shows a 'Patients Billing' table with a modal dialog box overlaying the bottom half. The table has the following data:

Appointment ID	Patient Name	Total Billing Amount	Mode of Payment
APT-1001	Rahul Patil	₹ 1200	Cheque ☒
APT-1002	Sneha Mehta	₹ 800	Select Pay
APT-1003	Amit Sharma	₹ 1000	Select Pay

The modal dialog box displays the message: 'Payment Successful' with a green checkmark and an 'OK' button.

8 Implementation:

8.1 User Guide:

The Hospital Management System is a web-based application designed to simplify hospital operations such as patient registration, appointment management, prescription handling, and coordination between different users. The system provides separate dashboards for Admin, Doctor, Patient, and Receptionist.

User Roles

1. Admin

The Admin monitors the system and manages overall information.

Functions of Admin:

View the number of registered doctors

View the number of registered receptionists

Access and update personal profile

2. Doctor

The Doctor module helps doctors manage patient treatment information.

Functions of Doctor:

View patient details

Check appointments

Provide prescriptions

Update profile information

3. Patient

The Patient module allows patients to interact with the hospital digitally.

Functions of Patient:

Book appointments

View prescriptions provided by doctors

Manage personal profile

4. Receptionist

The Receptionist assists in appointment management.

Functions of Receptionist:

Schedule appointments

Manage patient coordination

How to Use the System

Step 1: Registration

1. Open the registration page.
2. Select the user role (Admin, Doctor, Patient, or Receptionist).

3. Enter required details such as name, email, contact number, address, and password.

4. Submit the form to create an account.

Step 2: Login

1. Go to the login page.

2. Enter the username and password.

3. After successful authentication, the user will be redirected to the respective dashboard.

Step 3: Dashboard Access

After login, users can access features available according to their role.

Examples:

Doctor can manage prescriptions.

Patient can book appointments.

Admin can view system overview.

Receptionist can manage scheduling.

8.2 Installation Guide :

System Requirements

Hardware Requirements:

Computer or Laptop

Minimum 4 GB RAM

Internet connection

Software Requirements:

Node.js

Visual Studio Code

Web browser (Google Chrome recommended)

Installation Steps:-

Step 1: Install Node.js

Download and install Node.js from the official website: <https://nodejs.org>

Step 2: Open the Project

- 1. Open Visual Studio Code.**
- 2. Select File → Open Folder.**
- 3. Choose the project folder.**

Step 3: Install Dependencies

Open the terminal in VS Code and run the following command:

npm install

This will install all required project dependencies.

Step 4: Start the Application

Run the following command in the terminal:

npm start

Step 5: Open in Browser

After running the command, the application will open automatically in the browser.

If it does not open automatically, go to:

<http://localhost:5000>

System Access

Users can now register, log in, and access the features of the Hospital Management System according to their roles.

9. Conclusion:

Healthcare management in small clinics often faces challenges such as manual record keeping, appointment mismanagement, data redundancy, and delays in accessing patient information. Neural Med is designed as a digital solution to streamline clinic operations and improve overall efficiency.

This web-based Hospital Management System integrates essential modules such as patient registration, doctor management, appointment scheduling, medical record storage, and administrative control. The structured dashboard interface enables clinic staff to monitor daily activities efficiently, while secure login access ensures proper role-based management for administrators and doctors.

This project demonstrates how modern web technologies can be utilized to enhance healthcare services at the clinic level. By digitizing records and automating routine tasks, Neural Med reduces paperwork, minimizes human errors, and improves coordination between staff and patients. The system ensures quick data retrieval and organized information management, leading to better decision-making and improved patient care.

Through continuous development and feature enhancement, NuralMed has the potential to evolve into a comprehensive clinic management platform. This project represents a step toward digital transformation in healthcare, emphasizing the importance of technology in improving operational efficiency and delivering quality medical services.

10. Future Enhancement:

The current system provides essential functionalities for hospital management. However, several advanced modules and AI-based features can be implemented in the future to enhance the system's efficiency, scalability, and automation. The following improvements can be considered for future development:

1. Pharmacy Management Module:

A dedicated pharmacy module can be integrated into the system to manage medicines, prescriptions, and pharmacy billing. This module will allow pharmacists to track medicine stock, manage prescriptions from doctors, and ensure proper medicine distribution within the hospital.

2. Clinical Supply Management System:

For large hospitals, a clinical supply management system can be implemented to handle medical supplies such as surgical tools, equipment, and medicines. This module will help monitor inventory levels, track usage, and ensure timely restocking of essential hospital supplies.

3. AI-Based Supply Prediction:

Artificial Intelligence can be used to predict when medical supplies or medicines are likely to run out. By analyzing historical usage data, the system can automatically forecast demand and notify administrators to restock supplies in advance, reducing the risk of shortages.

4. Facility Management Module:

A facility management system can be added to manage hospital infrastructure such as rooms, beds, equipment maintenance, and sanitation schedules. This module will help hospital administrators efficiently manage hospital resources and improve operational efficiency.

5. AI-Based Appointment Summarization:

An AI feature can be introduced to automatically generate summaries of patient appointments. The system can analyze doctor-patient conversations or notes and produce a concise summary including diagnosis, prescribed medicines, and follow-up instructions.

6. AI Integration in Insurance Management:

Artificial Intelligence can also be integrated into the insurance management module to automate claim processing and verification. The system can analyze patient treatment details and insurance policies to speed up claim approval and reduce manual paperwork.

7. Smart Analytics and Decision Support:

Advanced analytics tools can be integrated to provide hospital administrators with insights into patient flow, treatment efficiency, and resource utilization. This can support better decision-making and improve overall healthcare service delivery.

11. References And Bibliography:

CoreUI. (n.d.). CoreUI – Open source admin dashboard template. Retrieved from <https://coreui.io>

eHospital. (n.d.). eHospital – Hospital Management Information System (HMIS). Retrieved from <https://ehospital.gov.in>

Medicare Frontend. (n.d.). Hospital Management System Website.

Retrieved from <https://medicare-frontend-kyt9.onrender.com/>

Panjwani, P. (2023). Hospital Management System using React | Node.js with Express | MySQL [Video]. YouTube.

<https://www.youtube.com/watch?v=nZSASdFL3EY>

Hospital Management System Project Tutorial [Video]. (n.d.). YouTube.

<https://youtu.be/hzHUMwGO0Pw>

Bibliography

The following sources were studied for conceptual understanding of healthcare workflow systems, dashboard layouts, and clinic-level management structures:

- Apollo Hospitals. (n.d.). *Apollo Hospitals Official Website*. Retrieved from <https://www.apollohospitals.com>
- Medanta. (n.d.). *Medanta – The Medicity*. Retrieved from <https://www.medanta.org>
- Online tutorials, blogs, and documentation related to React.js, Flask, MySQL integration, and dashboard-based healthcare web applications.

